STUDENT LABORATORY MANUAL

Module 4: Introduction to Computer Graphics and Visual Computing

20 Guided Examples • 20 Practice Exercises

Course: CS0045 – Computer Graphics and Visual Computing

Companion: Module 4 Student Guide

Toolchain: VS Code + MSYS2 UCRT64 GCC + FreeGLUT

How to Use This Manual

This manual has three parts:

• Part A — 20 worked Examples (Ex01–Ex20). Each is a complete, compiled, ready-to-run program demonstrating one concept at a time: the 5-step vertex array conversion, glDrawElements indexed rendering, color arrays, interleaved arrays, and a final composite scene. Type or copy each one into your SOURCE CODE\M4 folder, compile it (Ctrl+Shift+B), and run it (F5) before moving to the next.

- Part B — 20 Practice Exercises (Q01–Q20), split into Easy (Q01–Q07), Medium (Q08–Q14), and Hard (Q15–Q20). Each exercise gives you an Objective, a Task question describing exactly what your program must do, and Hints to point you in the right direction. Read the task and hints, then write the program YOURSELF before checking Part C.
- Part C — the full Answer Key for every exercise in Part B, with complete, compiled, working code.

Setup reminder

Use the shared SOURCE CODE\ VS Code workspace described in the Module 1 Student Guide. Compile with Ctrl+Shift+B ("Build OpenGL Program"), then run with F5 or from the terminal. All programs in this manual use the same cross-platform header block shown below.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;
```

Every program in this manual has been compiled and verified with g++ against FreeGLUT.

Part A — Guided Examples (Ex01–Ex20)

Example 01 — Immediate Mode Baseline (Before Conversion)

Concept: A plain glBegin()/glEnd() triangle — the starting point for Example 02's

conversion. **Expected output:** A solid green triangle, drawn the immediate-mode way.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

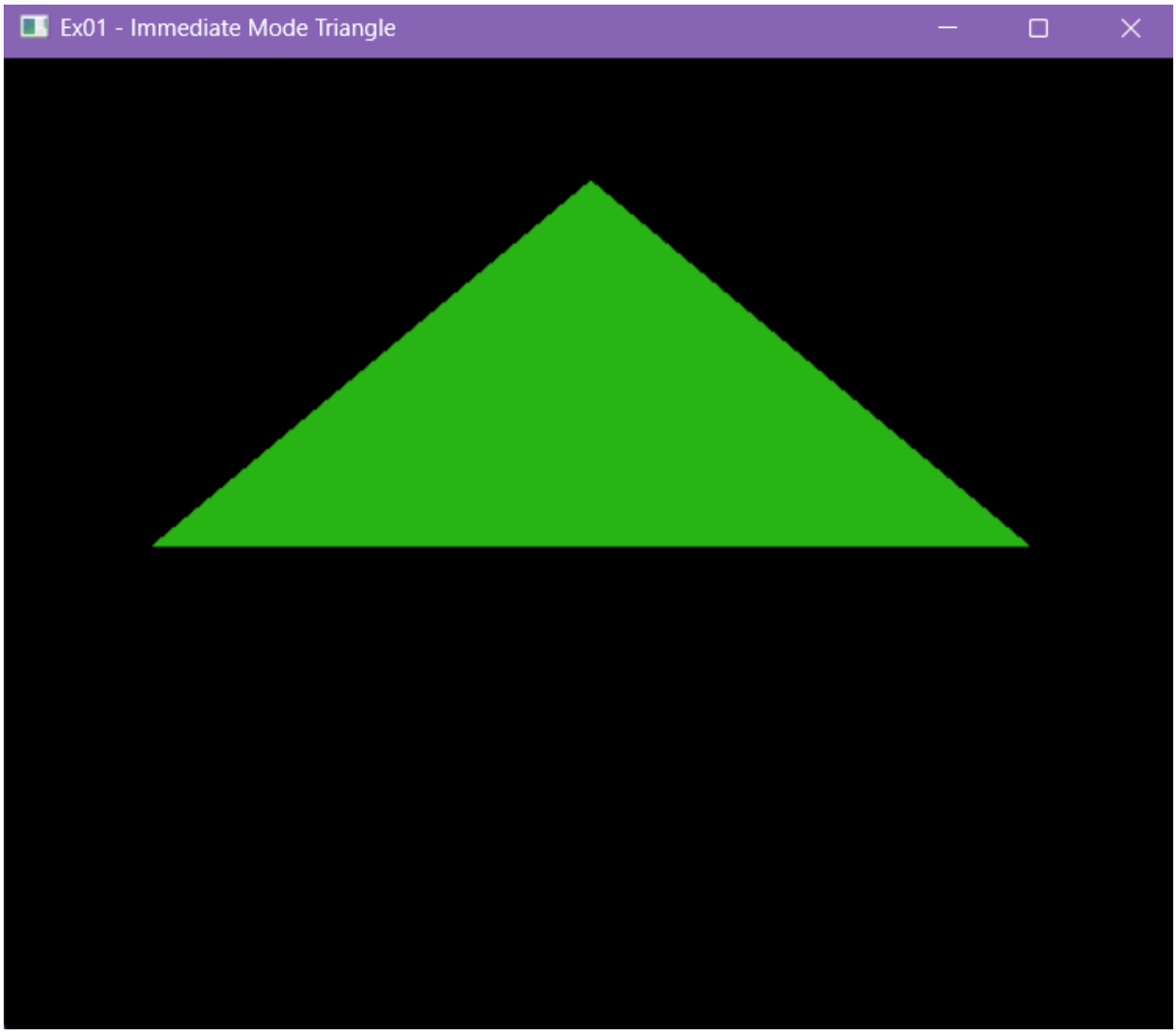
/*
 * Example 01 - Immediate Mode Baseline (Before Conversion)
 * ------------------------------------------------------------
 * Concept: A plain glBegin()/glEnd() triangle, drawn in "immediate
 mode" * (one glVertex2f call issued at a time). This is the STARTING
 POINT that * Example 02 converts into a vertex array, so you can
 compare both
 * approaches side by side.
 */

void triangle() {
 glBegin(GL_TRIANGLES);
 glVertex2f(0.0f, 0.75f);
 glVertex2f(-0.75f, 0.0f);
 glVertex2f(0.75f, 0.0f);
 glEnd();
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 glColor3f(0.16f, 0.72f, 0.08f);
 triangle();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex01 - Immediate Mode Triangle");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```

OUTPUT:



Example 02 — The Same Triangle, as a Vertex Array

Concept: The 5-step vertex array pattern: array -> glEnableClientState -> glVertexPointer -> glDrawArrays -> glDisableClientState.

Expected output: The identical green triangle from Example 01, now rendered from a vertex array.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 02 - Same Triangle, Converted to a Vertex Array
 * ------------------------------------------------------------
 * Concept: The 5-step vertex array pattern:
 * 1. Store the vertex coordinates in a plain array (GLfloat[]).
 * 2. glEnableClientState(GL_VERTEX_ARRAY) - tell OpenGL a vertex
 array * will be used.
 * 3. glVertexPointer(size, type, stride, pointer) - tell OpenGL
 WHERE * the data lives and how it's laid out.
 * 4. glDrawArrays(mode, first, count) - draw using the array data.
 * 5. glDisableClientState(GL_VERTEX_ARRAY) - turn the feature back
 off. */

void triangle() {
 // 1. initialize an array holding vertex coordinates
 GLfloat triangleVertex[] = {
 0.0f, 0.75f, 0.0f, // z is always 0.0 for 2D primitives
 -0.75f, 0.0f, 0.0f,
 0.75f, 0.0f, 0.0f
 };

 // 2. activate the vertex array feature
 glEnableClientState(GL_VERTEX_ARRAY);

 // 3. tell OpenGL where the vertex data is
 glVertexPointer(3, GL_FLOAT, 0, triangleVertex);

 // 4. draw using the array
 glDrawArrays(GL_TRIANGLES, 0, 3);

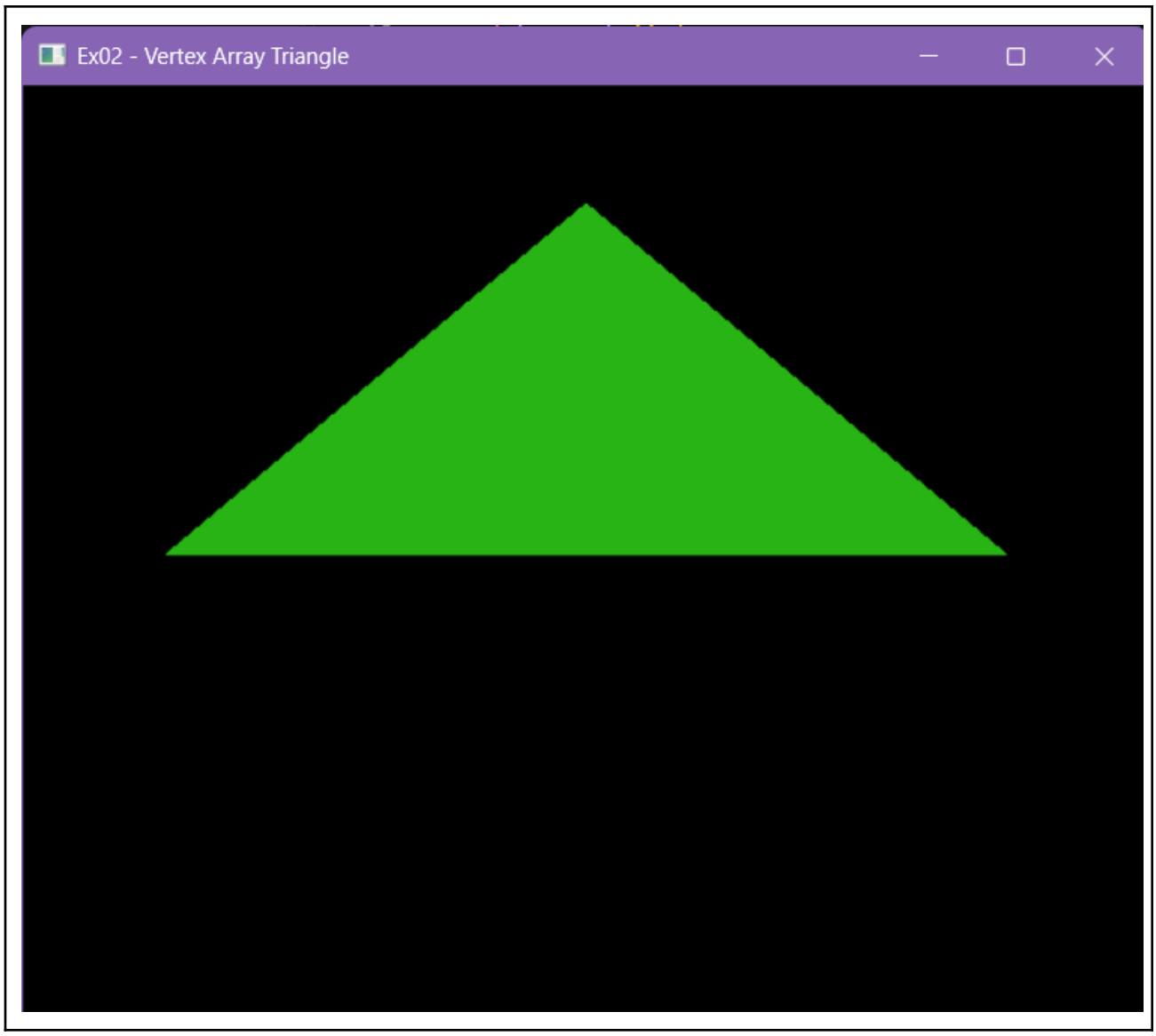
 // 5. deactivate the vertex array feature
 glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
```

```
 glClear(GL_COLOR_BUFFER_BIT);
 glColor3f(0.16f, 0.72f, 0.08f);
 triangle();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex02 - Vertex Array Triangle");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```

OUTPUT:



Example 03 — GL_POINTS From a Vertex Array

Concept: Applying the 5-step pattern to GL_POINTS.

Expected output: Four large white points, one at each corner of a square.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 03 - GL_POINTS Drawn From a Vertex Array
 * ------------------------------------------------------------
 * Concept: Applying the same 5-step vertex array pattern to GL_POINTS
 * instead of GL_TRIANGLES - only the array contents and glDrawArrays
 * mode change.
 */

void points() {
 glPointSize(20.0f);

 GLfloat pointVertex[] = {
 -0.75f, -0.75f, 0.0f,
 -0.75f, 0.75f, 0.0f,
 0.75f, 0.75f, 0.0f,
 0.75f, -0.75f, 0.0f
 };

 glEnableClientState(GL_VERTEX_ARRAY);
```

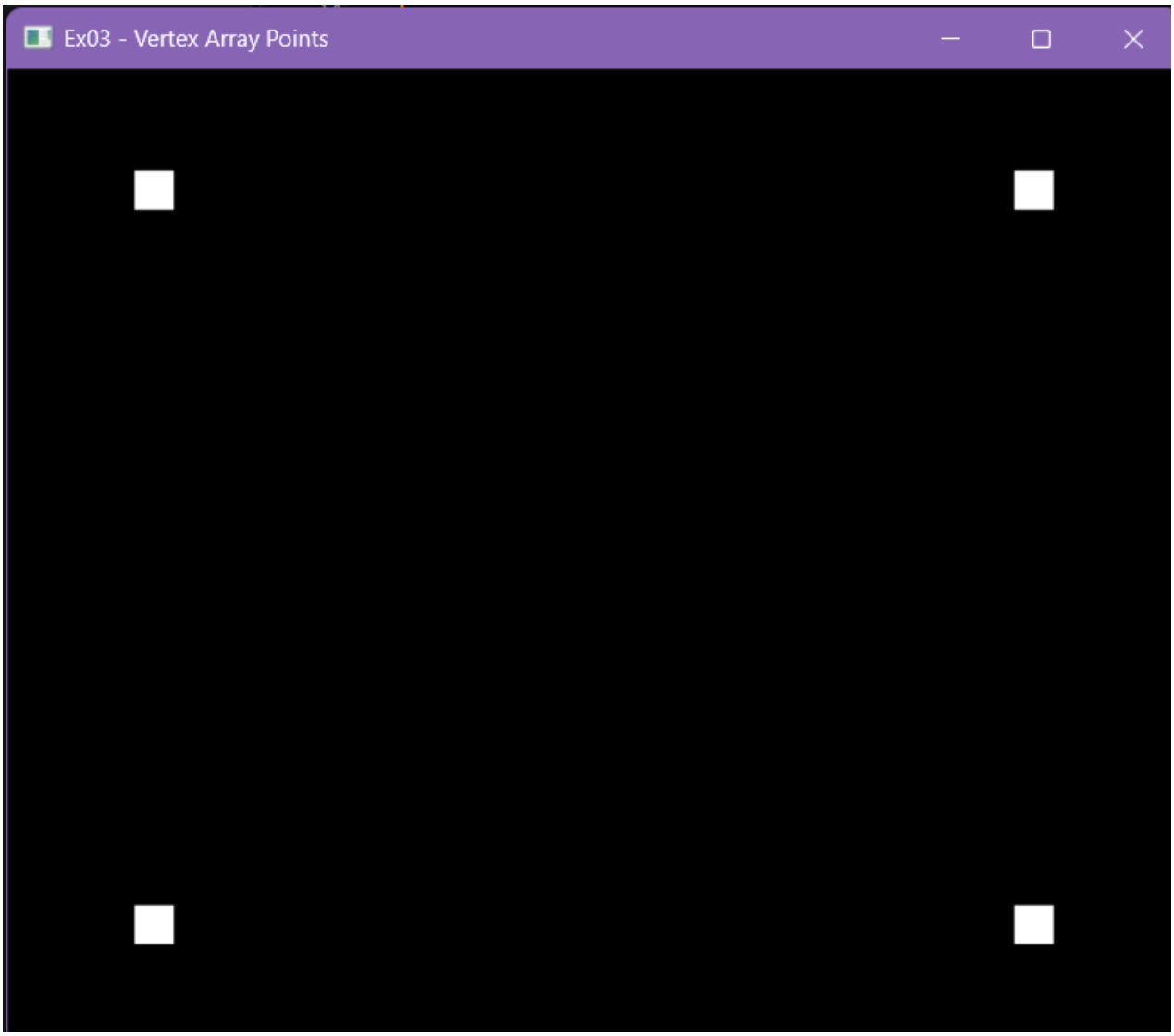


```
 glVertexPointer(3, GL_FLOAT, 0, pointVertex);
 glDrawArrays(GL_POINTS, 0, 4);
 glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 glColor3f(1.0f, 1.0f, 1.0f);
 points();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex03 - Vertex Array Points");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```

OUTPUT:



Example 04 — GL_LINES From a Vertex Array

Concept: A two-vertex array rendered with GL_LINES.

Expected output: A single thick horizontal yellow line.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 04 - GL_LINES Drawn From a Vertex Array
 * ------------------------------------------------------------
 * Concept: A two-vertex array rendered with GL_LINES via the vertex
 * array pipeline.
 */

void line() {
 GLfloat lineVertex[] = {
 -0.8f, 0.0f, 0.0f,
 0.8f, 0.0f, 0.0f
 };
```

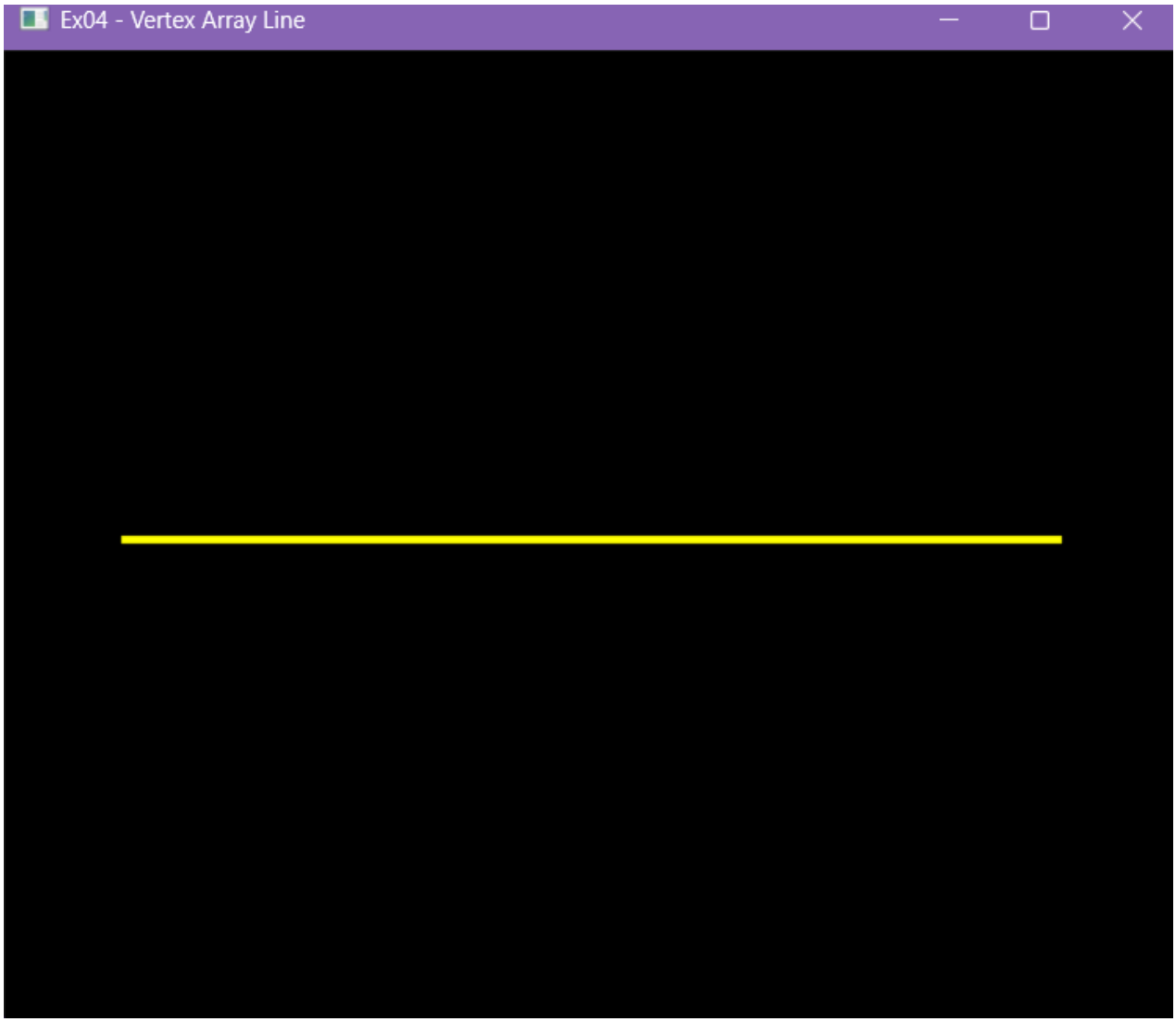


```
 glEnableClientState(GL_VERTEX_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, lineVertex);
 glLineWidth(4.0f);
 glDrawArrays(GL_LINES, 0, 2);
 glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 glColor3f(1.0f, 1.0f, 0.0f);
 line();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex04 - Vertex Array Line");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```

OUTPUT:



Example 05 — GL_QUADS From a Vertex Array

Concept: A 4-vertex array rendered with GL_QUADS.

Expected output: A solid blue filled square.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 05 - GL_QUADS Drawn From a Vertex Array
 * ------------------------------------------------------------
 * Concept: A filled square, built from a 4-vertex array and rendered
 * with GL_QUADS.
 */

void square() {
 GLfloat quadVertex[] = {
 -0.5f, -0.5f, 0.0f,
 0.5f, -0.5f, 0.0f,
```



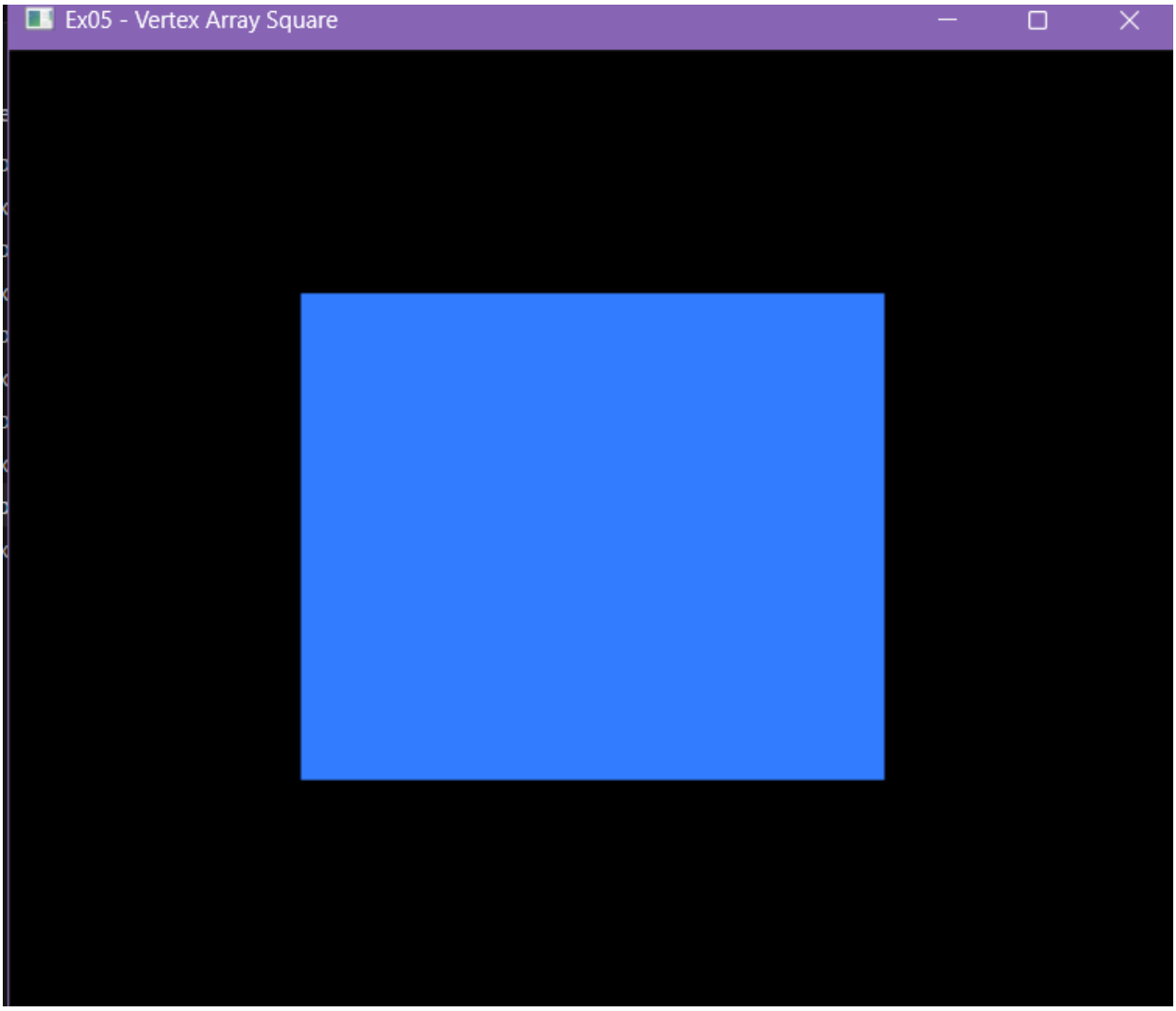
```
 0.5f, 0.5f, 0.0f,
 -0.5f, 0.5f, 0.0f
 };

 glEnableClientState(GL_VERTEX_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, quadVertex);
 glDrawArrays(GL_QUADS, 0, 4);
 glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 glColor3f(0.2f, 0.5f, 1.0f);
 square();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex05 - Vertex Array Square");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```

OUTPUT:



Example 06 — GL_POLYGON From a Vertex Array

Concept: A 5-vertex array rendered with GL_POLYGON.

Expected output: A solid orange filled pentagon.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 06 - GL_POLYGON Drawn From a Vertex Array
 * ------------------------------------------------------------
 * Concept: A five-sided convex polygon (pentagon), built from a
 vertex * array and rendered with GL_POLYGON, using ALL specified
 vertices.
 */

void pentagon() {
```



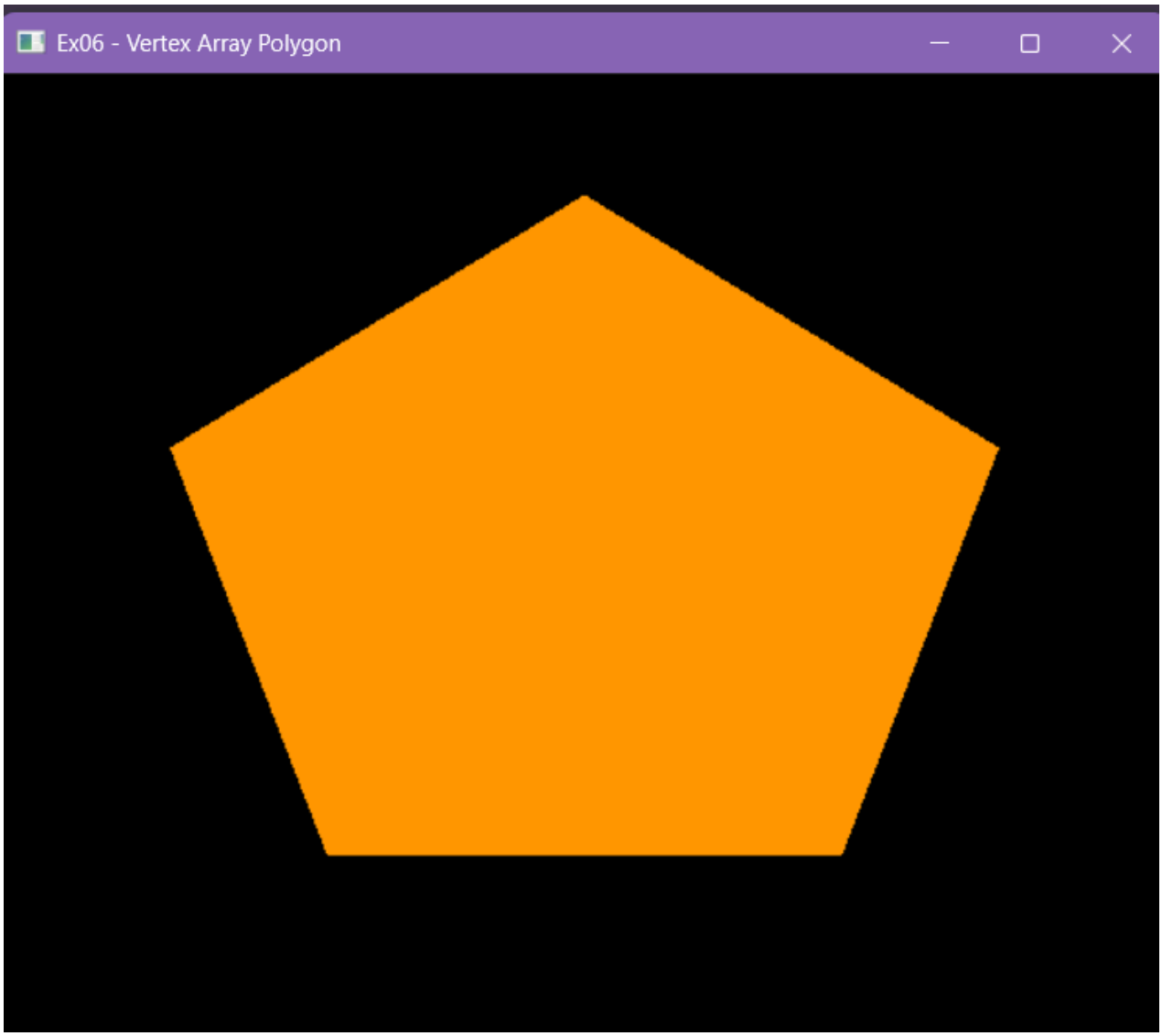
```
 GLfloat pentagonVertex[] = {
  0.0f, 0.75f, 0.0f,
  -0.71f, 0.23f, 0.0f,
  -0.44f, -0.61f, 0.0f,
  0.44f, -0.61f, 0.0f,
  0.71f, 0.23f, 0.0f
  };

  glEnableClientState(GL_VERTEX_ARRAY);
  glVertexPointer(3, GL_FLOAT, 0, pentagonVertex);
  glDrawArrays(GL_POLYGON, 0, 5);
  glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
  glClear(GL_COLOR_BUFFER_BIT);
  glColor3f(1.0f, 0.6f, 0.0f);
  pentagon();
  glFlush();
}

int main(int argc, char** argv) {
  glutInit(&argc, argv);
  glutInitWindowSize(600, 500);
  glutCreateWindow("Ex06 - Vertex Array Polygon");
  glutDisplayFunc(display);
  glutMainLoop();
  return 0;
}
```

OUTPUT:



Example 07 — Two Triangles, One Array, One Draw Call

Concept: Extending the array with more vertices lets one glDrawArrays render several separate primitives. **Expected output:** Two separate green triangles, one above the other, drawn from a single 6-vertex array.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 07 - Two Triangles From ONE Array, ONE Draw Call
 * ------------------------------------------------------------
 * Concept: Extending the vertex array with MORE vertices lets a single
```

```
 * glDrawArrays() call render multiple separate primitives at once -
 * here, two independent triangles (6 vertices, GL_TRIANGLES groups
 * them 3 at a time).
 */

void triangles() {
 GLfloat triangleVertex[] = {
 // triangle 1 (upper)
 0.0f, 0.75f, 0.0f,
 -0.75f, 0.15f, 0.0f,
 0.75f, 0.15f, 0.0f,

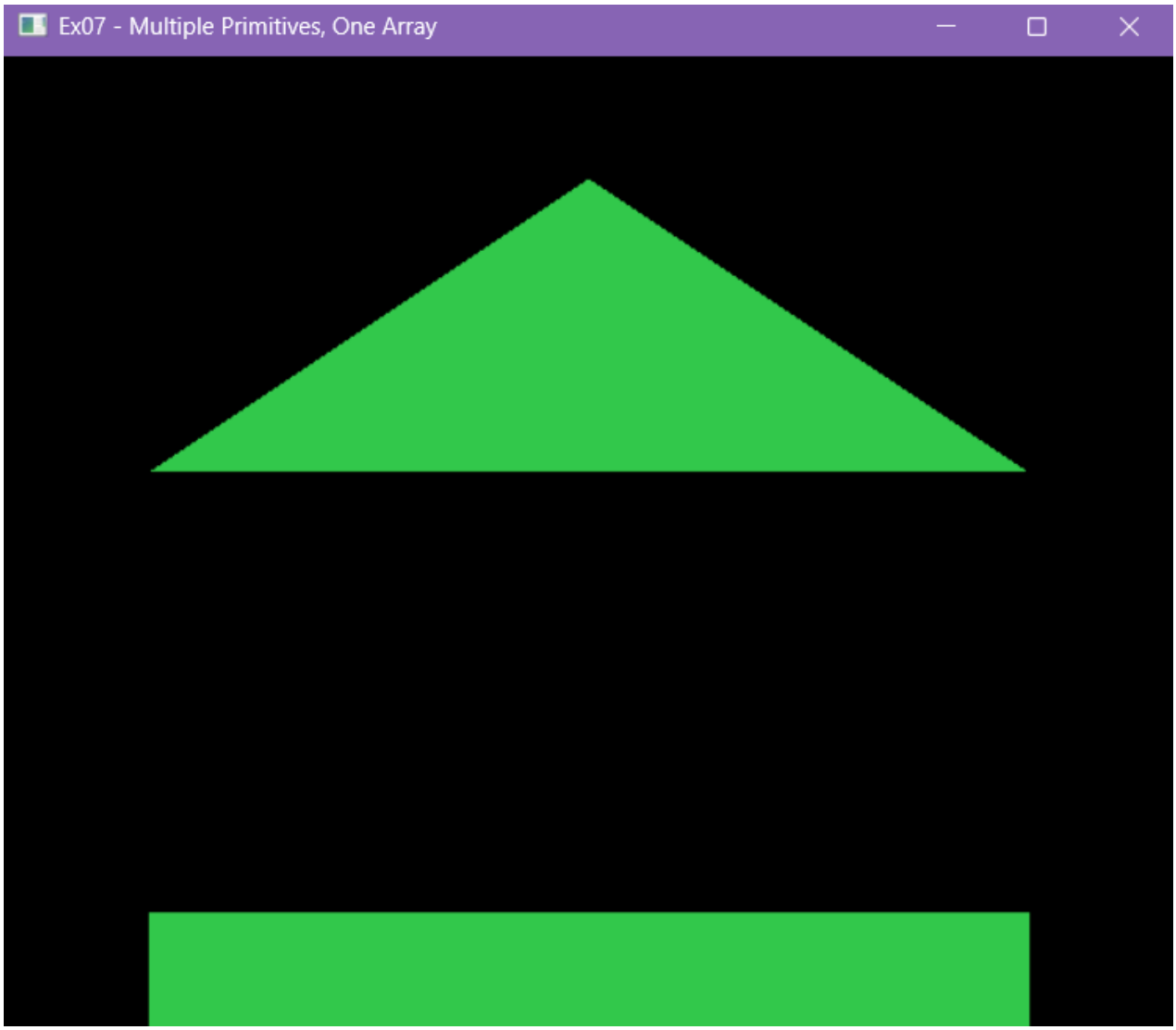
 // triangle 2 (lower) - additional vertices for the 2nd triangle
0.0f, -0.15f, 0.0f,
 -0.75f, -0.75f, 0.0f,
 0.75f, -0.75f, 0.0f
 };

 glEnableClientState(GL_VERTEX_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, triangleVertex);
 glDrawArrays(GL_TRIANGLES, 0, 6); // 6 vertices to be rendered
 glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 glColor3f(0.2f, 0.8f, 0.3f);
 triangles();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex07 - Multiple Primitives, One Array");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```

OUTPUT:



Example 08 — OpenGL Data Type Suffixes in Practice

Concept: glVertexPointer's `type` parameter with GLshort data instead of GLfloat. **Expected**

output: A pink triangle, its coordinates stored as GLshort and scaled down with glScalef.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
```



```
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;
```

```
/*
 * Example 08 - OpenGL Data Type Suffixes in Practice
 * ------------------------------------------------------------
 * Concept: glVertexPointer's "type" parameter is not limited to
 * GL_FLOAT. This example stores vertex coordinates as GLshort (16-bit *
 integer) instead of GLfloat, matching the "s" suffix from the OpenGL *
 data type table, to show the array-based pipeline works with several *
 underlying C types as long as `type` matches the array's declared type.
 */

void trianglesShort() {
 GLshort triangleVertex[] = {
 0, 75,
 -75, 0,
 75, 0
 };

 glEnableClientState(GL_VERTEX_ARRAY);
 glVertexPointer(2, GL_SHORT, 0, triangleVertex); // note: GL_SHORT, not GL_FLOAT
glDrawArrays(GL_TRIANGLES, 0, 3);
 glDisableClientState(GL_VERTEX_ARRAY);
}

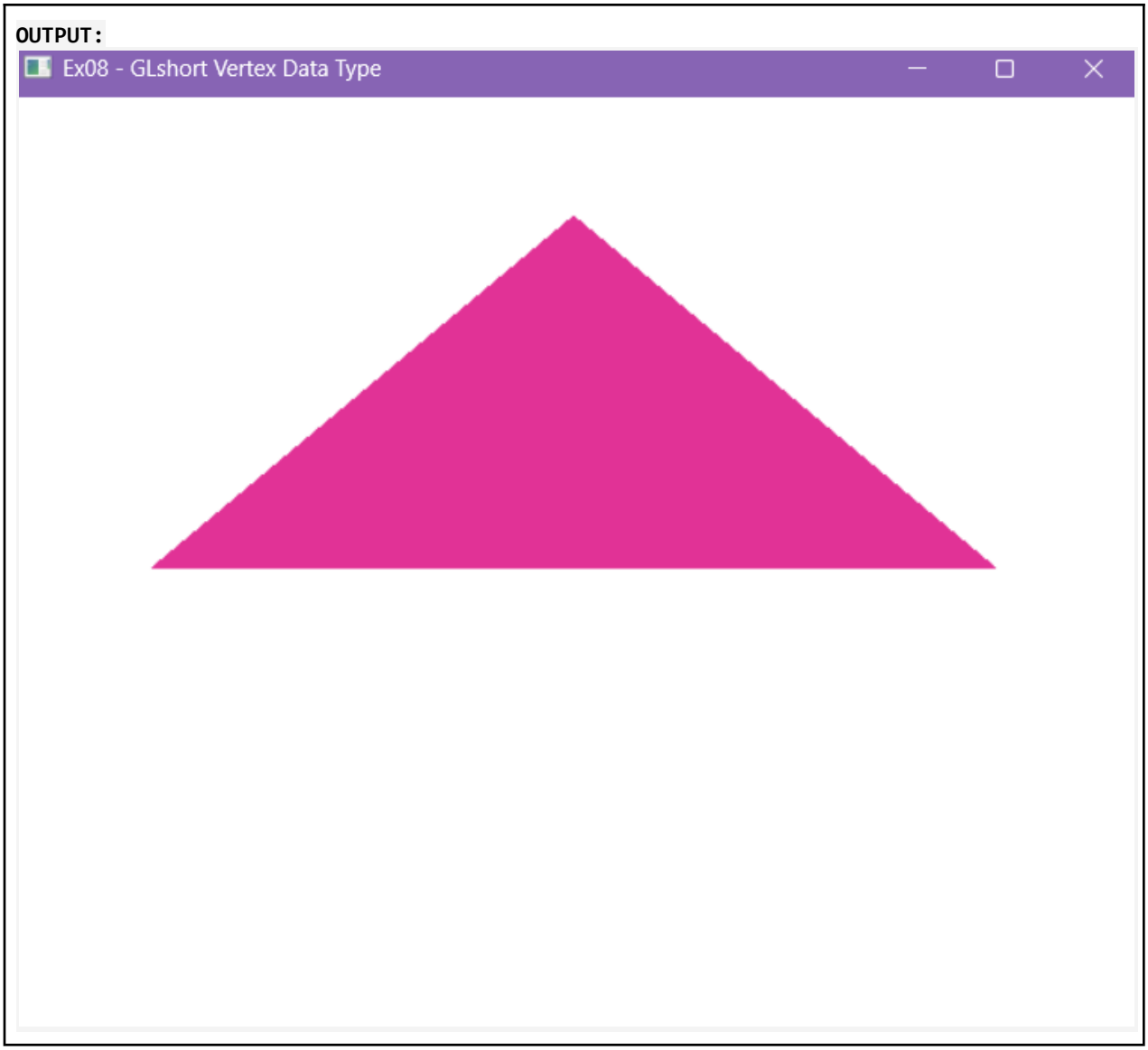
void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 glColor3f(0.9f, 0.2f, 0.6f);

 // GLshort coordinates above are scaled to fit within (-1,1) via glScalef
glPushMatrix();
 glScalef(0.01f, 0.01f, 1.0f);
 trianglesShort();
 glPopMatrix();

 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex08 - GLshort Vertex Data Type");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```

OUTPUT:



Example 09 — glDrawElements(): Indexed Rendering, Basic Case

Concept: The same triangle as Example 02, this time drawn via glDrawElements with a simple {0,1,2} index array.

Expected output: The same green triangle, drawn through the indexed-rendering path.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 09 - glDrawElements(): Indexed Rendering
 * ------------------------------------------------
 * Concept: Instead of glDrawArrays() walking the vertex array in
 * order, glDrawElements() reads a separate INDEX array telling it
 * which vertices to use, and in what order. Here the index array
 * simply selects vertices 0,1,2 - equivalent to Example 02, but using
 * the indexed drawing path.
 */

void triangle() {
 GLfloat triangleVertex[] = {
 0.0f, 0.75f, 0.0f,
 -0.75f, 0.0f, 0.0f,
 0.75f, 0.0f, 0.0f
 };
 GLubyte indices[] = { 0, 1, 2 };

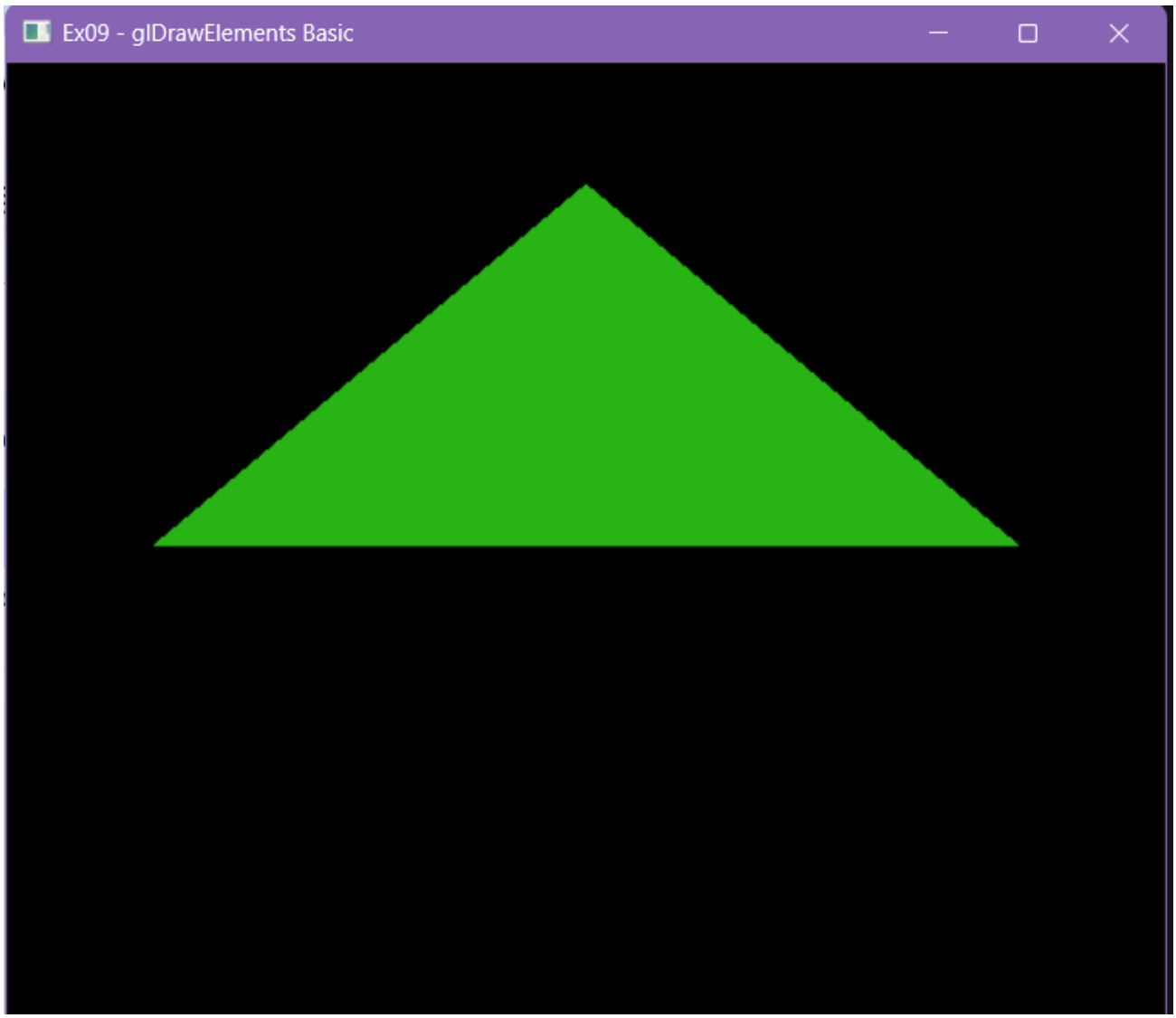
 glEnableClientState(GL_VERTEX_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, triangleVertex);
 glDrawElements(GL_TRIANGLES, 3, GL_UNSIGNED_BYTE, indices);
 glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 glColor3f(0.16f, 0.72f, 0.08f);
 triangle();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex09 - glDrawElements Basic");
 glutDisplayFunc(display);
```



```
 glutMainLoop();
 return 0;
}
```



Example 10 — glDrawElements(): Reusing Shared Vertices

Concept: A quad built from 4 unique vertices plus a 6-entry index array, instead of 6 (partly duplicated) vertices.

Expected output: A solid orange filled square, built from only 4 stored vertices.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 10 - glDrawElements(): Reusing Shared Vertices
 * ------------------------------------------------------
 * Concept: The real payoff of indexed rendering. A quad built from
 two * triangles normally needs 6 vertices with glDrawArrays (2 of
 them
 * duplicated). With glDrawElements, only the 4 UNIQUE corner
 * positions are stored once, and the index array tells OpenGL how to
 * reuse them to form both triangles - saving memory.
 */

void quadFromTriangles() {
 // Only 4 UNIQUE vertices needed, not 6
 GLfloat quadVertex[] = {
 -0.5f, -0.5f, 0.0f, // 0: bottom-left
 0.5f, -0.5f, 0.0f, // 1: bottom-right
 0.5f, 0.5f, 0.0f, // 2: top-right
 -0.5f, 0.5f, 0.0f // 3: top-left
 };

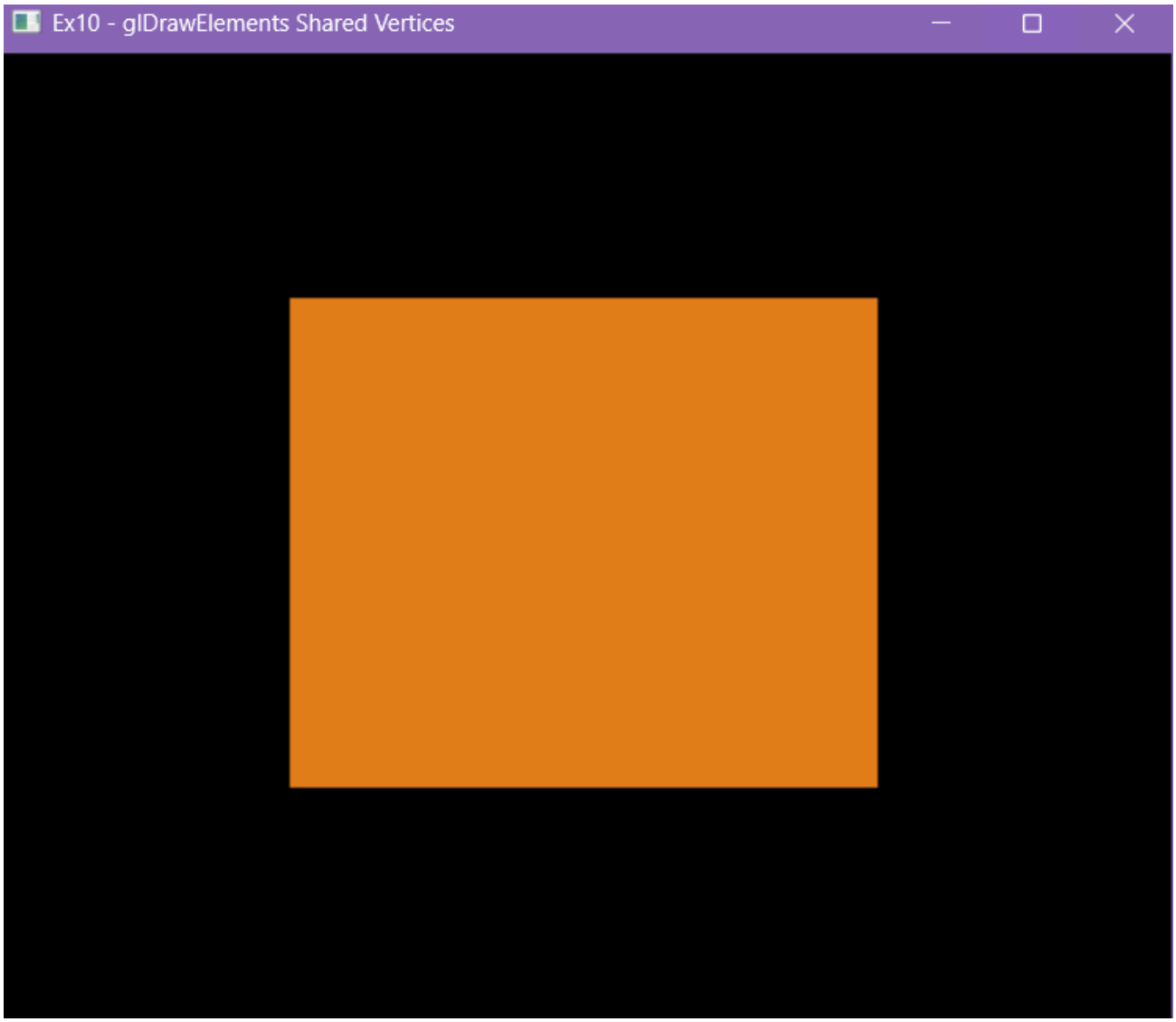
 // Triangle 1: 0,1,2 Triangle 2: 0,2,3 (vertices 0 and 2 are SHARED)
GLubyte indices[] = { 0, 1, 2, 0, 2, 3 };

 glEnableClientState(GL_VERTEX_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, quadVertex);
 glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_BYTE, indices);
 glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
```

```
 glClear(GL_COLOR_BUFFER_BIT);
 glColor3f(0.9f, 0.5f, 0.1f);
 quadFromTriangles();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex10 - glDrawElements Shared Vertices");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```



Example 11 — Vertex Array + Color Array Together

Concept: glColorPointer supplies per-vertex color alongside glVertexPointer's position data.

Expected output: A triangle with red, green, and blue corners, smoothly blending across its interior.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 11 - Combining a Vertex Array With a Color Array
 * ------------------------------------------------------------
 * Concept: glColorPointer works exactly like glVertexPointer, but
 * feeds per-vertex COLOR data instead of position data. Both arrays
 * are enabled and bound together, so glDrawArrays picks up matching
 * position[i] / color[i] pairs automatically.
 */

void triangle() {
 GLfloat vertices[] = {
 0.0f, 0.75f, 0.0f,
 -0.75f, 0.0f, 0.0f,
 0.75f, 0.0f, 0.0f
 };
 GLfloat colors[] = {
 1.0f, 0.0f, 0.0f, // red - top vertex
 0.0f, 1.0f, 0.0f, // green - bottom-left vertex
 0.0f, 0.0f, 1.0f // blue - bottom-right vertex
 };
```

```
 glEnableClientState(GL_VERTEX_ARRAY);
 glEnableClientState(GL_COLOR_ARRAY);

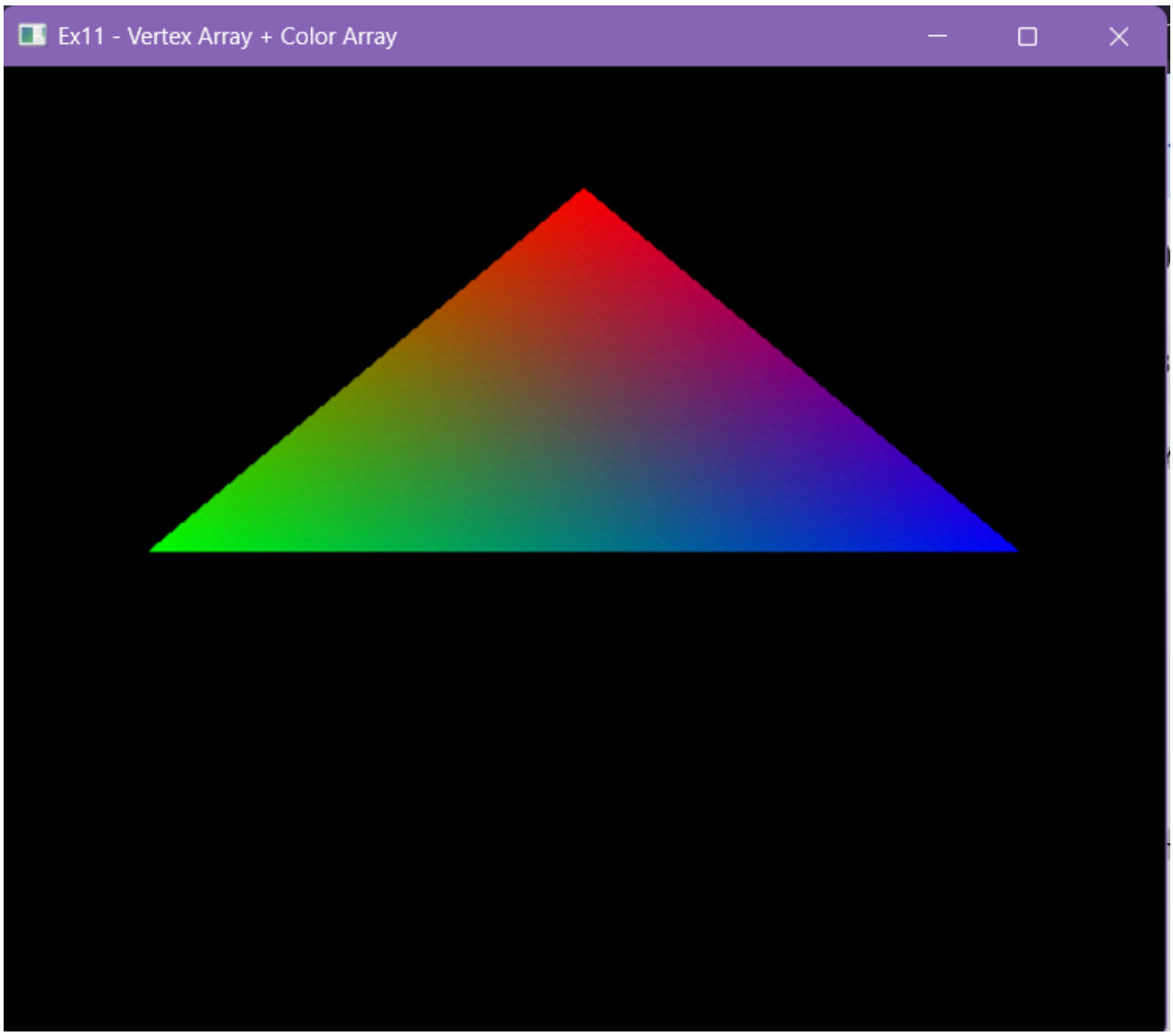
 glVertexPointer(3, GL_FLOAT, 0, vertices);
 glColorPointer(3, GL_FLOAT, 0, colors);

 glDrawArrays(GL_TRIANGLES, 0, 3);

 glDisableClientState(GL_VERTEX_ARRAY);
 glDisableClientState(GL_COLOR_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 triangle();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex11 - Vertex Array + Color Array");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```



Example 12 — Gradient Line Using a Color Array

Concept: Per-vertex colors from a color array interpolate across a primitive, same as Module 2's glColor3f approach.

Expected output: A thick line fading from yellow to purple.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 12 - Gradient Line Using a Color Array
 * ----------------------------------------------------------
 * Concept: Per-vertex colors from a color array interpolate smoothly
```

```
 * across a primitive, exactly like per-vertex glColor3f calls inside
 * glBegin/glEnd - only now the colors come from an array.
 */

void gradientLine() {
 GLfloat vertices[] = {
 -0.8f, 0.0f, 0.0f,
 0.8f, 0.0f, 0.0f
 };
 GLfloat colors[] = {
 1.0f, 1.0f, 0.0f, // yellow
 0.6f, 0.0f, 0.8f // purple
 };

 glEnableClientState(GL_VERTEX_ARRAY);
 glEnableClientState(GL_COLOR_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, vertices);
 glColorPointer(3, GL_FLOAT, 0, colors);
 glLineWidth(6.0f);
 glDrawArrays(GL_LINES, 0, 2);
 glDisableClientState(GL_VERTEX_ARRAY);
 glDisableClientState(GL_COLOR_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 gradientLine();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex12 - Gradient Line via Color Array");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```



Example 13 — Four-Triangle Pinwheel, One Array, One Draw Call

Concept: Recreates the lecture's colored pinwheel using one vertex array + one color array + a single glDrawArrays(GL_TRIANGLES, 0, 12) call.

Expected output: A four-armed pinwheel, each arm fading red-green-blue.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
```



```
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
```

```
 * Example 13 - Four-Triangle Pinwheel, One Array, One Draw Call *
 ------------------------------------------------------------ *
 Concept: Recreates the lecture's colored pinwheel (left/right/up/ *
 down triangles, each fading red->green->blue) using a SINGLE vertex
 * array + SINGLE color array + ONE glDrawArrays(GL_TRIANGLES, 0, 12)
 * call, instead of four separate immediate-mode glBegin/glEnd
 blocks. */

GLfloat triangleVertices[] = {
 0.0f, 0.0f, 0.0f, -0.5f, 0.10f, 0.0f, -0.5f, -0.10f, 0.0f, // left  0.0f,
0.0f, 0.0f, 0.5f, 0.10f, 0.0f, 0.5f, -0.10f, 0.0f, // right  0.0f, 0.0f,
0.0f, -0.10f, 0.50f, 0.0f, 0.10f, 0.50f, 0.0f, // up  0.0f, 0.0f, 0.0f,
-0.10f, -0.50f, 0.0f, 0.10f, -0.50f, 0.0f // down };

GLfloat triangleColors[] = {
 1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
 1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
 1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
 1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f
};

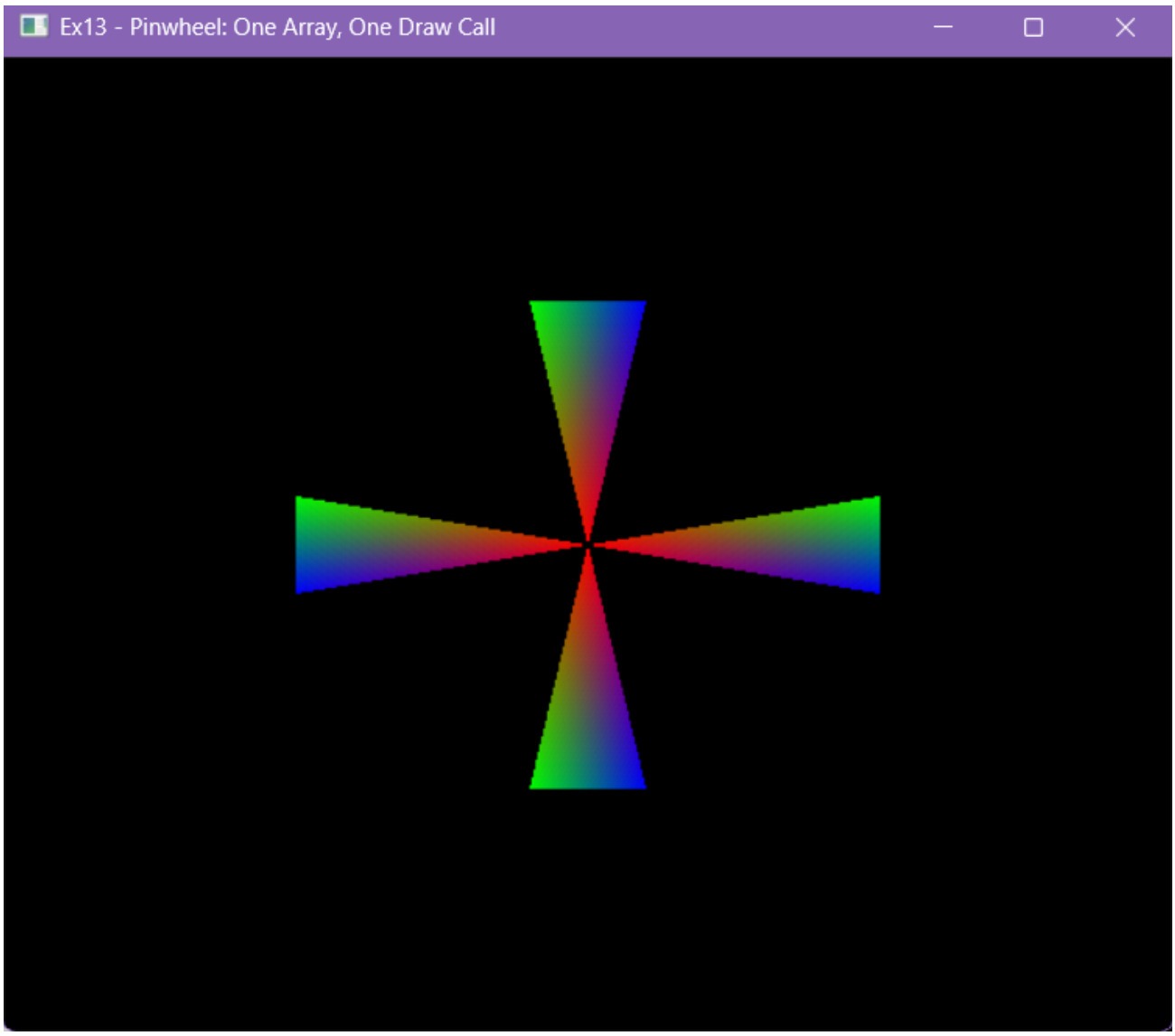
void pinwheel() {
 glEnableClientState(GL_VERTEX_ARRAY);
 glEnableClientState(GL_COLOR_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, triangleVertices);
 glColorPointer(3, GL_FLOAT, 0, triangleColors);
 glDrawArrays(GL_TRIANGLES, 0, 12);
 glDisableClientState(GL_VERTEX_ARRAY);
 glDisableClientState(GL_COLOR_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 pinwheel();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex13 - Pinwheel: One Array, One Draw Call");
glutDisplayFunc(display);
 glutMainLoop();
 return 0;
```



```
 }
```



Example 14 — Full Scene: Pinwheel + Rectangle

Concept: Two separate vertex-array draw calls combined into one scene, mirroring the lecture's final sample program.

Expected output: The colored pinwheel with a solid green rectangle underneath it.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 14 - Full Scene: Pinwheel + Rectangle (Two Draw Calls)
 * ------------------------------------------------------------
 * Concept: A complete scene composed of TWO separate vertex-array
 draw * calls: the colored pinwheel (vertex array + color array),
 and a
 * solid green rectangle underneath (vertex array only, flat color).
 * Mirrors the lecture's final combined sample program.
 */

GLfloat triangleVertices[] = {
 0.0f, 0.0f, 0.0f, -0.5f, 0.10f, 0.0f, -0.5f, -0.10f, 0.0f,
 0.0f, 0.0f, 0.0f, 0.5f, 0.10f, 0.0f, 0.5f, -0.10f, 0.0f,
 0.0f, 0.0f, 0.0f, -0.10f, 0.50f, 0.0f, 0.10f, 0.50f, 0.0f,
 0.0f, 0.0f, 0.0f, -0.10f, -0.50f, 0.0f, 0.10f, -0.50f, 0.0f
};

GLfloat triangleColors[] = {
 1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
 1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
 1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f,
 1.0f,0.0f,0.0f, 0.0f,1.0f,0.0f, 0.0f,0.0f,1.0f
};

GLfloat quadVertices[] = {
 -0.5f, -0.7f, 0.0f,
 0.5f, -0.7f, 0.0f,
 0.5f, -0.9f, 0.0f,
 -0.5f, -0.9f, 0.0f
};

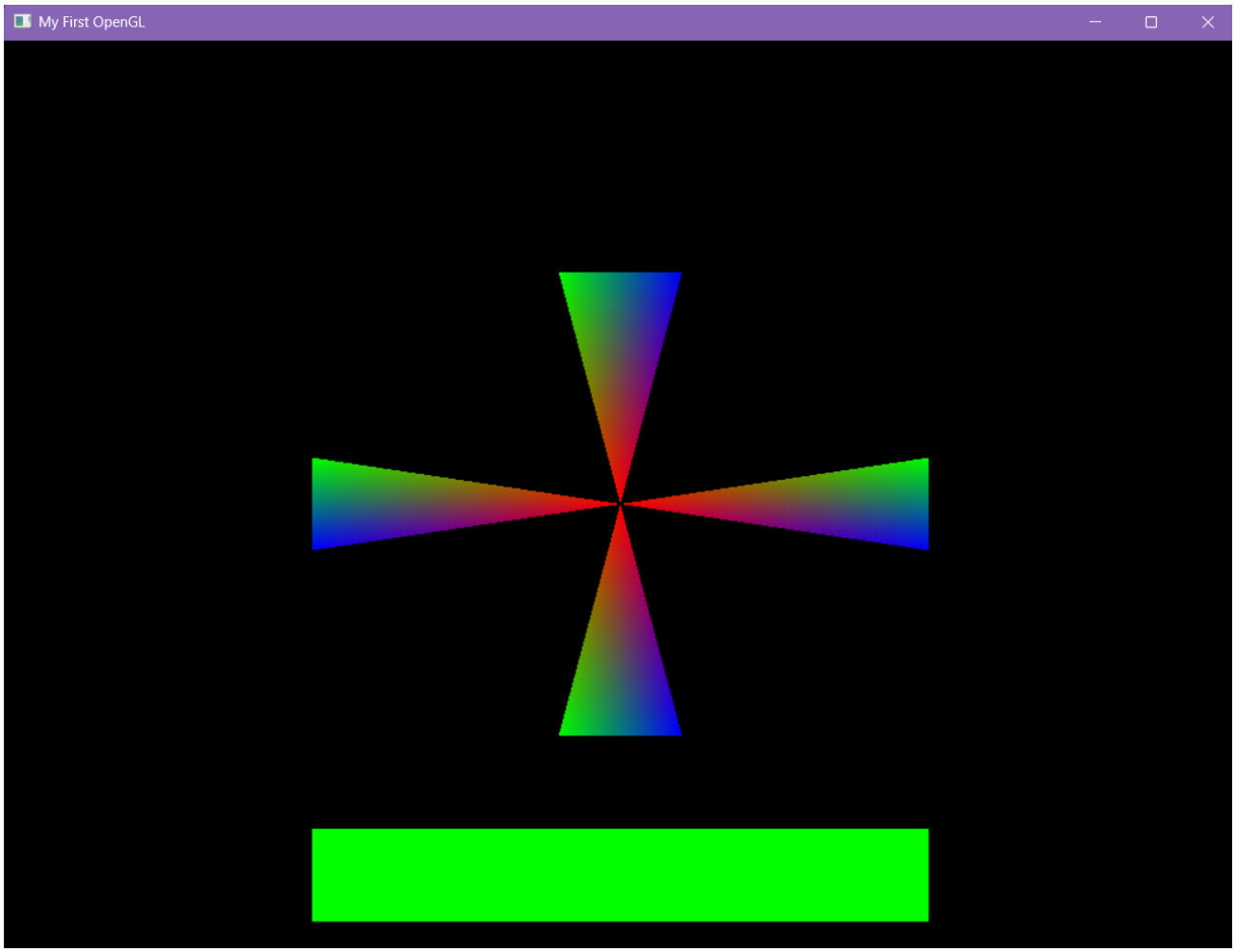
void pinwheel() {
```

```
 glEnableClientState(GL_VERTEX_ARRAY);
 glEnableClientState(GL_COLOR_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, triangleVertices);
 glColorPointer(3, GL_FLOAT, 0, triangleColors);
 glDrawArrays(GL_TRIANGLES, 0, 12);
 glDisableClientState(GL_VERTEX_ARRAY);
 glDisableClientState(GL_COLOR_ARRAY);
}

void rectangle() {
 glColor3f(0.0f, 1.0f, 0.0f);
 glEnableClientState(GL_VERTEX_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, quadVertices);
 glDrawArrays(GL_QUADS, 0, 4);
 glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 pinwheel();
 rectangle();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(1024, 768);
 glutInitWindowPosition(200, 50);
 glutCreateWindow("My First OpenGL");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```



Example 15 — Indexed Pinwheel: Sharing the Center Vertex

Concept: The same pinwheel, drawn with glDrawElements so the shared center vertex is stored only once (12 -> 9 unique vertices).

Expected output: The same pinwheel shape as Example 13, this time built from an indexed vertex/color array pair.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
```



```
using namespace std;

/*
 * Example 15 - Indexed Pinwheel: Sharing the Center Vertex *
 ---------------------------------------------------------- *
```

```
 Concept: All four pinwheel triangles share the exact same center *
 point (0,0,0). Using glDrawElements, that shared vertex only needs
 * to be stored ONCE, and every triangle's index list simply points
 * back to it - reducing the unique vertex count from 12 to 9. */

GLfloat pinwheelVertices[] = {
 0.0f, 0.0f, 0.0f, // 0: shared center
 -0.5f, 0.10f, 0.0f, // 1: left-top
 -0.5f, -0.10f, 0.0f, // 2: left-bottom
 0.5f, 0.10f, 0.0f, // 3: right-top
 0.5f, -0.10f, 0.0f, // 4: right-bottom
 -0.10f, 0.50f, 0.0f, // 5: up-left
 0.10f, 0.50f, 0.0f, // 6: up-right
 -0.10f, -0.50f, 0.0f, // 7: down-left
 0.10f, -0.50f, 0.0f // 8: down-right
};

GLfloat pinwheelColors[] = {
 1.0f, 1.0f, 1.0f, // center - white
 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, // left triangle outer colors
0.0f, 0.0f, 1.0f, 1.0f, 1.0f, 0.0f, // right triangle outer colors
1.0f, 0.0f, 1.0f, 0.0f, 1.0f, 1.0f, // up triangle outer colors  1.0f,
0.5f, 0.0f, 0.5f, 0.0f, 1.0f // down triangle outer colors };

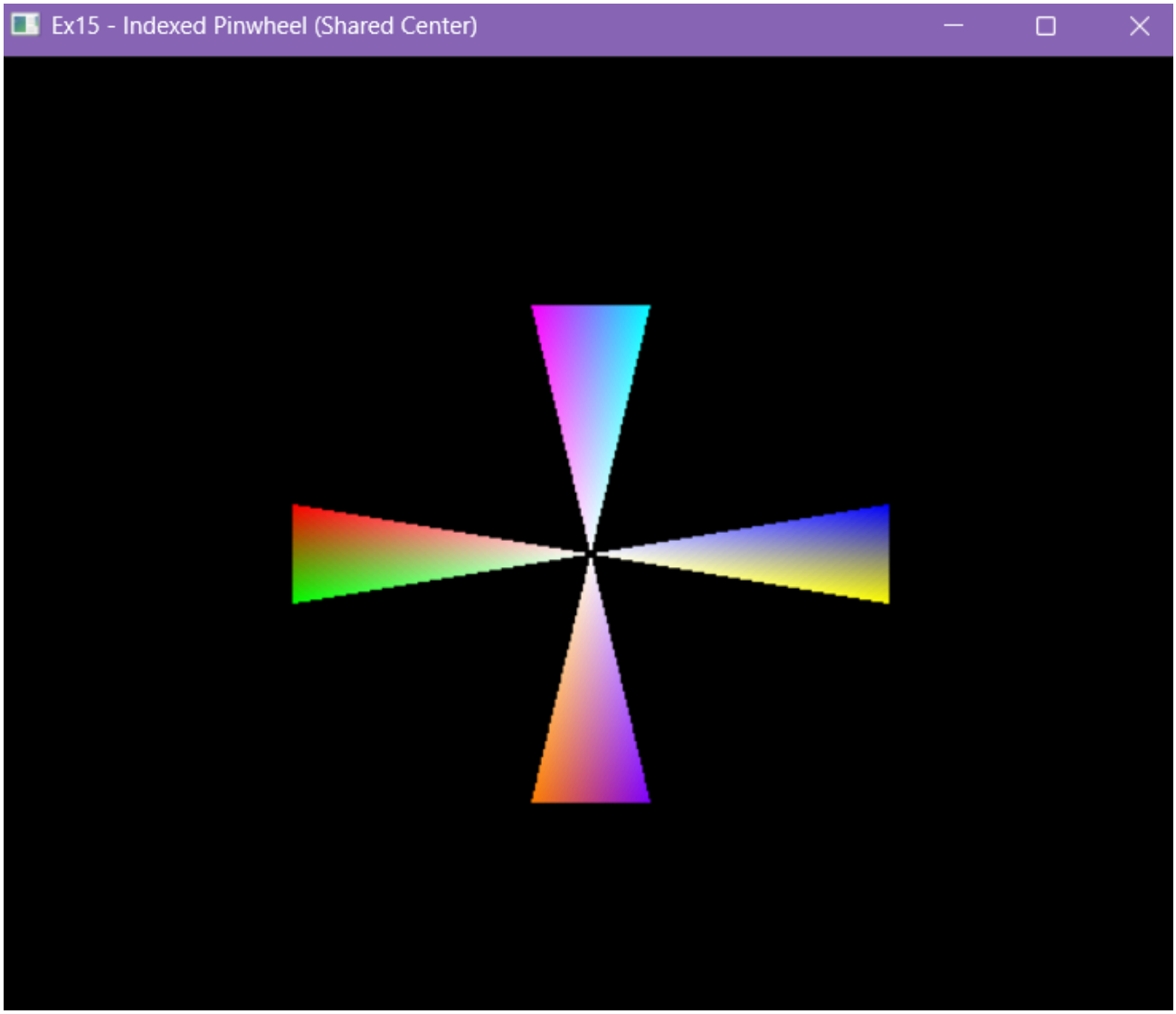
GLubyte pinwheelIndices[] = {
 0, 1, 2, // left triangle (reuses shared center = index 0)  0,
3, 4, // right triangle
 0, 5, 6, // up triangle
 0, 7, 8 // down triangle
};

void indexedPinwheel() {
 glEnableClientState(GL_VERTEX_ARRAY);
 glEnableClientState(GL_COLOR_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, pinwheelVertices);
 glColorPointer(3, GL_FLOAT, 0, pinwheelColors);
 glDrawElements(GL_TRIANGLES, 12, GL_UNSIGNED_BYTE, pinwheelIndices);
glDisableClientState(GL_VERTEX_ARRAY);
 glDisableClientState(GL_COLOR_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 indexedPinwheel();
 glFlush();
```

```
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex15 - Indexed Pinwheel (Shared Center)");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```



Example 16 — Interleaved Arrays and the stride Parameter

Concept: Position and color interleaved into ONE array, read out using a non-zero stride.

Expected output: A red/green/blue triangle, identical to Example 11's result, sourced from a single interleaved array.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 16 - Interleaved Arrays and the "stride" Parameter
 * ----------------------------------------------------------
 * Concept: So far, position and color have lived in SEPARATE arrays
 * (stride 0 = tightly packed, one array's own data only). Here they
 * are INTERLEAVED into a single array as [x,y,z,r,g,b,
 x,y,z,r,g,b,...] * and glVertexPointer/glColorPointer use a
 non-zero stride (the byte
 * offset to the NEXT vertex's data) plus a starting offset pointer to
 * read their own fields out of the same combined array.
 */

void interleavedTriangle() {
 // Each vertex: 3 position floats followed by 3 color floats
 GLfloat data[] = {
 // x, y, z, r, g, b
 0.0f, 0.75f, 0.0f, 1.0f, 0.0f, 0.0f,
 -0.75f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
 0.75f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f
 };

 GLsizei stride = 6 * sizeof(GLfloat); // bytes from one vertex to the next
```

```
 glEnableClientState(GL_VERTEX_ARRAY);
  glEnableClientState(GL_COLOR_ARRAY);

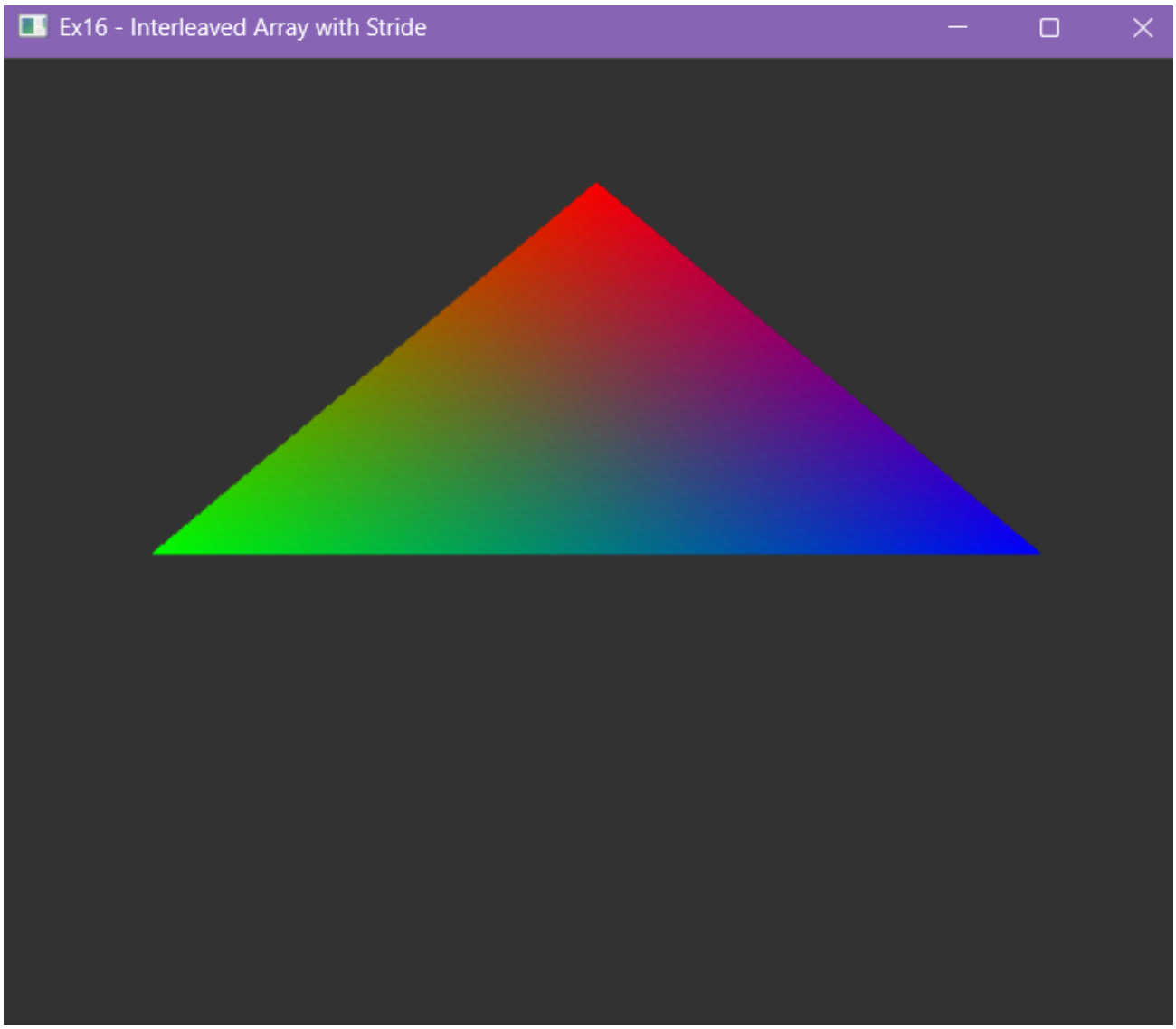
  glVertexPointer(3, GL_FLOAT, stride, data); // position starts at offset 0
glColorPointer(3, GL_FLOAT, stride, data + 3); // color starts 3 floats in

  glDrawArrays(GL_TRIANGLES, 0, 3);

  glDisableClientState(GL_VERTEX_ARRAY);
  glDisableClientState(GL_COLOR_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 interleavedTriangle();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex16 - Interleaved Array with Stride");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```



Example 17 — One Array, Multiple glDrawArrays 'first' Offsets

Concept: Reusing one big array across several glDrawArrays calls, each starting at a different `first` index.

Expected output: Three separate triangles in a row, each a different color, all sourced from one shared array.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 17 - One Big Array, Multiple glDrawArrays "first" Offsets
 * ----------------------------------------------------------------
 * Concept: A single vertex array can be reused across SEVERAL
 * glDrawArrays calls, each reading a different slice via the "first"
 * parameter - here, drawing three separate triangles stored back to
```

```
 * back in one array, one glDrawArrays call per triangle, each with a
 * different color and a different "first" starting index.
 */

GLfloat rowVertices[] = {
 -0.9f, -0.3f, 0.0f, -0.7f, 0.3f, 0.0f, -0.5f, -0.3f, 0.0f, // triangle 0
(indices 0-2)
 -0.15f, -0.3f, 0.0f, 0.05f, 0.3f, 0.0f, 0.25f, -0.3f, 0.0f, // triangle 1
(indices 3-5)
 0.5f, -0.3f, 0.0f, 0.7f, 0.3f, 0.0f, 0.9f, -0.3f, 0.0f // triangle 2
(indices 6-8)
};

void drawRow() {
 glEnableClientState(GL_VERTEX_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, rowVertices);

 glColor3f(1.0f, 0.2f, 0.2f);
 glDrawArrays(GL_TRIANGLES, 0, 3); // first = 0

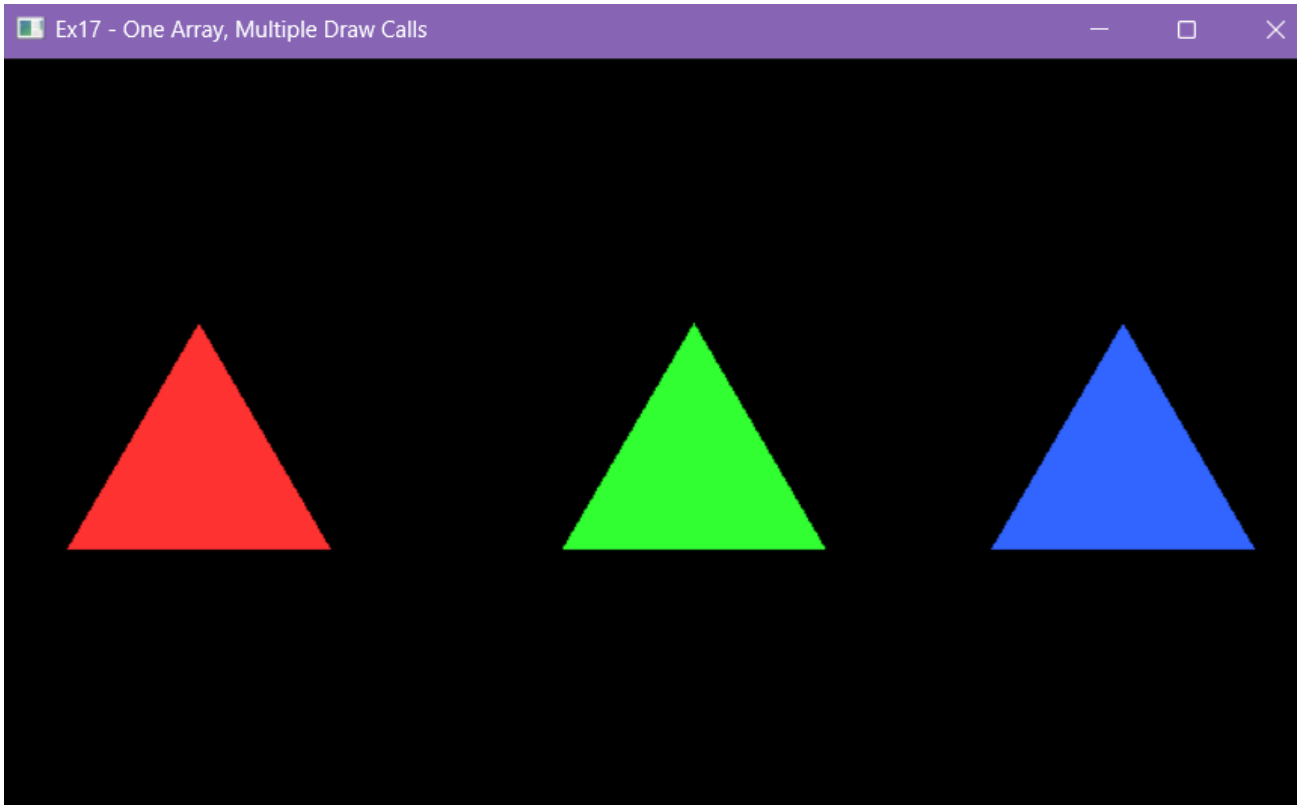
 glColor3f(0.2f, 1.0f, 0.2f);
 glDrawArrays(GL_TRIANGLES, 3, 3); // first = 3

 glColor3f(0.2f, 0.4f, 1.0f);
 glDrawArrays(GL_TRIANGLES, 6, 3); // first = 6

 glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 drawRow();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(700, 400);
 glutCreateWindow("Ex17 - One Array, Multiple Draw Calls");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```



Example 18 — glDrawElements() Across Multiple Shapes

Concept: One unique-vertex pool feeding two separate indexed draw calls for two different shapes.

Expected output: An orange quad and a blue triangle, built from two index lists into the same vertex pool.


```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 18 - glDrawElements() Across Multiple Shapes *
 ------------------------------------------------------- *
 Concept: A single unique-vertex pool feeding TWO separate indexed
 * draws - a triangle pair (quad) built with one index list, and a
 * standalone triangle built from three OTHER vertices in the same
 * pool, using a second glDrawElements call.
 */

GLfloat pool[] = {
 -0.9f, -0.5f, 0.0f, // 0
```

```
 -0.3f, -0.5f, 0.0f, // 1
 -0.3f, 0.5f, 0.0f, // 2
 -0.9f, 0.5f, 0.0f, // 3
 0.3f, -0.5f, 0.0f, // 4
 0.9f, -0.5f, 0.0f, // 5
 0.6f, 0.5f, 0.0f // 6
};

GLubyte quadIndices[] = { 0, 1, 2, 0, 2, 3 };
GLubyte triIndices[] = { 4, 5, 6 };

void drawShapes() {
 glEnableClientState(GL_VERTEX_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, pool);

 glColor3f(0.9f, 0.6f, 0.1f);
 glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_BYTE, quadIndices);

 glColor3f(0.1f, 0.6f, 0.9f);
 glDrawElements(GL_TRIANGLES, 3, GL_UNSIGNED_BYTE, triIndices);

 glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 drawShapes();
 glFlush();
}

int main(int argc, char** argv) {
```



```
 glutInit(&argc, argv);
 glutInitWindowSize(700, 400);
 glutCreateWindow("Ex18 - glDrawElements, Multiple Shapes");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```



Example 19 — Smoothly Shaded Quad Using a Color Array

Concept: Four vertices, four colors, one color array — OpenGL interpolates all four across the quad. **Expected output:** A quad with red, green, blue, and yellow corners, smoothly blended.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 19 - Smoothly Shaded Quad Using a Color Array
 * ------------------------------------------------------
 * Concept: Four vertices, four different colors, one color array -
 * OpenGL interpolates all four across the quad's interior, exactly as
 * it would with four separate glColor3f calls inside glBegin/glEnd.
 */

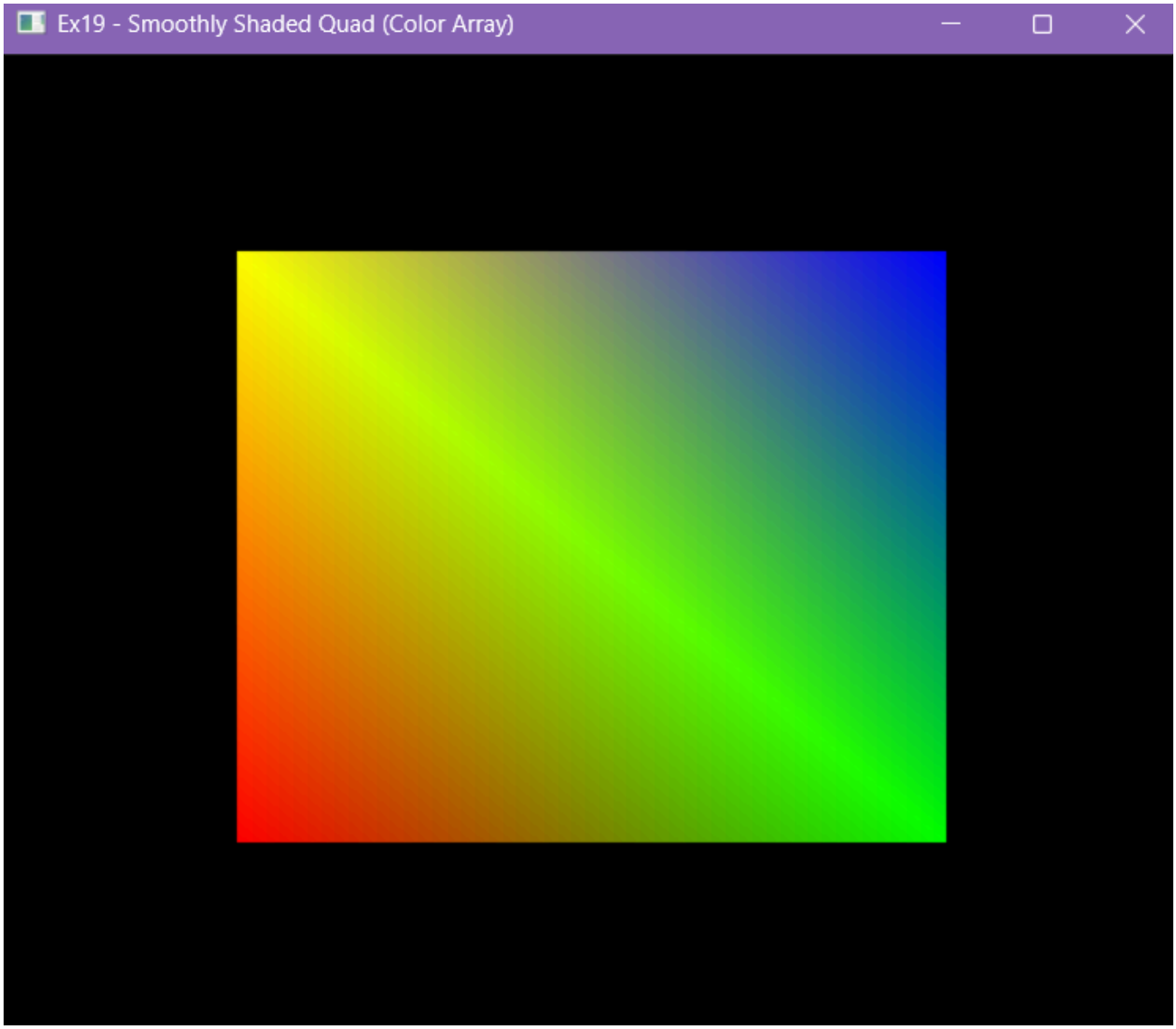
void shadedQuad() {
 GLfloat vertices[] = {
 -0.6f, -0.6f, 0.0f,
 0.6f, -0.6f, 0.0f,
 0.6f, 0.6f, 0.0f,
 -0.6f, 0.6f, 0.0f
 };
 GLfloat colors[] = {
 1.0f, 0.0f, 0.0f,
 0.0f, 1.0f, 0.0f,
 0.0f, 0.0f, 1.0f,
 1.0f, 1.0f, 0.0f
 };

 glEnableClientState(GL_VERTEX_ARRAY);
 glEnableClientState(GL_COLOR_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, vertices);
 glColorPointer(3, GL_FLOAT, 0, colors);
 glDrawArrays(GL_QUADS, 0, 4);
```

```
 glDisableClientState(GL_VERTEX_ARRAY);
 glDisableClientState(GL_COLOR_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 shadedQuad();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(600, 500);
 glutCreateWindow("Ex19 - Smoothly Shaded Quad (Color Array)");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```



Example 20 — Capstone: An Efficient Composite Scene

Concept: A star (indexed triangle fan), a colored quad (vertex+color array), and a ground rectangle — every shape sourced from array data, no glVertex2f calls anywhere.

Expected output: A small scene: a yellow star, a multicolor quad, and a green ground strip.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
using namespace std;

/*
 * Example 20 - Capstone: An Efficient Composite Scene
 * ------------------------------------------------------------
 * Concept: Combines everything from this module into one program:
 * - A star-like GL_TRIANGLE_FAN built with glDrawElements, sharing
 * one center vertex across all its slices (indexed rendering).
 * - A colored quad rendered with a combined vertex array + color
 * array in a single glDrawArrays call.
 * - A plain ground rectangle rendered from its own small vertex
array. * Every shape in the scene is drawn from array data - there
is not a
 * single glVertex2f call left in this program.
 */

// ---- Star (triangle fan sharing a center vertex) ----
```



```
GLfloat starVertices[] = {
 0.0f, 0.35f, 0.0f, // 0: shared center
 0.0f, 0.75f, 0.0f, // 1
 0.15f, 0.45f, 0.0f, // 2
 0.5f, 0.45f, 0.0f, // 3
 0.2f, 0.25f, 0.0f, // 4
 0.3f, -0.05f, 0.0f, // 5
 0.0f, 0.15f, 0.0f, // 6
 -0.3f, -0.05f, 0.0f, // 7
 -0.2f, 0.25f, 0.0f, // 8
 -0.5f, 0.45f, 0.0f, // 9
 -0.15f, 0.45f, 0.0f, // 10
 0.0f, 0.75f, 0.0f // 11 (repeat of vertex 1 to close the fan) };

GLubyte starIndices[] = { 0,1,2,3,4,5,6,7,8,9,10,11 };

// ---- Colored quad ----
GLfloat quadVertices[] = {
 -0.8f, -0.55f, 0.0f,
 -0.3f, -0.55f, 0.0f,
 -0.3f, -0.15f, 0.0f,
```

```
 -0.8f, -0.15f, 0.0f
};
GLfloat quadColors[] = {
 1.0f, 0.4f, 0.0f,
 0.0f, 0.8f, 1.0f,
 0.6f, 0.0f, 1.0f,
 1.0f, 1.0f, 0.0f
};

// ---- Ground rectangle ----
GLfloat groundVertices[] = {
 -0.9f, -0.8f, 0.0f,
 0.9f, -0.8f, 0.0f,
 0.9f, -0.95f, 0.0f,
 -0.9f, -0.95f, 0.0f
};

void star() {
 glColor3f(1.0f, 0.85f, 0.1f);
 glEnableClientState(GL_VERTEX_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, starVertices);
 glDrawElements(GL_TRIANGLE_FAN, 12, GL_UNSIGNED_BYTE, starIndices);
glDisableClientState(GL_VERTEX_ARRAY);
}

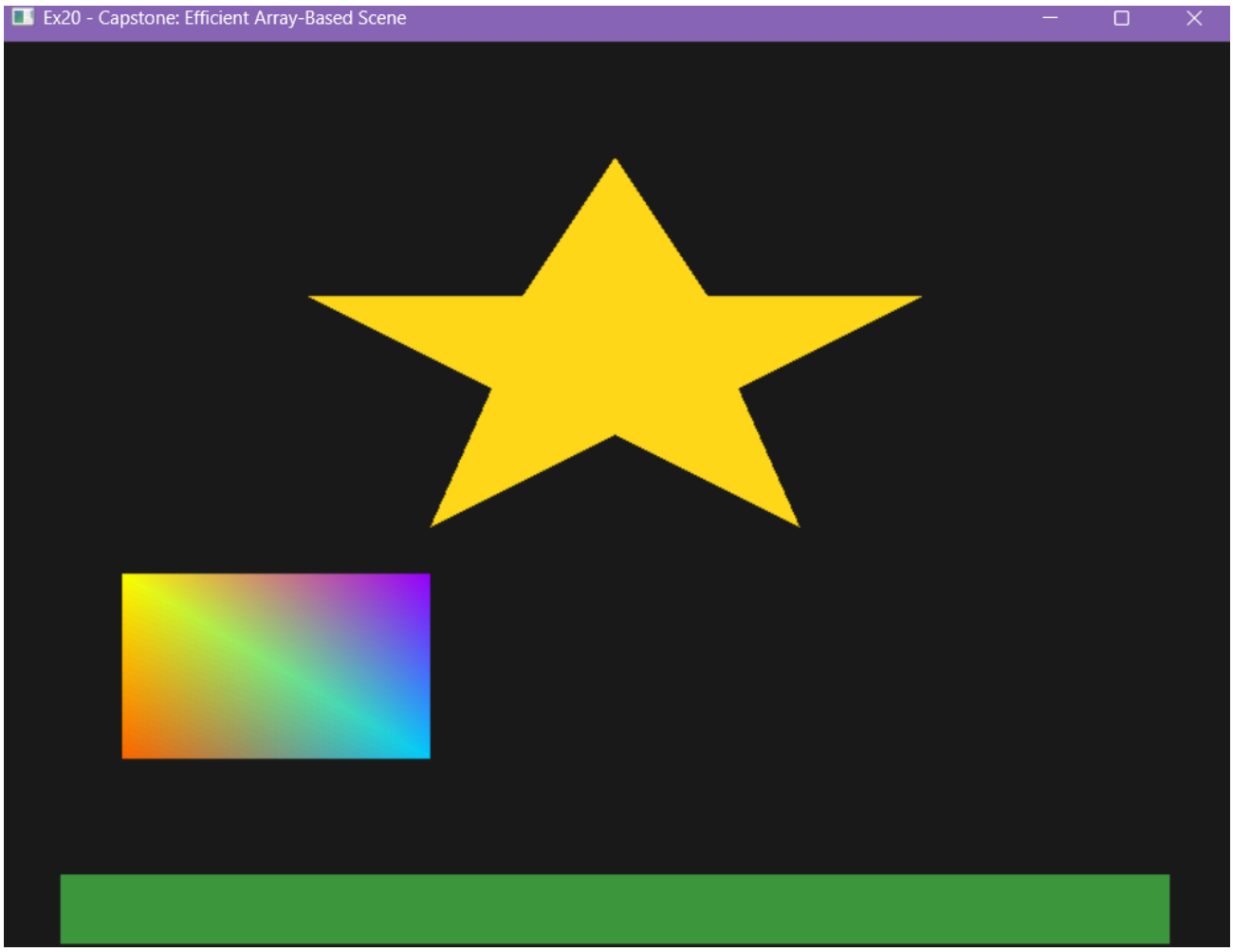
void coloredQuad() {
 glEnableClientState(GL_VERTEX_ARRAY);
 glEnableClientState(GL_COLOR_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, quadVertices);
 glColorPointer(3, GL_FLOAT, 0, quadColors);
```

```
 glDrawArrays(GL_QUADS, 0, 4);
 glDisableClientState(GL_VERTEX_ARRAY);
 glDisableClientState(GL_COLOR_ARRAY);
}

void ground() {
 glColor3f(0.25f, 0.6f, 0.25f);
 glEnableClientState(GL_VERTEX_ARRAY);
 glVertexPointer(3, GL_FLOAT, 0, groundVertices);
 glDrawArrays(GL_QUADS, 0, 4);
 glDisableClientState(GL_VERTEX_ARRAY);
}

void display() {
 glClear(GL_COLOR_BUFFER_BIT);
 star();
 coloredQuad();
 ground();
 glFlush();
}

int main(int argc, char** argv) {
 glutInit(&argc, argv);
 glutInitWindowSize(800, 600);
 glutCreateWindow("Ex20 - Capstone: Efficient Array-Based Scene");
 glutDisplayFunc(display);
 glutMainLoop();
 return 0;
}
```



Part B — Practice Exercises (Q01–Q20)

Attempt each exercise on your own first. Compile and run your own solution, compare its output to the task description, and only THEN check Part C. Difficulty is marked Easy, Medium, or Hard. Each exercise's "Task" line is the exact question you must answer with your program.

Easy Exercises

Exercise Q01 — Vertex Array X-Pattern Points [Easy]

Objective: Practice converting GL_POINTS to a vertex array using the 5-step pattern.

Task (Question): Convert an immediate-mode GL_POINTS block (5 points forming an X pattern with a center point) into a vertex array, using all 5 steps.

Hints:

- Set glPointSize BEFORE glDrawArrays, just as you would before glBegin.
- Double-check that `size` in glVertexPointer matches how many floats you use per vertex (2 or 3).

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

GLfloat xPoints[] = {
    -0.5f,  0.5f, // Top-left
     0.5f, -0.5f, // Bottom-right
     0.0f,  0.0f, // Center
    -0.5f, -0.5f, // Bottom-left
     0.5f,  0.5f  // Top-right
};

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glPointSize(8.0f);
    glColor3f(1.0f, 0.5f, 0.0f);

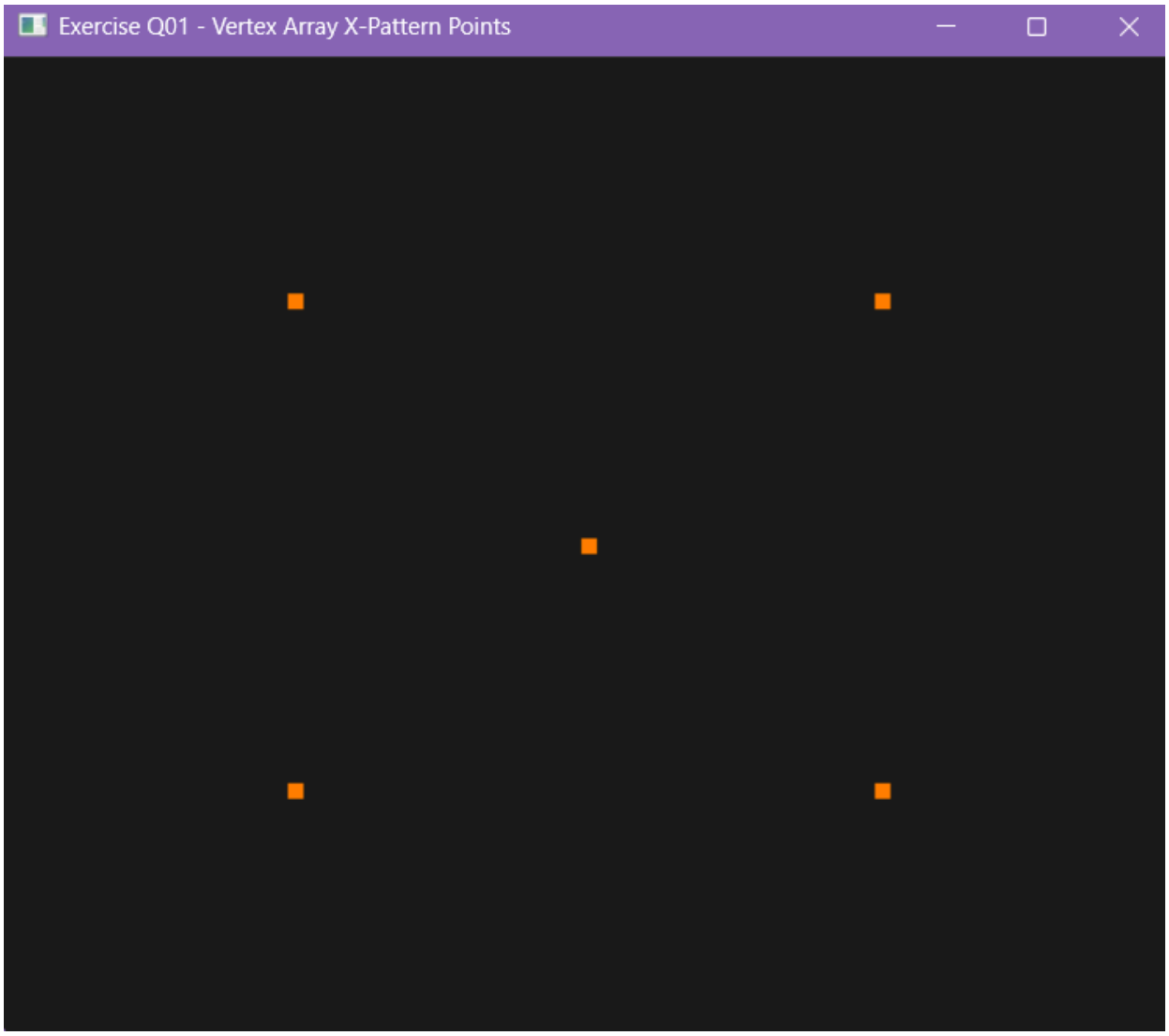
    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(2, GL_FLOAT, 0, xPoints);
    glDrawArrays(GL_POINTS, 0, 5);
    glDisableClientState(GL_VERTEX_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Exercise Q01 - Vertex Array X-Pattern Points");
```

```
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q02 — Filled Hexagon via Vertex Array [Easy]

Objective: Practice GL_POLYGON with a larger vertex count than the lecture's pentagon example.

Task (Question): Draw a filled, six-sided hexagon using GL_POLYGON via a vertex array, with 6 unique vertices of your own choosing.

Hints:

- Plan your six (x, y) points on paper first, going consistently clockwise or counter-clockwise.
- Remember `count` in glDrawArrays must equal exactly 6.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// 6 unique vertices forming a regular-like hexagon in counter-clockwise order
GLfloat hexVertices[] = {
     0.5f,  0.0f, // Vertex 0: Right
     0.25f, 0.43f,// Vertex 1: Top-right
    -0.25f, 0.43f,// Vertex 2: Top-left
    -0.5f,  0.0f, // Vertex 3: Left
    -0.25f,-0.43f,// Vertex 4: Bottom-left
     0.25f,-0.43f // Vertex 5: Bottom-right
};

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

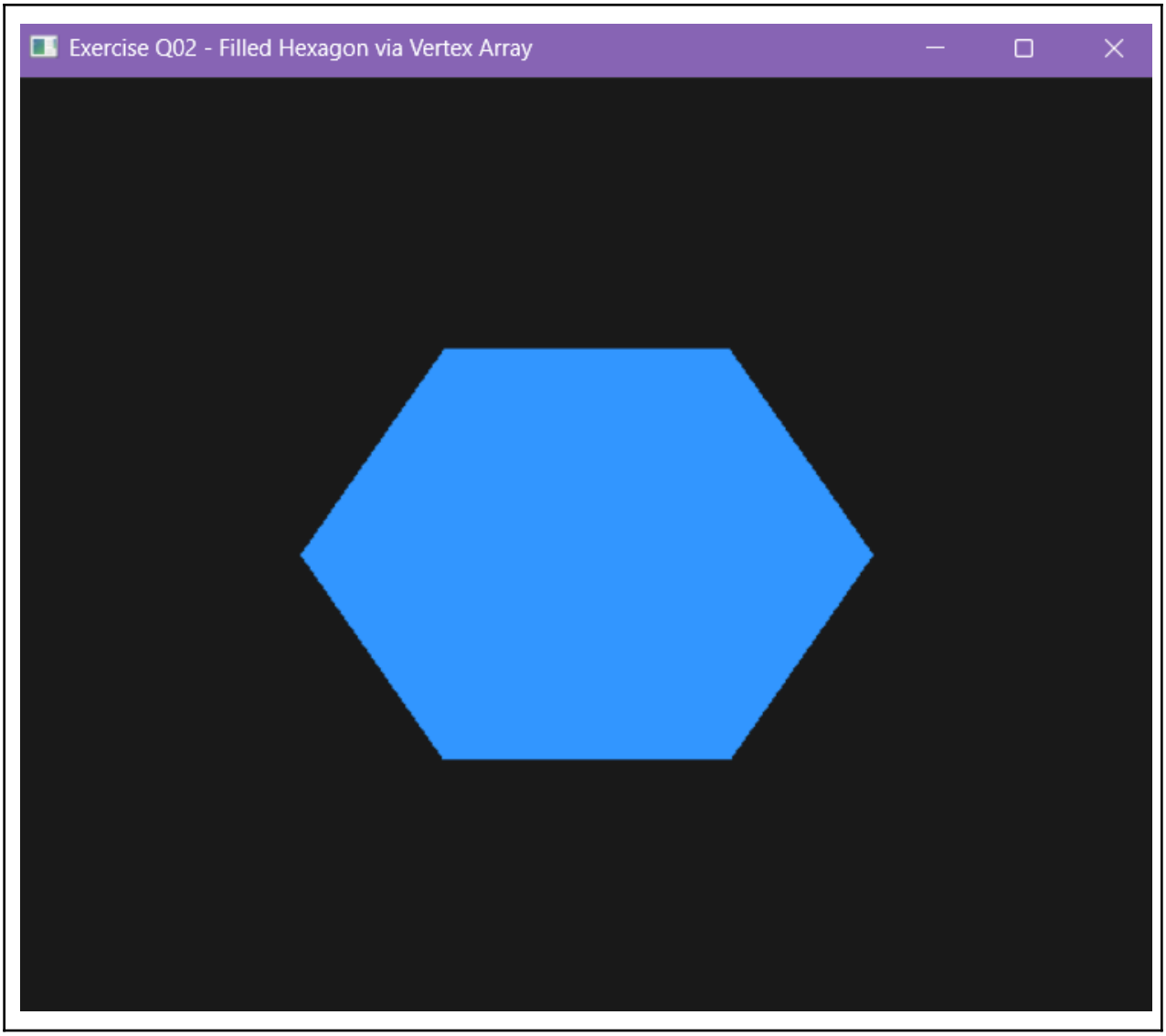
    glColor3f(0.2f, 0.6f, 1.0f); // Blue fill color

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(2, GL_FLOAT, 0, hexVertices);
    glDrawArrays(GL_POLYGON, 0, 6); // Exactly 6 vertices for the hexagon[cite:
4]
    glDisableClientState(GL_VERTEX_ARRAY);
```

```
    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Exercise Q02 - Filled Hexagon via Vertex Array");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q03 — Two Lines, One Array [Easy]

Objective: Practice packing multiple independent primitives into one vertex array.

Task (Question): Draw two separate GL_LINES segments from ONE vertex array and ONE glDrawArrays call (4 vertices total, grouped in pairs).

Hints:

- GL_LINES groups vertices in pairs — vertices 0-1 form segment one, vertices 2-3 form segment two.
- Make sure the two segments don't visually overlap so both are clearly visible.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// 4 vertices grouped in pairs: vertices 0-1 form segment one, 2-3 form segment
two[cite: 5]
GLfloat lineVertices[] = {
    -0.6f,  0.3f,  // Vertex 0: Segment 1 start
     0.6f,  0.3f,  // Vertex 1: Segment 1 end
    -0.6f, -0.3f,  // Vertex 2: Segment 2 start
     0.6f, -0.3f   // Vertex 3: Segment 2 end
};

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glLineWidth(4.0f);
    glColor3f(0.0f, 0.8f, 1.0f);

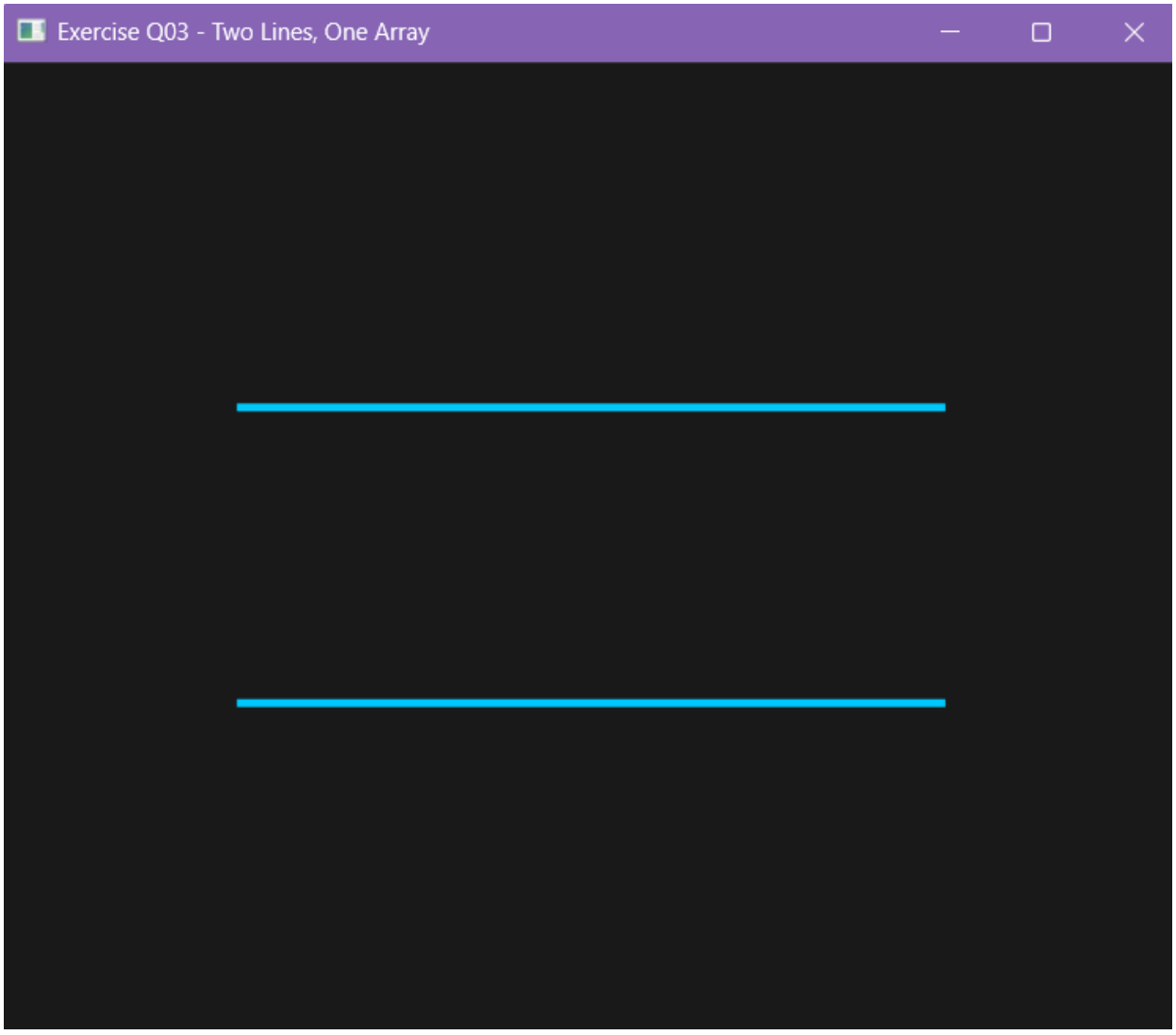
    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(2, GL_FLOAT, 0, lineVertices);
    glDrawArrays(GL_LINES, 0, 4); // Draw 4 vertices as individual line
pairs[cite: 5]
    glDisableClientState(GL_VERTEX_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
```

```
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Exercise Q03 - Two Lines, One Array");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q04 — Square Plus Outline, Same Array [Easy]

Objective: Practice reusing ONE vertex array across TWO different glDrawArrays calls.

Task (Question): Convert an immediate-mode filled square into a vertex array using GL_QUADS, then ALSO draw its outline using GL_LINE_LOOP with the SAME array (two draw calls, one array).

Hints:

- You only need to call glEnableClientState/glVertexPointer ONCE if both draw calls happen back to back.
- Change color between the two glDrawArrays calls so the outline is visible against the fill.

CODE:

```
#ifndef  __FREEGLUT_STD_H__
#define  __FREEGLUT_STD_H__

/*
 * freeglut_std.h
 *
 * The GLUT-compatible part of the freeglut library include file
 *
 * Copyright (c) 1999-2000 Pawel W. Olszta. All Rights Reserved.
 * Written by Pawel W. Olszta, <olszta@sourceforge.net>
 * Creation date: Thu Dec 2 1999
 *
 * Permission is hereby granted, free of charge, to any person obtaining a
 * copy of this software and associated documentation files (the "Software"),
 * to deal in the Software without restriction, including without limitation
 * the rights to use, copy, modify, merge, publish, distribute, sublicense,
 * and/or sell copies of the Software, and to permit persons to whom the
 * Software is furnished to do so, subject to the following conditions:
 *
 * The above copyright notice and this permission notice shall be included
 * in all copies or substantial portions of the Software.
 *
 * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
```

```
 * OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
 * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.  IN NO EVENT SHALL
 * PAWEL W. OLSZTA BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER
 * IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
 * CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
 */

#ifdef __cplusplus
    extern "C" {
#endif

/*
 * Under windows, we have to differentiate between static and dynamic libraries
 */
#ifdef _WIN32
/* #pragma may not be supported by some compilers.
 * Discussion by FreeGLUT developers suggests that
 * Visual C++ specific code involving pragmas may
 * need to move to a separate header.  24th Dec 2003
 */

/* Define FREEGLUT_LIB_PRAGMAS to 1 to include library
 * pragmas or to 0 to exclude library pragmas.
 * The default behavior depends on the compiler/platform.
 */
#   ifndef FREEGLUT_LIB_PRAGMAS
#       if ( defined(_MSC_VER) || defined(__WATCOMC__) ) && !defined(_WIN32_WCE)
#           define FREEGLUT_LIB_PRAGMAS 1
#       else
#           define FREEGLUT_LIB_PRAGMAS 0
#       endif
#   endif

#  ifndef WIN32_LEAN_AND_MEAN
#    define WIN32_LEAN_AND_MEAN 1
#  endif
#  ifndef NOMINMAX
#    define NOMINMAX
#  endif
```

```
#   include <windows.h>

/* Windows static library */
#   ifdef FREEGLUT_STATIC

#       define FGAPI
#       define FGAPIENTRY

        /* Link with Win32 static freeglut lib */
#       if FREEGLUT_LIB_PRAGMAS
#           ifdef NDEBUG
#               pragma comment (lib, "freeglut_static.lib")
#           else
#               pragma comment (lib, "freeglut_staticd.lib")
#           endif
#       endif

/* Windows shared library (DLL) */
#   else

#       define FGAPIENTRY __stdcall
#       if defined(FREEGLUT_EXPORTS)
#           define FGAPI __declspec(dllexport)
#       else
#           define FGAPI __declspec(dllimport)

            /* Link with Win32 shared freeglut lib */
#           if FREEGLUT_LIB_PRAGMAS
#               ifdef NDEBUG
#                   pragma comment (lib, "freeglut.lib")
#               else
#                   pragma comment (lib, "freeglutd.lib")
#               endif
#           endif

#       endif

#   endif
```

```
/* Drag in other Windows libraries as required by FreeGLUT */
#   if FREEGLUT_LIB_PRAGMAS
#       pragma comment (lib, "glu32.lib")    /* link OpenGL Utility lib     */
#       pragma comment (lib, "opengl32.lib") /* link Microsoft OpenGL lib   */
#       pragma comment (lib, "gdi32.lib")    /* link Windows GDI lib        */
#       pragma comment (lib, "winmm.lib")    /* link Windows MultiMedia lib */
#       pragma comment (lib, "user32.lib")   /* link Windows user lib       */
#   endif

#else

/* Non-Windows definition of FGAPI and FGAPIENTRY  */
#        define FGAPI
#        define FGAPIENTRY

#endif

/*
 * The freeglut and GLUT API versions
 */
#define  FREEGLUT             1
#define  GLUT_API_VERSION     4
#define  GLUT_XLIB_IMPLEMENTATION 13
/* Deprecated:
   cf.
http://sourceforge.net/mailarchive/forum.php?thread_name=CABcAi1hw7cr4xtigckaGXB
5X8wddLfMcbA_rZ3NAuwMrX_zmsw%40mail.gmail.com&forum_name=freeglut-developer */
#define  FREEGLUT_VERSION_2_0 1

/*
 * Include OpenGL and GLU headers unless explicitly excluded
(FREEGLUT_NO_GL_INCLUDE).
 *
 * On some platforms, such as macOS, OpenGL headers are split:
 *   - <OpenGL/gl.h> for the compatibility profile (OpenGL 2.1 and earlier).
 *   - <OpenGL/gl3.h> for the core profile (OpenGL 3.2+).
 *
 *  For example to use the core profile on macOS, you can do:
 *
```

```
 *    #ifdef __APPLE__
 *    #define FREEGLUT_NO_GL_INCLUDE
 *    #include <OpenGL/gl3.h>
 *    #endif
 *
 *    #include <GL/freeglut.h>
 */
#ifndef FREEGLUT_NO_GL_INCLUDE
/* Note: FREEGLUT_GLES is only used to cleanly bootstrap headers
   inclusion here; use GLES constants directly
   (e.g. GL_ES_VERSION_2_0) for all other needs */
#ifdef FREEGLUT_GLES
#   include <EGL/egl.h>
#   include <GLES/gl.h>
#   include <GLES2/gl2.h>
#elif __APPLE__
/* stop MacOSX GL headers for complaining that OpenGL is deprecated */
#   ifndef GL_SILENCE_DEPRECATION
#       define GL_SILENCE_DEPRECATION
#   endif
#   include <OpenGL/gl.h>
#   include <OpenGL/glu.h>
#else
#   include <GL/gl.h>
#   include <GL/glu.h>
#endif
#endif /* !defined(FREEGLUT_NO_GL_INCLUDE) */

/*
 * GLUT API macro definitions -- the special key codes:
 */
#define  GLUT_KEY_F1                        0x0001
#define  GLUT_KEY_F2                        0x0002
#define  GLUT_KEY_F3                        0x0003
#define  GLUT_KEY_F4                        0x0004
#define  GLUT_KEY_F5                        0x0005
#define  GLUT_KEY_F6                        0x0006
#define  GLUT_KEY_F7                        0x0007
#define  GLUT_KEY_F8                        0x0008
```

```
#define  GLUT_KEY_F9                        0x0009
#define  GLUT_KEY_F10                       0x000A
#define  GLUT_KEY_F11                       0x000B
#define  GLUT_KEY_F12                       0x000C
#define  GLUT_KEY_LEFT                      0x0064
#define  GLUT_KEY_UP                        0x0065
#define  GLUT_KEY_RIGHT                     0x0066
#define  GLUT_KEY_DOWN                      0x0067
#define  GLUT_KEY_PAGE_UP                   0x0068
#define  GLUT_KEY_PAGE_DOWN                 0x0069
#define  GLUT_KEY_HOME                      0x006A
#define  GLUT_KEY_END                       0x006B
#define  GLUT_KEY_INSERT                    0x006C

/*
 * GLUT API macro definitions -- mouse state definitions
 */
#define  GLUT_LEFT_BUTTON                   0x0000
#define  GLUT_MIDDLE_BUTTON                 0x0001
#define  GLUT_RIGHT_BUTTON                  0x0002
#define  GLUT_DOWN                          0x0000
#define  GLUT_UP                            0x0001
#define  GLUT_LEFT                          0x0000
#define  GLUT_ENTERED                       0x0001

/*
 * GLUT API macro definitions -- the display mode definitions
 */
#define  GLUT_RGB                           0x0000
#define  GLUT_RGBA                          0x0000
#define  GLUT_INDEX                         0x0001
#define  GLUT_SINGLE                        0x0000
#define  GLUT_DOUBLE                        0x0002
#define  GLUT_ACCUM                         0x0004
#define  GLUT_ALPHA                         0x0008
#define  GLUT_DEPTH                         0x0010
#define  GLUT_STENCIL                       0x0020
#define  GLUT_MULTISAMPLE                   0x0080
#define  GLUT_STEREO                        0x0100
```

```
#define  GLUT_LUMINANCE                     0x0200

/*
 * GLUT API macro definitions -- windows and menu related definitions
 */
#define  GLUT_MENU_NOT_IN_USE               0x0000
#define  GLUT_MENU_IN_USE                   0x0001
#define  GLUT_NOT_VISIBLE                   0x0000
#define  GLUT_VISIBLE                       0x0001
#define  GLUT_HIDDEN                        0x0000
#define  GLUT_FULLY_RETAINED                0x0001
#define  GLUT_PARTIALLY_RETAINED            0x0002
#define  GLUT_FULLY_COVERED                 0x0003

/*
 * GLUT API macro definitions -- fonts definitions
 *
 * Steve Baker suggested to make it binary compatible with GLUT:
 */
#if defined(_MSC_VER) || defined(__CYGWIN__) || defined(__MINGW32__) ||
defined(__WATCOMC__) || defined(__DEVKITPRO__)
#   define  GLUT_STROKE_ROMAN               ((void *)0x0000)
#   define  GLUT_STROKE_MONO_ROMAN          ((void *)0x0001)
#   define  GLUT_BITMAP_9_BY_15             ((void *)0x0002)
#   define  GLUT_BITMAP_8_BY_13             ((void *)0x0003)
#   define  GLUT_BITMAP_TIMES_ROMAN_10      ((void *)0x0004)
#   define  GLUT_BITMAP_TIMES_ROMAN_24      ((void *)0x0005)
#   define  GLUT_BITMAP_HELVETICA_10        ((void *)0x0006)
#   define  GLUT_BITMAP_HELVETICA_12        ((void *)0x0007)
#   define  GLUT_BITMAP_HELVETICA_18        ((void *)0x0008)
#else
    /*
     * I don't really know if it's a good idea... But here it goes:
     */
    extern void* glutStrokeRoman;
    extern void* glutStrokeMonoRoman;
    extern void* glutBitmap9By15;
    extern void* glutBitmap8By13;
    extern void* glutBitmapTimesRoman10;
```

```
    extern void* glutBitmapTimesRoman24;
    extern void* glutBitmapHelvetica10;
    extern void* glutBitmapHelvetica12;
    extern void* glutBitmapHelvetica18;

    /*
     * Those pointers will be used by following definitions:
     */
#   define  GLUT_STROKE_ROMAN               ((void *) &glutStrokeRoman)
#   define  GLUT_STROKE_MONO_ROMAN          ((void *) &glutStrokeMonoRoman)
#   define  GLUT_BITMAP_9_BY_15             ((void *) &glutBitmap9By15)
#   define  GLUT_BITMAP_8_BY_13             ((void *) &glutBitmap8By13)
#   define  GLUT_BITMAP_TIMES_ROMAN_10      ((void *) &glutBitmapTimesRoman10)
#   define  GLUT_BITMAP_TIMES_ROMAN_24      ((void *) &glutBitmapTimesRoman24)
#   define  GLUT_BITMAP_HELVETICA_10        ((void *) &glutBitmapHelvetica10)
#   define  GLUT_BITMAP_HELVETICA_12        ((void *) &glutBitmapHelvetica12)
#   define  GLUT_BITMAP_HELVETICA_18        ((void *) &glutBitmapHelvetica18)
#endif

/*
 * GLUT API macro definitions -- the glutGet parameters
 */
#define  GLUT_WINDOW_X                      0x0064
#define  GLUT_WINDOW_Y                      0x0065
#define  GLUT_WINDOW_WIDTH                  0x0066
#define  GLUT_WINDOW_HEIGHT                 0x0067
#define  GLUT_WINDOW_BUFFER_SIZE            0x0068
#define  GLUT_WINDOW_STENCIL_SIZE           0x0069
#define  GLUT_WINDOW_DEPTH_SIZE             0x006A
#define  GLUT_WINDOW_RED_SIZE               0x006B
#define  GLUT_WINDOW_GREEN_SIZE             0x006C
#define  GLUT_WINDOW_BLUE_SIZE              0x006D
#define  GLUT_WINDOW_ALPHA_SIZE             0x006E
#define  GLUT_WINDOW_ACCUM_RED_SIZE         0x006F
#define  GLUT_WINDOW_ACCUM_GREEN_SIZE       0x0070
#define  GLUT_WINDOW_ACCUM_BLUE_SIZE        0x0071
#define  GLUT_WINDOW_ACCUM_ALPHA_SIZE       0x0072
#define  GLUT_WINDOW_DOUBLEBUFFER           0x0073
#define  GLUT_WINDOW_RGBA                   0x0074
```

```
#define  GLUT_WINDOW_PARENT                 0x0075
#define  GLUT_WINDOW_NUM_CHILDREN           0x0076
#define  GLUT_WINDOW_COLORMAP_SIZE          0x0077
#define  GLUT_WINDOW_NUM_SAMPLES            0x0078
#define  GLUT_WINDOW_STEREO                 0x0079
#define  GLUT_WINDOW_CURSOR                 0x007A

#define  GLUT_SCREEN_WIDTH                  0x00C8
#define  GLUT_SCREEN_HEIGHT                 0x00C9
#define  GLUT_SCREEN_WIDTH_MM               0x00CA
#define  GLUT_SCREEN_HEIGHT_MM              0x00CB
#define  GLUT_MENU_NUM_ITEMS                0x012C
#define  GLUT_DISPLAY_MODE_POSSIBLE         0x0190
#define  GLUT_INIT_WINDOW_X                 0x01F4
#define  GLUT_INIT_WINDOW_Y                 0x01F5
#define  GLUT_INIT_WINDOW_WIDTH             0x01F6
#define  GLUT_INIT_WINDOW_HEIGHT            0x01F7
#define  GLUT_INIT_DISPLAY_MODE             0x01F8
#define  GLUT_ELAPSED_TIME                  0x02BC
#define  GLUT_WINDOW_FORMAT_ID              0x007B

/*
 * GLUT API macro definitions -- the glutDeviceGet parameters
 */
#define  GLUT_HAS_KEYBOARD                  0x0258
#define  GLUT_HAS_MOUSE                     0x0259
#define  GLUT_HAS_SPACEBALL                 0x025A
#define  GLUT_HAS_DIAL_AND_BUTTON_BOX       0x025B
#define  GLUT_HAS_TABLET                    0x025C
#define  GLUT_NUM_MOUSE_BUTTONS             0x025D
#define  GLUT_NUM_SPACEBALL_BUTTONS         0x025E
#define  GLUT_NUM_BUTTON_BOX_BUTTONS        0x025F
#define  GLUT_NUM_DIALS                     0x0260
#define  GLUT_NUM_TABLET_BUTTONS            0x0261
#define  GLUT_DEVICE_IGNORE_KEY_REPEAT      0x0262
#define  GLUT_DEVICE_KEY_REPEAT             0x0263
#define  GLUT_HAS_JOYSTICK                  0x0264
#define  GLUT_OWNS_JOYSTICK                 0x0265
#define  GLUT_JOYSTICK_BUTTONS              0x0266
```

```
#define  GLUT_JOYSTICK_AXES                 0x0267
#define  GLUT_JOYSTICK_POLL_RATE            0x0268

/*
 * GLUT API macro definitions -- the glutLayerGet parameters
 */
#define  GLUT_OVERLAY_POSSIBLE              0x0320
#define  GLUT_LAYER_IN_USE                  0x0321
#define  GLUT_HAS_OVERLAY                   0x0322
#define  GLUT_TRANSPARENT_INDEX             0x0323
#define  GLUT_NORMAL_DAMAGED                0x0324
#define  GLUT_OVERLAY_DAMAGED               0x0325

/*
 * GLUT API macro definitions -- the glutVideoResizeGet parameters
 */
#define  GLUT_VIDEO_RESIZE_POSSIBLE         0x0384
#define  GLUT_VIDEO_RESIZE_IN_USE           0x0385
#define  GLUT_VIDEO_RESIZE_X_DELTA          0x0386
#define  GLUT_VIDEO_RESIZE_Y_DELTA          0x0387
#define  GLUT_VIDEO_RESIZE_WIDTH_DELTA      0x0388
#define  GLUT_VIDEO_RESIZE_HEIGHT_DELTA     0x0389
#define  GLUT_VIDEO_RESIZE_X                0x038A
#define  GLUT_VIDEO_RESIZE_Y                0x038B
#define  GLUT_VIDEO_RESIZE_WIDTH            0x038C
#define  GLUT_VIDEO_RESIZE_HEIGHT           0x038D

/*
 * GLUT API macro definitions -- the glutUseLayer parameters
 */
#define  GLUT_NORMAL                        0x0000
#define  GLUT_OVERLAY                       0x0001

/*
 * GLUT API macro definitions -- the glutGetModifiers parameters
 */
#define  GLUT_ACTIVE_SHIFT                  0x0001
#define  GLUT_ACTIVE_CTRL                   0x0002
#define  GLUT_ACTIVE_ALT                    0x0004
```

```
/*
 * GLUT API macro definitions -- the glutSetCursor parameters
 */
#define  GLUT_CURSOR_RIGHT_ARROW            0x0000
#define  GLUT_CURSOR_LEFT_ARROW             0x0001
#define  GLUT_CURSOR_INFO                   0x0002
#define  GLUT_CURSOR_DESTROY                0x0003
#define  GLUT_CURSOR_HELP                   0x0004
#define  GLUT_CURSOR_CYCLE                  0x0005
#define  GLUT_CURSOR_SPRAY                  0x0006
#define  GLUT_CURSOR_WAIT                   0x0007
#define  GLUT_CURSOR_TEXT                   0x0008
#define  GLUT_CURSOR_CROSSHAIR              0x0009
#define  GLUT_CURSOR_UP_DOWN                0x000A
#define  GLUT_CURSOR_LEFT_RIGHT             0x000B
#define  GLUT_CURSOR_TOP_SIDE               0x000C
#define  GLUT_CURSOR_BOTTOM_SIDE            0x000D
#define  GLUT_CURSOR_LEFT_SIDE              0x000E
#define  GLUT_CURSOR_RIGHT_SIDE             0x000F
#define  GLUT_CURSOR_TOP_LEFT_CORNER        0x0010
#define  GLUT_CURSOR_TOP_RIGHT_CORNER       0x0011
#define  GLUT_CURSOR_BOTTOM_RIGHT_CORNER    0x0012
#define  GLUT_CURSOR_BOTTOM_LEFT_CORNER     0x0013
#define  GLUT_CURSOR_INHERIT                0x0064
#define  GLUT_CURSOR_NONE                   0x0065
#define  GLUT_CURSOR_FULL_CROSSHAIR         0x0066

/*
 * GLUT API macro definitions -- RGB color component specification definitions
 */
#define  GLUT_RED                           0x0000
#define  GLUT_GREEN                         0x0001
#define  GLUT_BLUE                          0x0002

/*
 * GLUT API macro definitions -- additional keyboard and joystick definitions
 */
#define  GLUT_KEY_REPEAT_OFF                0x0000
```

```
#define  GLUT_KEY_REPEAT_ON                 0x0001
#define  GLUT_KEY_REPEAT_DEFAULT            0x0002

#define  GLUT_JOYSTICK_BUTTON_A             0x0001
#define  GLUT_JOYSTICK_BUTTON_B             0x0002
#define  GLUT_JOYSTICK_BUTTON_C             0x0004
#define  GLUT_JOYSTICK_BUTTON_D             0x0008

/*
 * GLUT API macro definitions -- game mode definitions
 */
#define  GLUT_GAME_MODE_ACTIVE              0x0000
#define  GLUT_GAME_MODE_POSSIBLE            0x0001
#define  GLUT_GAME_MODE_WIDTH               0x0002
#define  GLUT_GAME_MODE_HEIGHT              0x0003
#define  GLUT_GAME_MODE_PIXEL_DEPTH         0x0004
#define  GLUT_GAME_MODE_REFRESH_RATE        0x0005
#define  GLUT_GAME_MODE_DISPLAY_CHANGED     0x0006

/*
 * Initialization functions, see fglut_init.c
 */
FGAPI void    FGAPIENTRY glutInit( int* pargc, char** argv );
FGAPI void    FGAPIENTRY glutInitWindowPosition( int x, int y );
FGAPI void    FGAPIENTRY glutInitWindowSize( int width, int height );
FGAPI void    FGAPIENTRY glutInitDisplayMode( unsigned int displayMode );
FGAPI void    FGAPIENTRY glutInitDisplayString( const char* displayMode );

/*
 * Process loop function, see fg_main.c
 */
FGAPI void    FGAPIENTRY glutMainLoop( void );

/*
 * Window management functions, see fg_window.c
 */
FGAPI int     FGAPIENTRY glutCreateWindow( const char* title );
FGAPI int     FGAPIENTRY glutCreateSubWindow( int window, int x, int y, int
width, int height );
```

```
FGAPI void    FGAPIENTRY glutDestroyWindow( int window );
FGAPI void    FGAPIENTRY glutSetWindow( int window );
FGAPI int     FGAPIENTRY glutGetWindow( void );
FGAPI void    FGAPIENTRY glutSetWindowTitle( const char* title );
FGAPI void    FGAPIENTRY glutSetIconTitle( const char* title );
FGAPI void    FGAPIENTRY glutReshapeWindow( int width, int height );
FGAPI void    FGAPIENTRY glutPositionWindow( int x, int y );
FGAPI void    FGAPIENTRY glutShowWindow( void );
FGAPI void    FGAPIENTRY glutHideWindow( void );
FGAPI void    FGAPIENTRY glutIconifyWindow( void );
FGAPI void    FGAPIENTRY glutPushWindow( void );
FGAPI void    FGAPIENTRY glutPopWindow( void );
FGAPI void    FGAPIENTRY glutFullScreen( void );

/*
 * Display-related functions, see fg_display.c
 */
FGAPI void    FGAPIENTRY glutPostWindowRedisplay( int window );
FGAPI void    FGAPIENTRY glutPostRedisplay( void );
FGAPI void    FGAPIENTRY glutSwapBuffers( void );

/*
 * Mouse cursor functions, see fg_cursor.c
 */
FGAPI void    FGAPIENTRY glutWarpPointer( int x, int y );
FGAPI void    FGAPIENTRY glutSetCursor( int cursor );

/*
 * Overlay stuff, see fg_overlay.c
 */
FGAPI void    FGAPIENTRY glutEstablishOverlay( void );
FGAPI void    FGAPIENTRY glutRemoveOverlay( void );
FGAPI void    FGAPIENTRY glutUseLayer( GLenum layer );
FGAPI void    FGAPIENTRY glutPostOverlayRedisplay( void );
FGAPI void    FGAPIENTRY glutPostWindowOverlayRedisplay( int window );
FGAPI void    FGAPIENTRY glutShowOverlay( void );
FGAPI void    FGAPIENTRY glutHideOverlay( void );

/*
```

```
 * Menu stuff, see fg_menu.c
 */
FGAPI int     FGAPIENTRY glutCreateMenu( void (* callback)( int menu ) );
FGAPI void    FGAPIENTRY glutDestroyMenu( int menu );
FGAPI int     FGAPIENTRY glutGetMenu( void );
FGAPI void    FGAPIENTRY glutSetMenu( int menu );
FGAPI void    FGAPIENTRY glutAddMenuEntry( const char* label, int value );
FGAPI void    FGAPIENTRY glutAddSubMenu( const char* label, int subMenu );
FGAPI void    FGAPIENTRY glutChangeToMenuEntry( int item, const char* label, int
value );
FGAPI void    FGAPIENTRY glutChangeToSubMenu( int item, const char* label, int
value );
FGAPI void    FGAPIENTRY glutRemoveMenuItem( int item );
FGAPI void    FGAPIENTRY glutAttachMenu( int button );
FGAPI void    FGAPIENTRY glutDetachMenu( int button );

/*
 * Global callback functions, see fg_callbacks.c
 */
FGAPI void    FGAPIENTRY glutTimerFunc( unsigned int time, void (* callback)(
int ), int value );
FGAPI void    FGAPIENTRY glutIdleFunc( void (* callback)( void ) );

/*
 * Window-specific callback functions, see fg_callbacks.c
 */
FGAPI void    FGAPIENTRY glutKeyboardFunc( void (* callback)( unsigned char,
int, int ) );
FGAPI void    FGAPIENTRY glutSpecialFunc( void (* callback)( int, int, int ) );
FGAPI void    FGAPIENTRY glutReshapeFunc( void (* callback)( int, int ) );
FGAPI void    FGAPIENTRY glutVisibilityFunc( void (* callback)( int ) );
FGAPI void    FGAPIENTRY glutDisplayFunc( void (* callback)( void ) );
FGAPI void    FGAPIENTRY glutMouseFunc( void (* callback)( int, int, int, int )
);
FGAPI void    FGAPIENTRY glutMotionFunc( void (* callback)( int, int ) );
FGAPI void    FGAPIENTRY glutPassiveMotionFunc( void (* callback)( int, int ) );
FGAPI void    FGAPIENTRY glutEntryFunc( void (* callback)( int ) );

FGAPI void    FGAPIENTRY glutKeyboardUpFunc( void (* callback)( unsigned char,
```

```
int, int ) );
FGAPI void    FGAPIENTRY glutSpecialUpFunc( void (* callback)( int, int, int )
);
FGAPI void    FGAPIENTRY glutJoystickFunc( void (* callback)( unsigned int, int,
int, int ), int pollInterval );
FGAPI void    FGAPIENTRY glutMenuStateFunc( void (* callback)( int ) );
FGAPI void    FGAPIENTRY glutMenuStatusFunc( void (* callback)( int, int, int )
);
FGAPI void    FGAPIENTRY glutOverlayDisplayFunc( void (* callback)( void ) );
FGAPI void    FGAPIENTRY glutWindowStatusFunc( void (* callback)( int ) );

FGAPI void    FGAPIENTRY glutSpaceballMotionFunc( void (* callback)( int, int,
int ) );
FGAPI void    FGAPIENTRY glutSpaceballRotateFunc( void (* callback)( int, int,
int ) );
FGAPI void    FGAPIENTRY glutSpaceballButtonFunc( void (* callback)( int, int )
);
FGAPI void    FGAPIENTRY glutButtonBoxFunc( void (* callback)( int, int ) );
FGAPI void    FGAPIENTRY glutDialsFunc( void (* callback)( int, int ) );
FGAPI void    FGAPIENTRY glutTabletMotionFunc( void (* callback)( int, int ) );
FGAPI void    FGAPIENTRY glutTabletButtonFunc( void (* callback)( int, int, int,
int ) );

/*
 * State setting and retrieval functions, see fg_state.c
 */
FGAPI int     FGAPIENTRY glutGet( GLenum query );
FGAPI int     FGAPIENTRY glutDeviceGet( GLenum query );
FGAPI int     FGAPIENTRY glutGetModifiers( void );
FGAPI int     FGAPIENTRY glutLayerGet( GLenum query );

/*
 * Font stuff, see fg_font.c
 */
FGAPI void    FGAPIENTRY glutBitmapCharacter( void* font, int character );
FGAPI int     FGAPIENTRY glutBitmapWidth( void* font, int character );
FGAPI void    FGAPIENTRY glutStrokeCharacter( void* font, int character );
FGAPI int     FGAPIENTRY glutStrokeWidth( void* font, int character );
FGAPI GLfloat FGAPIENTRY glutStrokeWidthf( void* font, int character ); /* GLUT
```

```
3.8 */
FGAPI int     FGAPIENTRY glutBitmapLength( void* font, const unsigned char*
string );
FGAPI int     FGAPIENTRY glutStrokeLength( void* font, const unsigned char*
string );
FGAPI GLfloat FGAPIENTRY glutStrokeLengthf( void* font, const unsigned char
*string ); /* GLUT 3.8 */

/*
 * Geometry functions, see fg_geometry.c
 */

FGAPI void    FGAPIENTRY glutWireCube( double size );
FGAPI void    FGAPIENTRY glutSolidCube( double size );
FGAPI void    FGAPIENTRY glutWireSphere( double radius, GLint slices, GLint
stacks );
FGAPI void    FGAPIENTRY glutSolidSphere( double radius, GLint slices, GLint
stacks );
FGAPI void    FGAPIENTRY glutWireCone( double base, double height, GLint slices,
GLint stacks );
FGAPI void    FGAPIENTRY glutSolidCone( double base, double height, GLint
slices, GLint stacks );
FGAPI void    FGAPIENTRY glutWireTorus( double innerRadius, double outerRadius,
GLint sides, GLint rings );
FGAPI void    FGAPIENTRY glutSolidTorus( double innerRadius, double outerRadius,
GLint sides, GLint rings );
FGAPI void    FGAPIENTRY glutWireDodecahedron( void );
FGAPI void    FGAPIENTRY glutSolidDodecahedron( void );
FGAPI void    FGAPIENTRY glutWireOctahedron( void );
FGAPI void    FGAPIENTRY glutSolidOctahedron( void );
FGAPI void    FGAPIENTRY glutWireTetrahedron( void );
FGAPI void    FGAPIENTRY glutSolidTetrahedron( void );
FGAPI void    FGAPIENTRY glutWireIcosahedron( void );
FGAPI void    FGAPIENTRY glutSolidIcosahedron( void );

/*
 * Teapot rendering functions, found in fg_teapot.c
 * NB: front facing polygons have clockwise winding, not counter clockwise
 */
```

```
FGAPI void    FGAPIENTRY glutWireTeapot( double size );
FGAPI void    FGAPIENTRY glutSolidTeapot( double size );

/*
 * Game mode functions, see fg_gamemode.c
 */
FGAPI void    FGAPIENTRY glutGameModeString( const char* string );
FGAPI int     FGAPIENTRY glutEnterGameMode( void );
FGAPI void    FGAPIENTRY glutLeaveGameMode( void );
FGAPI int     FGAPIENTRY glutGameModeGet( GLenum query );

/*
 * Video resize functions, see fg_videoresize.c
 */
FGAPI int     FGAPIENTRY glutVideoResizeGet( GLenum query );
FGAPI void    FGAPIENTRY glutSetupVideoResizing( void );
FGAPI void    FGAPIENTRY glutStopVideoResizing( void );
FGAPI void    FGAPIENTRY glutVideoResize( int x, int y, int width, int height );
FGAPI void    FGAPIENTRY glutVideoPan( int x, int y, int width, int height );

/*
 * Colormap functions, see fg_misc.c
 */
FGAPI void    FGAPIENTRY glutSetColor( int color, GLfloat red, GLfloat green,
GLfloat blue );
FGAPI GLfloat FGAPIENTRY glutGetColor( int color, int component );
FGAPI void    FGAPIENTRY glutCopyColormap( int window );

/*
 * Misc keyboard and joystick functions, see fg_misc.c
 */
FGAPI void    FGAPIENTRY glutIgnoreKeyRepeat( int ignore );
FGAPI void    FGAPIENTRY glutSetKeyRepeat( int repeatMode );
FGAPI void    FGAPIENTRY glutForceJoystickFunc( void );

/*
 * Misc functions, see fg_misc.c
 */
FGAPI int     FGAPIENTRY glutExtensionSupported( const char* extension );
```

```
FGAPI void    FGAPIENTRY glutReportErrors( void );

/* Comment from glut.h of classic GLUT:

   Win32 has an annoying issue where there are multiple C run-time
   libraries (CRTs).  If the executable is linked with a different CRT
   from the GLUT DLL, the GLUT DLL will not share the same CRT static
   data seen by the executable.  In particular, atexit callbacks registered
   in the executable will not be called if GLUT calls its (different)
   exit routine).  GLUT is typically built with the
   "/MD" option (the CRT with multithreading DLL support), but the Visual
   C++ linker default is "/ML" (the single threaded CRT).

   One workaround to this issue is requiring users to always link with
   the same CRT as GLUT is compiled with.  That requires users supply a
   non-standard option.  GLUT 3.7 has its own built-in workaround where
   the executable's "exit" function pointer is covertly passed to GLUT.
   GLUT then calls the executable's exit function pointer to ensure that
   any "atexit" calls registered by the application are called if GLUT
   needs to exit.

   Note that the __glut*WithExit routines should NEVER be called directly.
   To avoid the atexit workaround, #define GLUT_DISABLE_ATEXIT_HACK. */

/* to get the prototype for exit() */
#include <stdlib.h>

#if defined(_WIN32) && !defined(GLUT_DISABLE_ATEXIT_HACK) &&
!defined(__WATCOMC__)
FGAPI void FGAPIENTRY __glutInitWithExit(int *argcp, char **argv, void (__cdecl
*exitfunc)(int));
FGAPI int FGAPIENTRY __glutCreateWindowWithExit(const char *title, void (__cdecl
*exitfunc)(int));
FGAPI int FGAPIENTRY __glutCreateMenuWithExit(void (* func)(int), void (__cdecl
*exitfunc)(int));
#ifndef FREEGLUT_BUILDING_LIB
#if defined(__GNUC__)
#define FGUNUSED __attribute__((unused))
#else
```

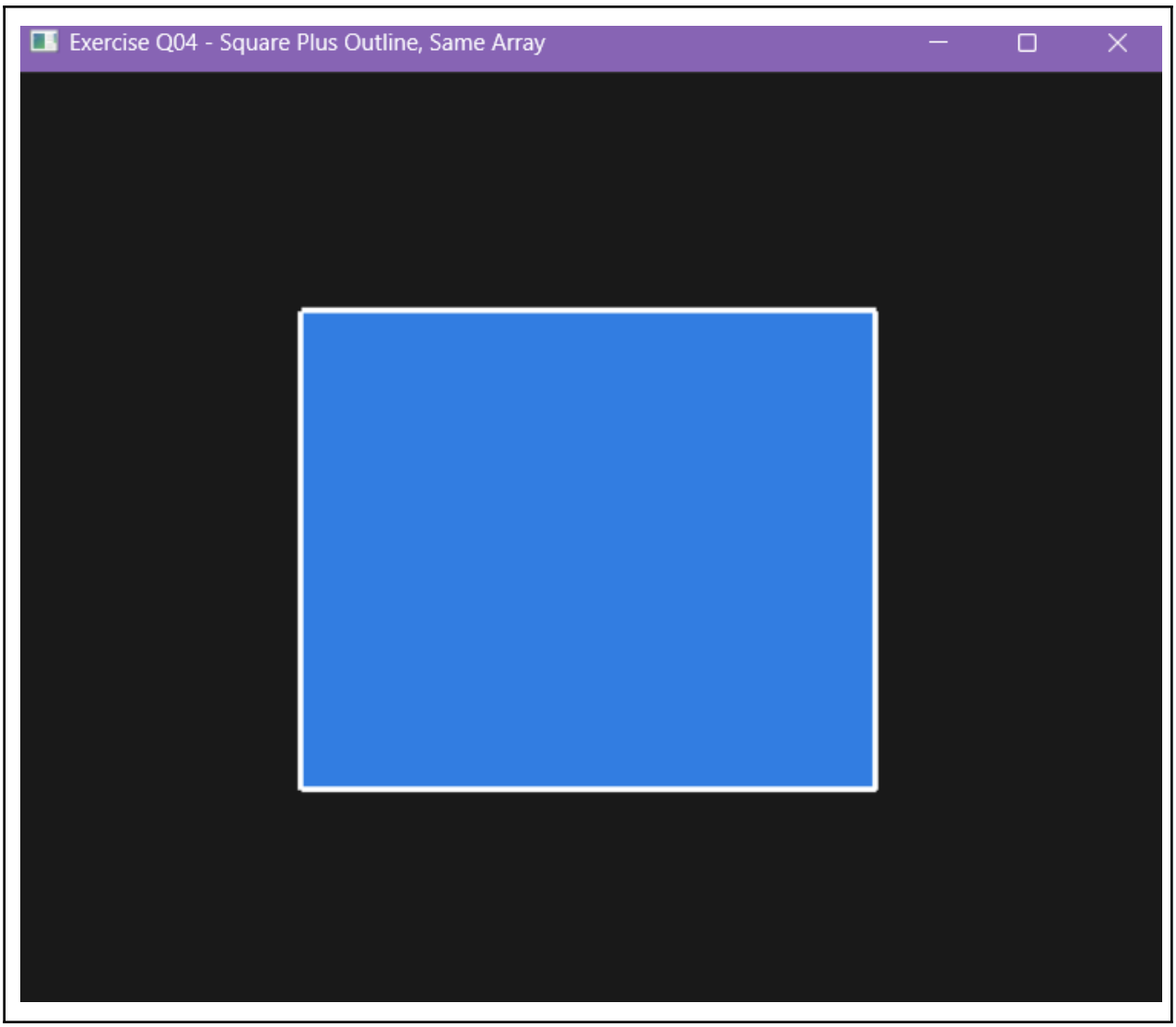
```
#define FGUNUSED
#endif
static void FGAPIENTRY FGUNUSED glutInit_ATEXIT_HACK(int *argcp, char **argv) {
__glutInitWithExit(argcp, argv, exit); }
#define glutInit glutInit_ATEXIT_HACK
static int FGAPIENTRY FGUNUSED glutCreateWindow_ATEXIT_HACK(const char *title) {
return __glutCreateWindowWithExit(title, exit); }
#define glutCreateWindow glutCreateWindow_ATEXIT_HACK
static int FGAPIENTRY FGUNUSED glutCreateMenu_ATEXIT_HACK(void (* func)(int)) {
return __glutCreateMenuWithExit(func, exit); }
#define glutCreateMenu glutCreateMenu_ATEXIT_HACK
#endif
#endif

#ifdef __cplusplus
    }
#endif

/*** END OF FILE ***/

#endif /* __FREEGLUT_STD_H__ */
```

OUTPUT:



Exercise Q05 — GLint Vertex Data Type [Easy]

Objective: Practice using a non-GLfloat data type with glVertexPointer, per the OpenGL data type table.

Task (Question): Draw a triangle using a vertex array where the coordinates are stored as GLint (not GLfloat), scaled down with glScalef so it still fits within the -1..1 viewport.

Hints:

- The `type` argument to glVertexPointer must be GL_INT to match a GLint[] array.
- Use glPushMatrix/glScalef/glPopMatrix around the draw call to bring large integer coordinates back into range.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// Triangle coordinates stored as GLint (integer values)
GLint intTriangleVertices[] = {
     0,   50, // Top vertex
   -50,  -50, // Bottom-left vertex
    50,  -50  // Bottom-right vertex
};

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1.0f, 0.6f, 0.2f); // Orange fill color

    glPushMatrix();
    // Scale down the large integer coordinates to fit within the -1..1
viewport[cite: 7]
    glScalef(0.01f, 0.01f, 1.0f);

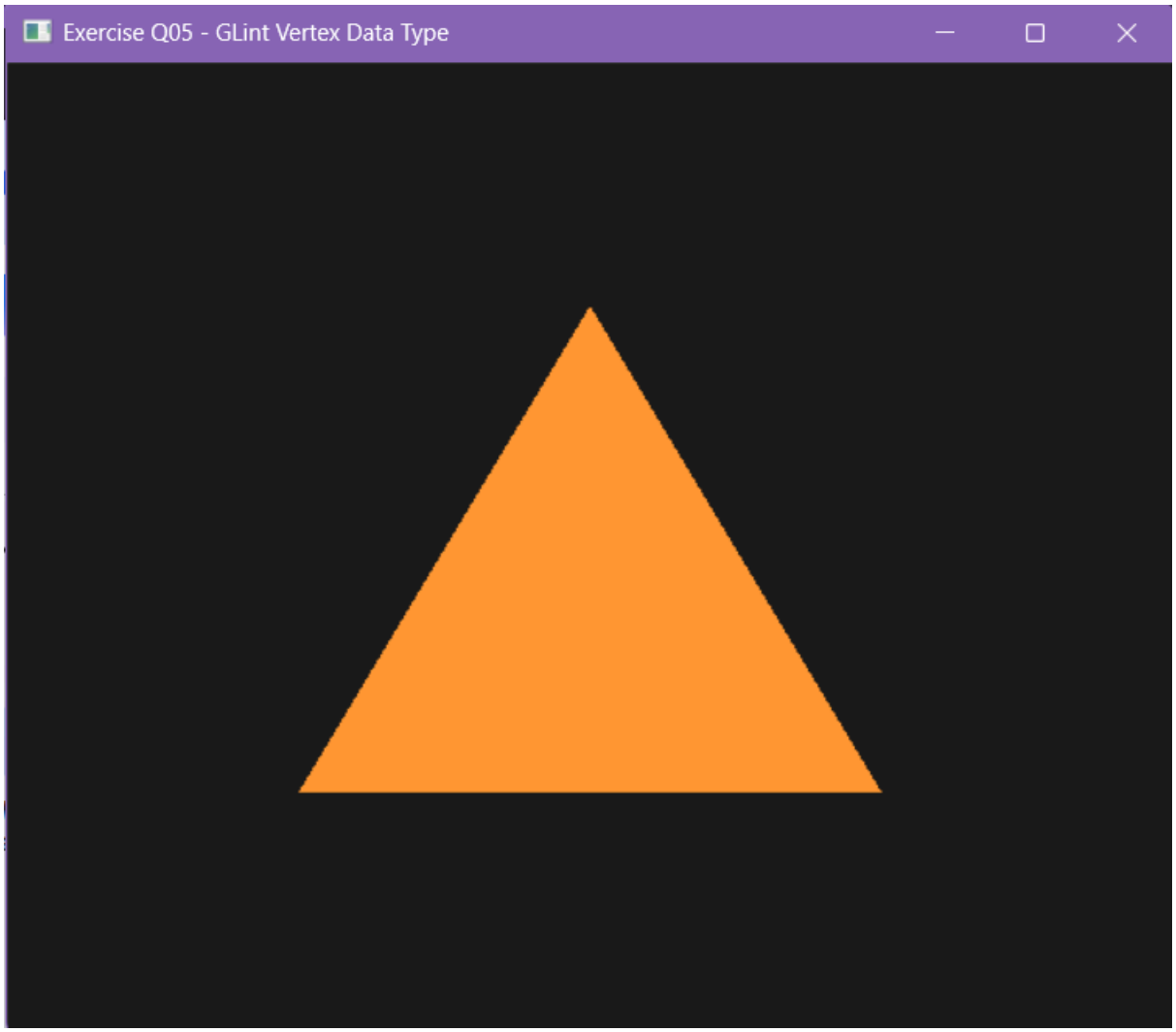
    glEnableClientState(GL_VERTEX_ARRAY);
    // Must use GL_INT to match the GLint[] array[cite: 7]
    glVertexPointer(2, GL_INT, 0, intTriangleVertices);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDisableClientState(GL_VERTEX_ARRAY);

    glPopMatrix();

    glutSwapBuffers();
}
```

```c
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Exercise Q05 - GLint Vertex Data Type");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q06 — glDrawElements Index Order [Easy]

Objective: Practice understanding that the INDEX ARRAY, not the array's storage order, controls draw order.

Task (Question): Draw a single triangle using glDrawElements with an index array, where the index order does NOT simply match the vertex array's storage order (e.g., indices {2, 0, 1} instead of {0, 1, 2}).

Hints:

- The rendered triangle should look identical either way — indices just control the ORDER vertices are visited, not which vertices exist.
- Try printing/commenting the intended winding order so you can verify it matches what you expect.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// 3 unique vertices stored in the array
GLfloat triangleVertices[] = {
    -0.5f, -0.5f, 0.0f, // Index 0: bottom-left
     0.5f, -0.5f, 0.0f, // Index 1: bottom-right
     0.0f,  0.5f, 0.0f  // Index 2: top-center
};

// Index order does NOT match storage order (uses {2, 0, 1} pattern)[cite: 10]
GLubyte triangleIndices[] = { 2, 0, 1 };

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
```

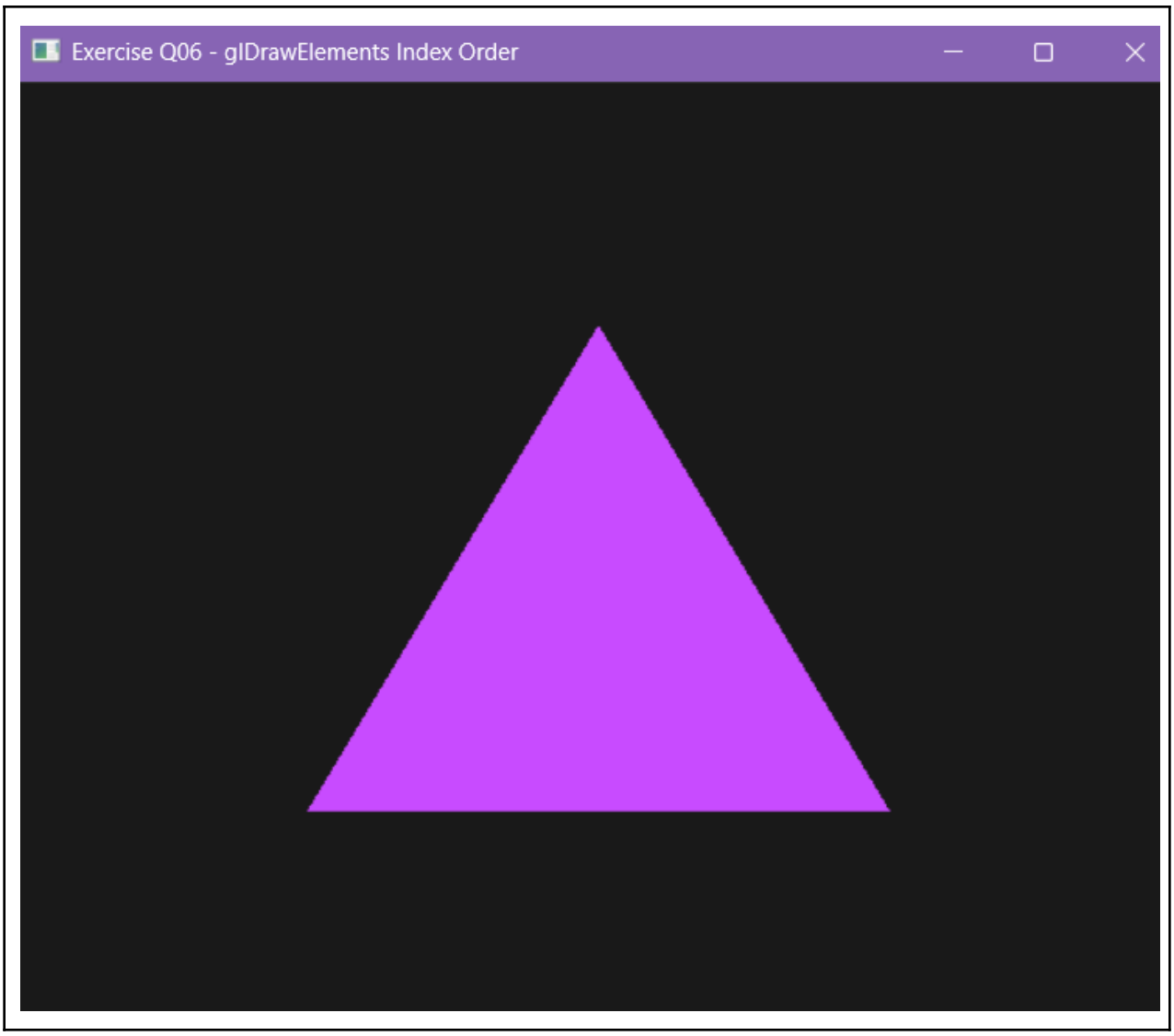
```
    glColor3f(0.8f, 0.3f, 1.0f); // Purple fill color

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, triangleVertices);
    glDrawElements(GL_TRIANGLES, 3, GL_UNSIGNED_BYTE, triangleIndices);
    glDisableClientState(GL_VERTEX_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Exercise Q06 - glDrawElements Index Order");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q07 — Vertex + Color Array Triangle [Easy]

Objective: Practice the basic combination of glVertexPointer and glColorPointer.

Task (Question): Draw a triangle with a DIFFERENT color at each vertex, using a vertex array combined with a color array (not individual glColor3f calls).

Hints:

- Both GL_VERTEX_ARRAY and GL_COLOR_ARRAY must be enabled before the draw call. • The i-th entry in the color array colors the i-th vertex in the vertex array — keep the two arrays in the same order.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// 3 vertices for the triangle
GLfloat triangleVertices[] = {
     0.0f,  0.75f, 0.0f, // Top vertex
    -0.75f, -0.75f, 0.0f, // Bottom-left vertex
     0.75f, -0.75f, 0.0f  // Bottom-right vertex
};

// Different color for each corresponding vertex
GLfloat triangleColors[] = {
     1.0f, 0.0f, 0.0f, // Red for top vertex
     0.0f, 1.0f, 0.0f, // Green for bottom-left vertex
     0.0f, 0.0f, 1.0f  // Blue for bottom-right vertex
};

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // Enable both vertex and color arrays[cite: 11]
    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glVertexPointer(3, GL_FLOAT, 0, triangleVertices);
    glColorPointer(3, GL_FLOAT, 0, triangleColors);

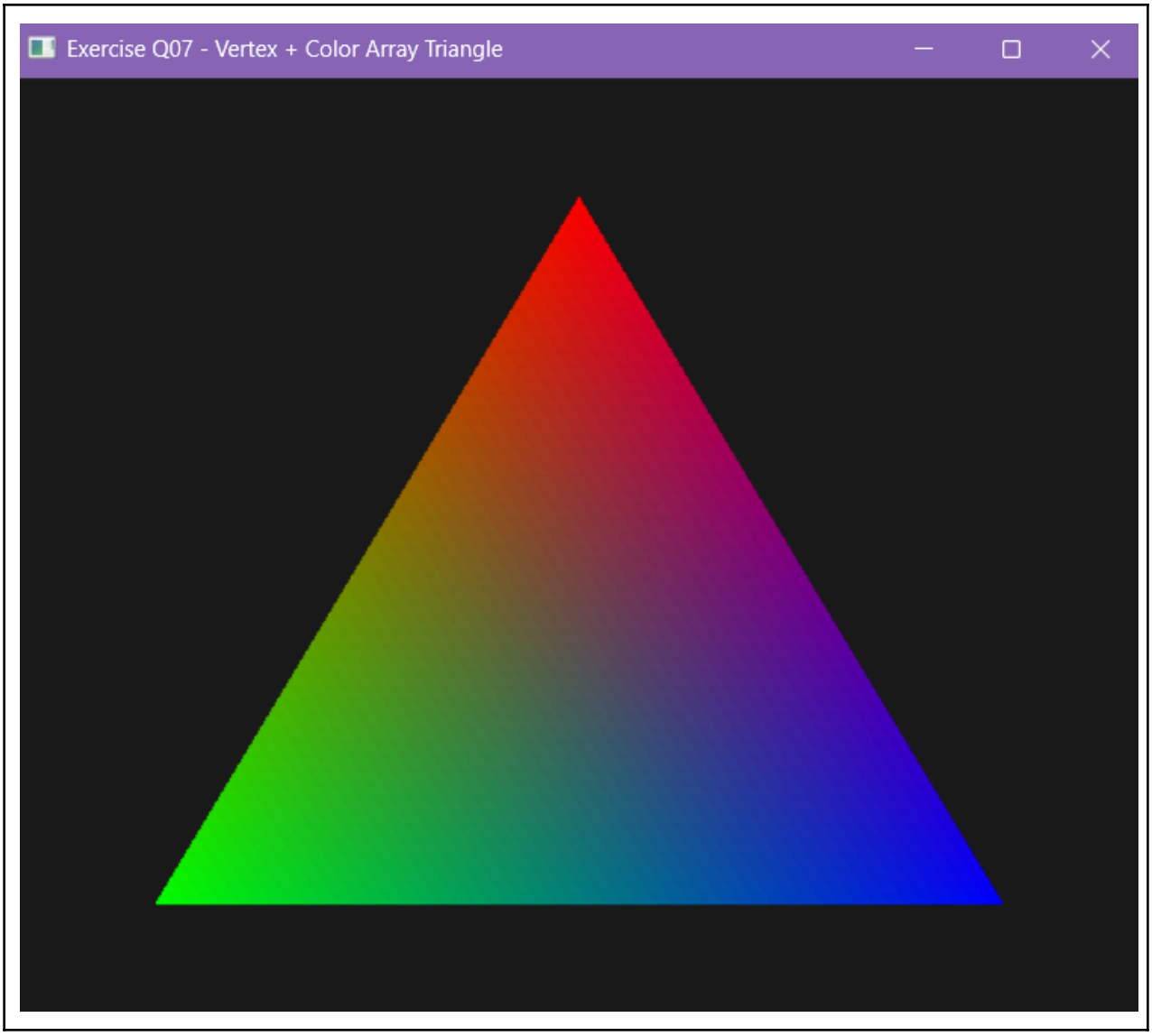
    glDrawArrays(GL_TRIANGLES, 0, 3);
```

```
    // Disable client states after drawing
    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Exercise Q07 - Vertex + Color Array Triangle");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Medium Exercises

Exercise Q08 — Three Triangles, One Array [Medium]

Objective: Practice scaling up the 'multiple primitives, one array' pattern from Example 07.

Task (Question): Draw THREE separate triangles from one vertex array and ONE glDrawArrays call (9 vertices total).

Hints:

- GL_TRIANGLES groups vertices in sets of 3 — with 9 vertices you get exactly 3 triangles.
- Space the three triangles out horizontally so they don't overlap.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// 9 vertices total grouped in sets of 3, spaced out horizontally[cite: 12]
GLfloat threeTrianglesVertices[] = {
    // Triangle 1 (Left)
    -0.7f, -0.2f, 0.0f,
    -0.5f,  0.2f, 0.0f,
    -0.3f, -0.2f, 0.0f,

    // Triangle 2 (Middle)
    -0.1f, -0.2f, 0.0f,
     0.1f,  0.2f, 0.0f,
     0.3f, -0.2f, 0.0f,

    // Triangle 3 (Right)
     0.3f, -0.2f, 0.0f,
     0.5f,  0.2f, 0.0f,
     0.7f, -0.2f, 0.0f
};

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.0f, 0.8f, 0.6f); // Teal fill color

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, threeTrianglesVertices);
```

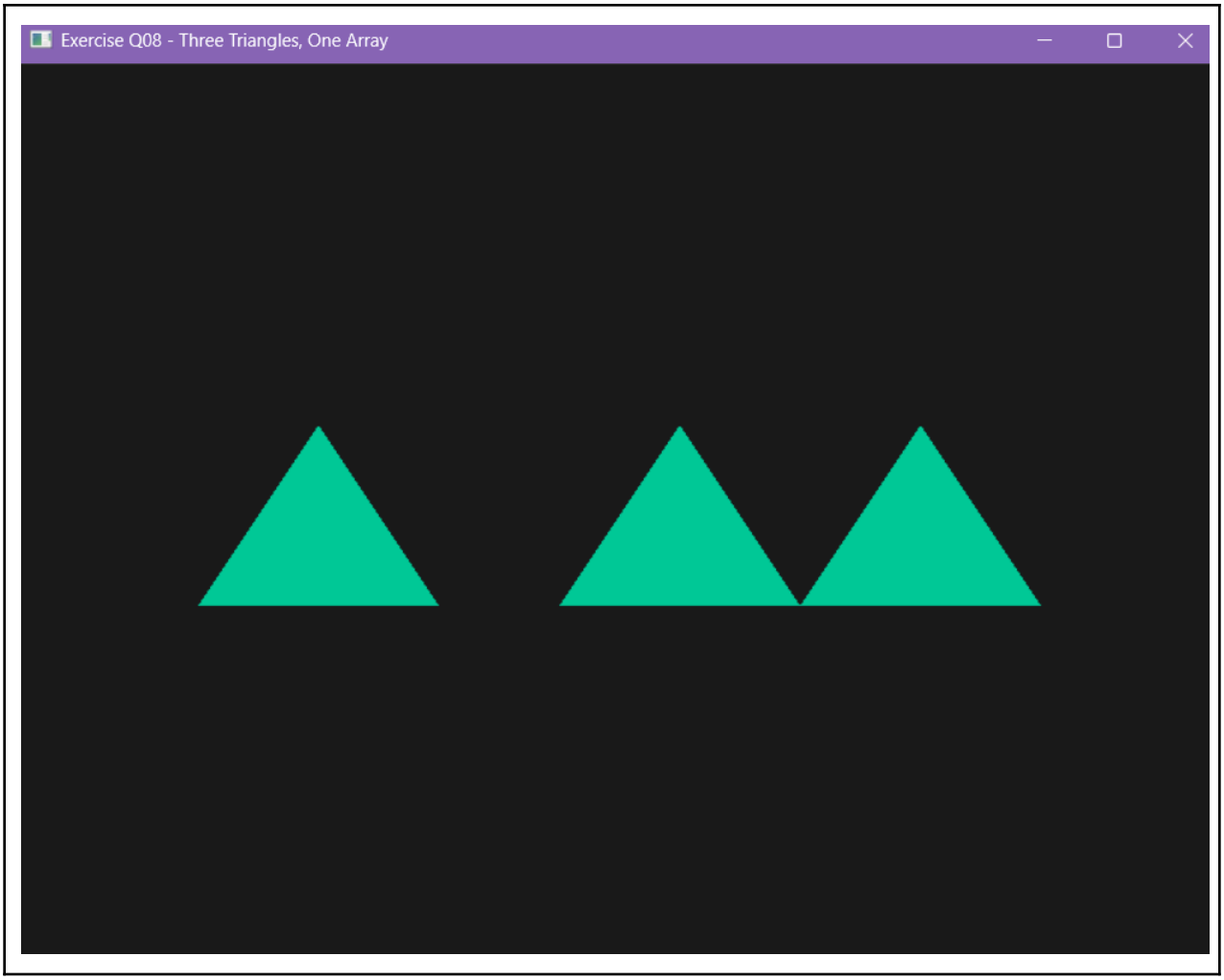
```
    // Draw 3 separate triangles from one array with a single call[cite: 12]
    glDrawArrays(GL_TRIANGLES, 0, 9);

    glDisableClientState(GL_VERTEX_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Exercise Q08 - Three Triangles, One Array");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q09 — Indexed Triangle Fan Pentagon [Medium]

Objective: Practice glDrawElements with GL_TRIANGLE_FAN and a shared center vertex.

Task (Question): Build a pentagon from GL_TRIANGLE_FAN using glDrawElements, where the shared center vertex is stored only ONCE in the vertex array.

Hints:

- A triangle fan's first vertex is the shared center — every triangle in the fan reuses it.
- Your index array should list the center index first, then the outer ring vertices in order, closing back to the first outer vertex.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// Vertex array: Center at index 0, followed by 5 outer ring vertices[cite:
13]
GLfloat pentagonVertices[] = {
     0.0f,  0.0f, 0.0f, // 0: Shared center vertex[cite: 13]
     0.0f,  0.6f, 0.0f, // 1: Top
     0.57f, 0.19f, 0.0f, // 2: Top-right
     0.35f, -0.49f, 0.0f,// 3: Bottom-right
    -0.35f, -0.49f, 0.0f,// 4: Bottom-left
    -0.57f, 0.19f, 0.0f  // 5: Top-left
};

// Index array: Center first, then outer ring in order, closing back to vertex
1[cite: 13]
GLubyte pentagonIndices[] = { 0, 1, 2, 3, 4, 5, 1 };

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.9f, 0.4f, 0.7f); // Pinkish-purple fill color

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, pentagonVertices);

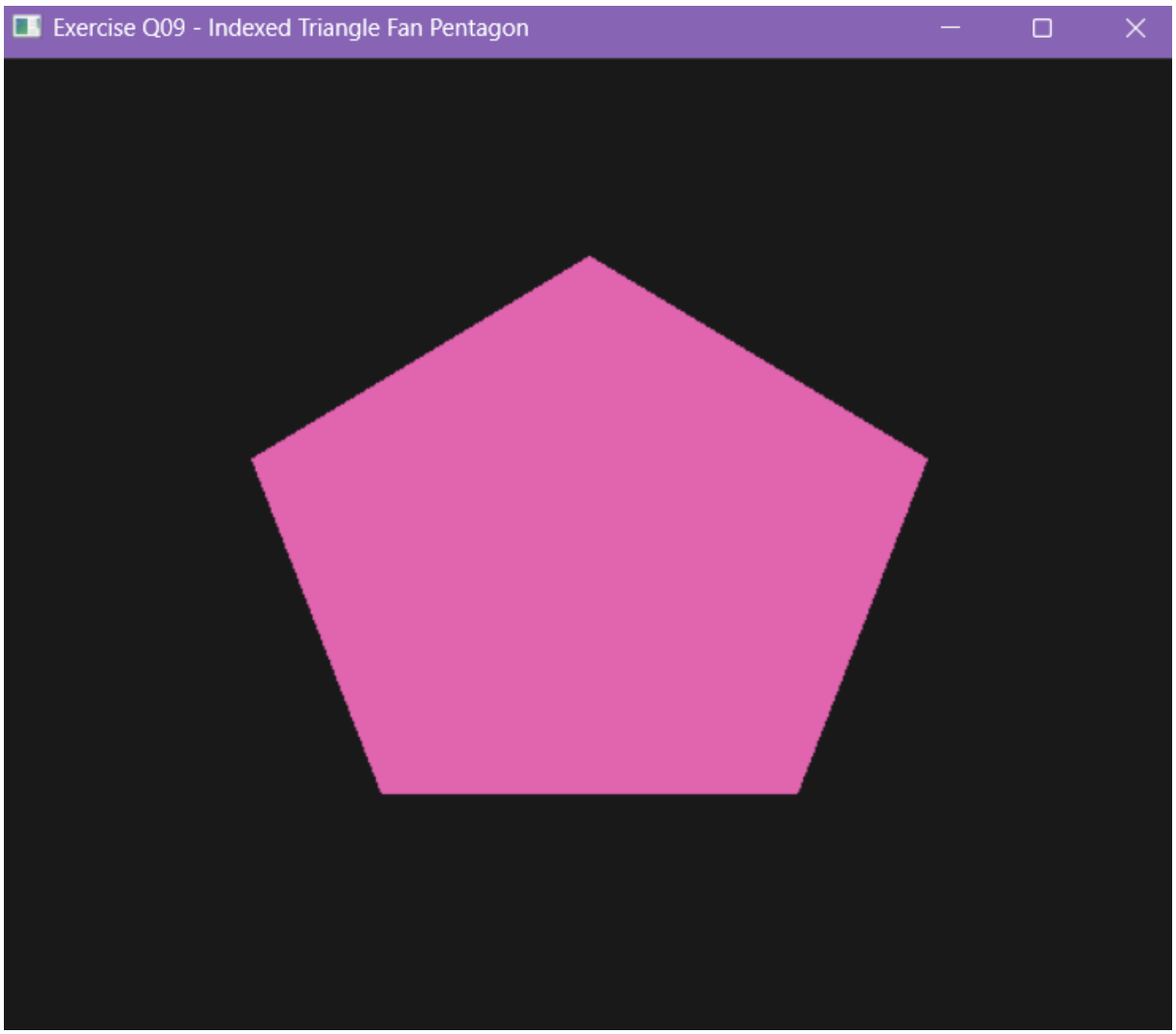
    // Draw using GL_TRIANGLE_FAN and glDrawElements[cite: 13]
    glDrawElements(GL_TRIANGLE_FAN, 7, GL_UNSIGNED_BYTE, pentagonIndices);

    glDisableClientState(GL_VERTEX_ARRAY);
```

```
    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Exercise Q09 - Indexed Triangle Fan Pentagon");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q10 — Checkerboard Row via glDrawElements [Medium]

Objective: Practice building several GL_QUADS primitives from ONE shared vertex pool with SEPARATE index lists.

Task (Question): Build a checkerboard-style row of 4 quads using glDrawElements, alternating black and white, sharing vertices between neighboring quads where the edges touch.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// Shared vertex pool for 4 connected quads in a row (10 vertices total)
GLfloat sharedVertices[] = {
    // Quad 0 left edge
    -0.8f, -0.2f, 0.0f, // 0
    -0.8f,  0.2f, 0.0f, // 1
    // Quad 0 right / Quad 1 right edge
    -0.4f,  0.2f, 0.0f, // 2
    -0.4f, -0.2f, 0.0f, // 3
    // Quad 2 right edge
     0.0f,  0.2f, 0.0f, // 4
     0.0f, -0.2f, 0.0f, // 5
    // Quad 3 right edge
     0.4f,  0.2f, 0.0f, // 6
     0.4f, -0.2f, 0.0f, // 7
    // Far right edge
     0.8f,  0.2f, 0.0f, // 8
     0.8f, -0.2f, 0.0f  // 9
};
```

```
// Separate index lists for each of the 4 quads sharing edges
GLubyte quad0Indices[] = { 0, 3, 2, 1 };
GLubyte quad1Indices[] = { 3, 5, 4, 2 };
GLubyte quad2Indices[] = { 5, 7, 6, 4 };
GLubyte quad3Indices[] = { 7, 9, 8, 6 };

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, sharedVertices);

    // Quad 1 (Black/Dark)[cite: 13]
    glColor3f(0.15f, 0.15f, 0.15f);
    glDrawElements(GL_QUADS, 4, GL_UNSIGNED_BYTE, quad0Indices);

    // Quad 2 (White/Bright)[cite: 13]
    glColor3f(0.9f, 0.9f, 0.9f);
    glDrawElements(GL_QUADS, 4, GL_UNSIGNED_BYTE, quad1Indices);

    // Quad 3 (Black/Dark)[cite: 13]
    glColor3f(0.15f, 0.15f, 0.15f);
    glDrawElements(GL_QUADS, 4, GL_UNSIGNED_BYTE, quad2Indices);

    // Quad 4 (White/Bright)[cite: 13]
    glColor3f(0.9f, 0.9f, 0.9f);
    glDrawElements(GL_QUADS, 4, GL_UNSIGNED_BYTE, quad3Indices);

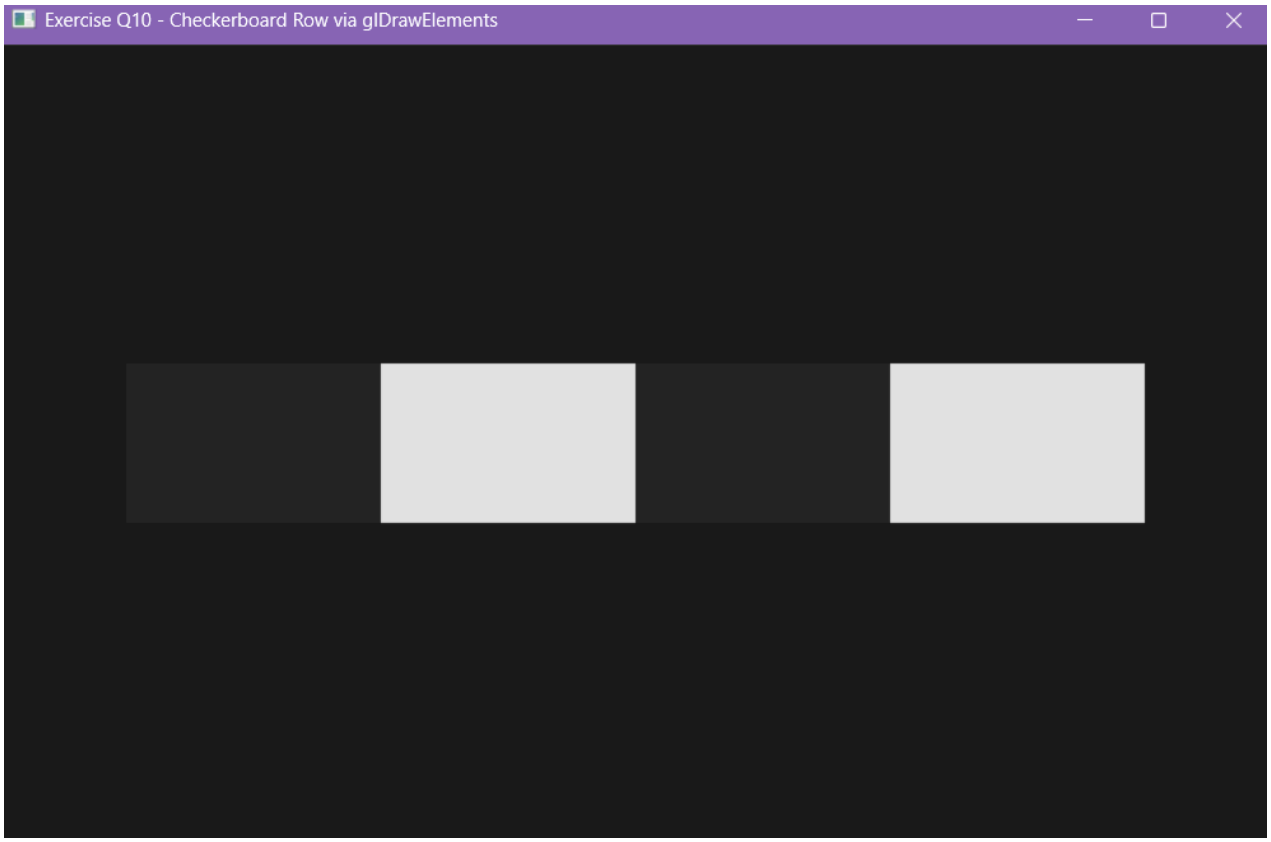
    glDisableClientState(GL_VERTEX_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(800, 500);
```

```
    glutCreateWindow("Exercise Q10 - Checkerboard Row via glDrawElements");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Hints:

- Neighboring quads that share an edge can reuse the SAME two vertices from the pool — no need to store them twice.
- Alternate the glColor3f call between each glDrawElements call for the checkerboard effect.

Exercise Q11 — Procedural Shaded Circle [Medium]

Objective: Practice generating vertex AND color array contents in a loop instead of hand-typing them.

Task (Question): Draw a smoothly shaded circle using GL_TRIANGLE_FAN, where BOTH the vertex array and the color array are filled in procedurally (using sin/cos and a loop), not hand-typed vertex by vertex.

Hints:

- Reserve arrays sized for (segments + 2) vertices: one center vertex, plus one extra to close the fan back to the start.
- Vary the color gradually as the loop index increases, for a visible color sweep around the circle.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>

using namespace std;

const int SEGMENTS = 30;
const int NUM_VERTICES = SEGMENTS + 2;

GLfloat circleVertices[NUM_VERTICES * 3];
GLfloat circleColors[NUM_VERTICES * 3];

void generateCircleData() {
    // Center vertex (Index 0)
    circleVertices[0] = 0.0f;
    circleVertices[1] = 0.0f;
    circleVertices[2] = 0.0f;

    circleColors[0] = 1.0f; // Center color (White)
    circleColors[1] = 1.0f;
    circleColors[2] = 1.0f;

    float radius = 0.6f;
```

```
    for (int i = 0; i <= SEGMENTS; i++) {
        float theta = 2.0f * 3.14159265359f * float(i) / float(SEGMENTS);
        float x = radius * cosf(theta);
        float y = radius * sinf(theta);

        int vIdx = (i + 1) * 3;
        circleVertices[vIdx]     = x;
        circleVertices[vIdx + 1] = y;
        circleVertices[vIdx + 2] = 0.0f;

        // Vary the color gradually around the circle[cite: 14]
        circleColors[vIdx]     = sinf(theta) * 0.5f + 0.5f;
        circleColors[vIdx + 1] = cosf(theta) * 0.5f + 0.5f;
        circleColors[vIdx + 2] = 0.8f;
    }
}

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glVertexPointer(3, GL_FLOAT, 0, circleVertices);
    glColorPointer(3, GL_FLOAT, 0, circleColors);

    glDrawArrays(GL_TRIANGLE_FAN, 0, NUM_VERTICES);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);

    glutSwapBuffers();
}

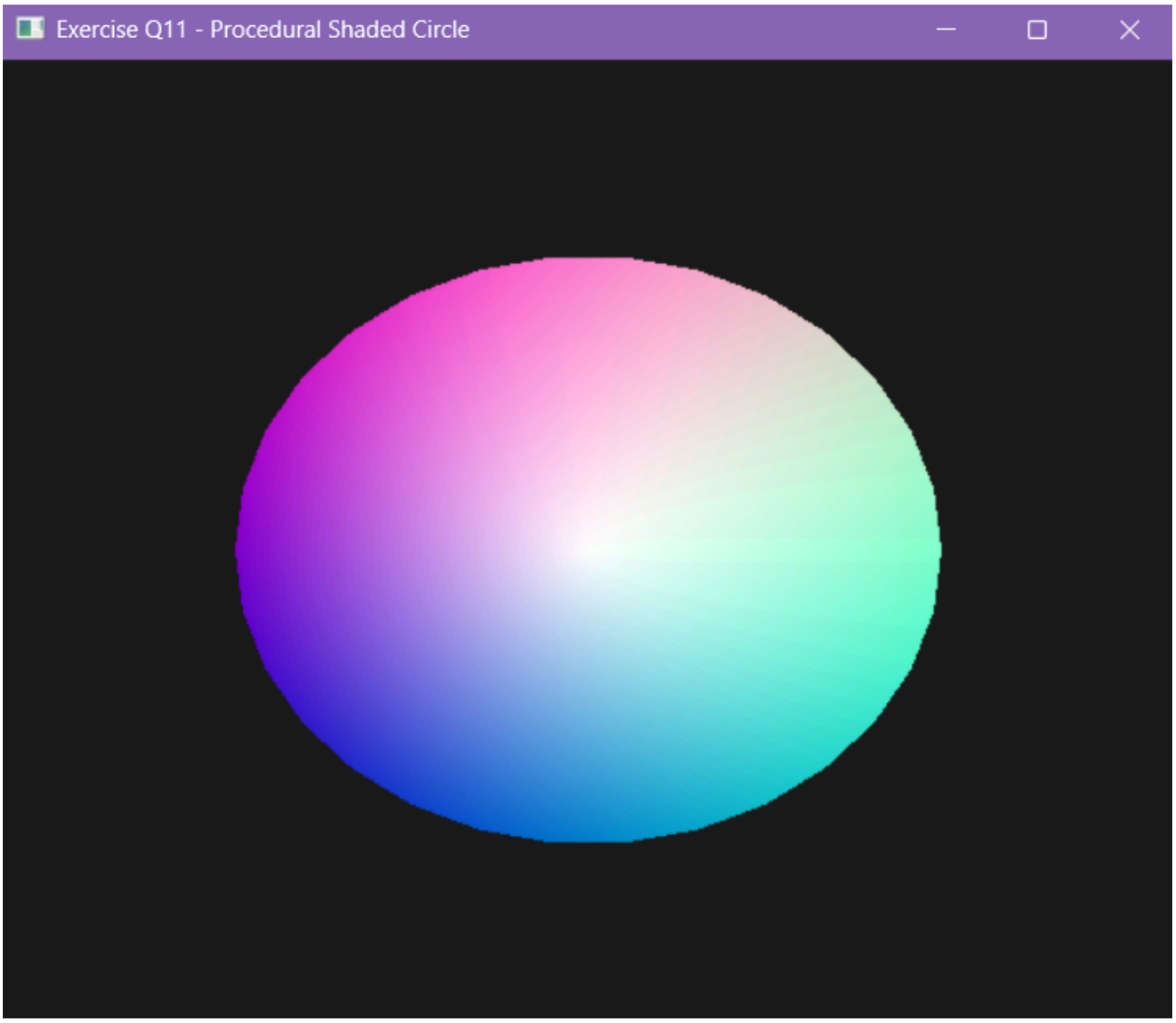
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
```

```
    glutCreateWindow("Exercise Q11 - Procedural Shaded Circle");

    generateCircleData();

    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q12 — Interleaved Array Quad [Medium]

Objective: Practice building and reading an interleaved position+color array with a non-zero stride.

Task (Question): Build an interleaved position+color array for a quad (like Example 16), and render it using stride-based glVertexPointer/glColorPointer calls.

Hints:

- Each vertex's 6 floats are x,y,z,r,g,b in that order — stride is 6 * sizeof(GLfloat).
- The color pointer's starting address must be offset by 3 floats past the vertex pointer's start.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// Interleaved array: x, y, z, r, g, b for each vertex (stride = 6
floats)[cite: 1, 15]
GLfloat interleavedQuadData[] = {
    // x,       y,      z,    r,    g,    b
    -0.6f, -0.6f,  0.0f,  1.0f, 0.0f, 0.0f, // Vertex 0: Bottom-left (Red)
     0.6f, -0.6f,  0.0f,  0.0f, 1.0f, 0.0f, // Vertex 1: Bottom-right (Green)
     0.6f,  0.6f,  0.0f,  0.0f, 0.0f, 1.0f, // Vertex 2: Top-right (Blue)
    -0.6f,  0.6f,  0.0f,  1.0f, 1.0f, 0.0f  // Vertex 3: Top-left (Yellow)
};

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    GLsizei stride = 6 * sizeof(GLfloat);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);
```

```
    // Vertex pointer starts at the beginning of the array[cite: 1, 15]
    glVertexPointer(3, GL_FLOAT, stride, interleavedQuadData);

    // Color pointer is offset by 3 floats to point to r, g, b[cite: 1, 15]
    glColorPointer(3, GL_FLOAT, stride, interleavedQuadData + 3);

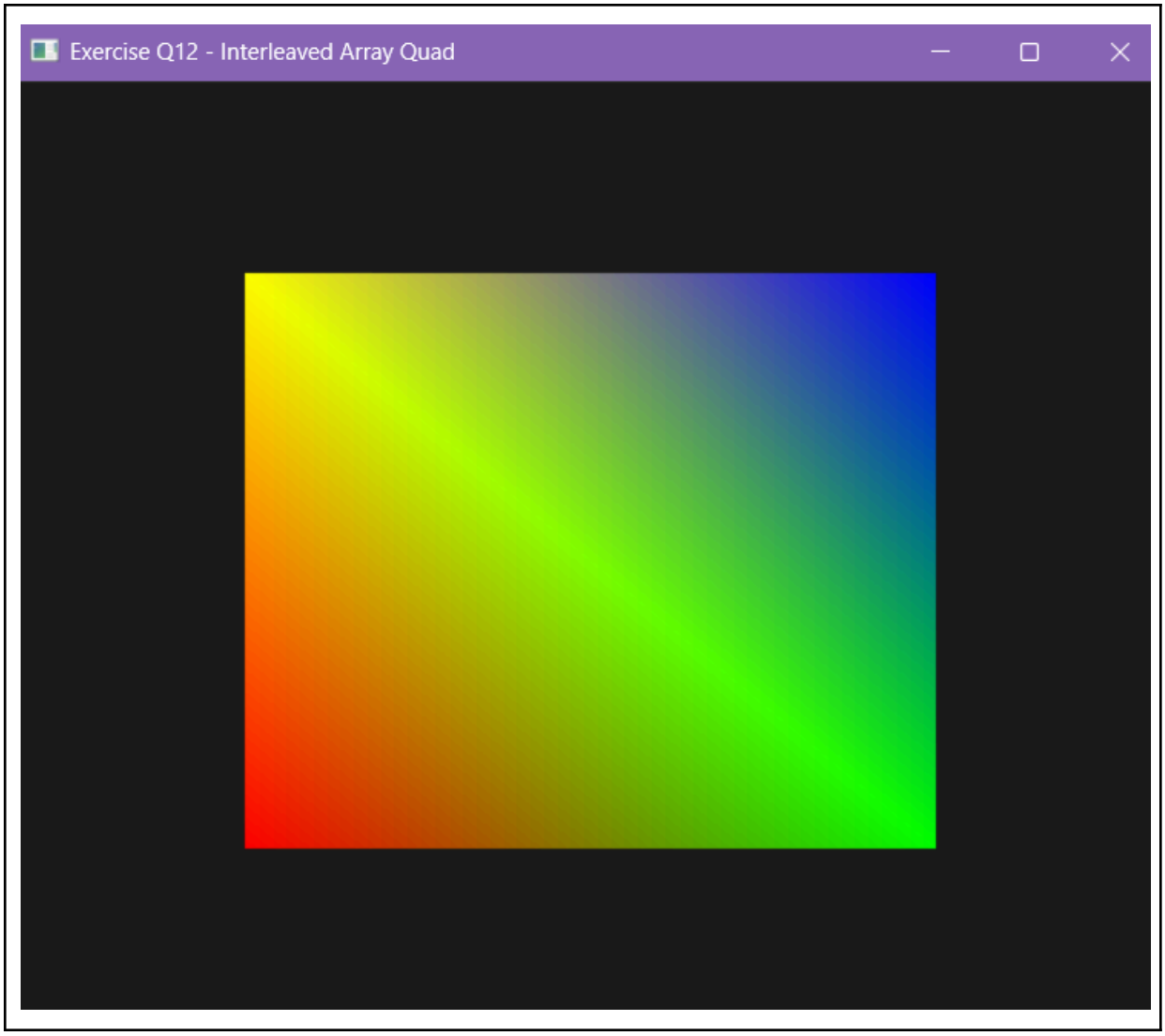
    glDrawArrays(GL_QUADS, 0, 4);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Exercise Q12 - Interleaved Array Quad");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q13 — Pinwheel via glDrawElements [Medium]

Objective: Practice re-implementing a known shape (the lecture's pinwheel) using indexed rendering instead of glDrawArrays.

Task (Question): Recreate the lecture's four-triangle pinwheel using glDrawElements INSTEAD of glDrawArrays, sharing the center vertex across all four triangles.

Hints:

- The center vertex (0,0,0) should appear only ONCE in the vertex array, no matter how many triangles use it.

- Your index array will have 12 entries (4 triangles x 3 indices), but your vertex array will have only 9 unique vertices.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// Vertex array: Center at index 0, followed by 4 outer blade vertices
GLfloat pinwheelVertices[] = {
     0.0f,  0.0f, 0.0f, // 0: Shared center vertex
     0.0f,  0.7f, 0.0f, // 1: Top
     0.7f,  0.0f, 0.0f, // 2: Right
     0.0f, -0.7f, 0.0f, // 3: Bottom
    -0.7f,  0.0f, 0.0f  // 4: Left
};

// Index array defining 4 individual triangles sharing vertex 0 as the center
GLubyte pinwheelIndices[] = {
    0, 1, 2, // Triangle 1
    0, 2, 3, // Triangle 2
    0, 3, 4, // Triangle 3
    0, 4, 1  // Triangle 4
};

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.2f, 0.7f, 1.0f); // Bright blue pinwheel color

    glEnableClientState(GL_VERTEX_ARRAY);
```

```
    glVertexPointer(3, GL_FLOAT, 0, pinwheelVertices);

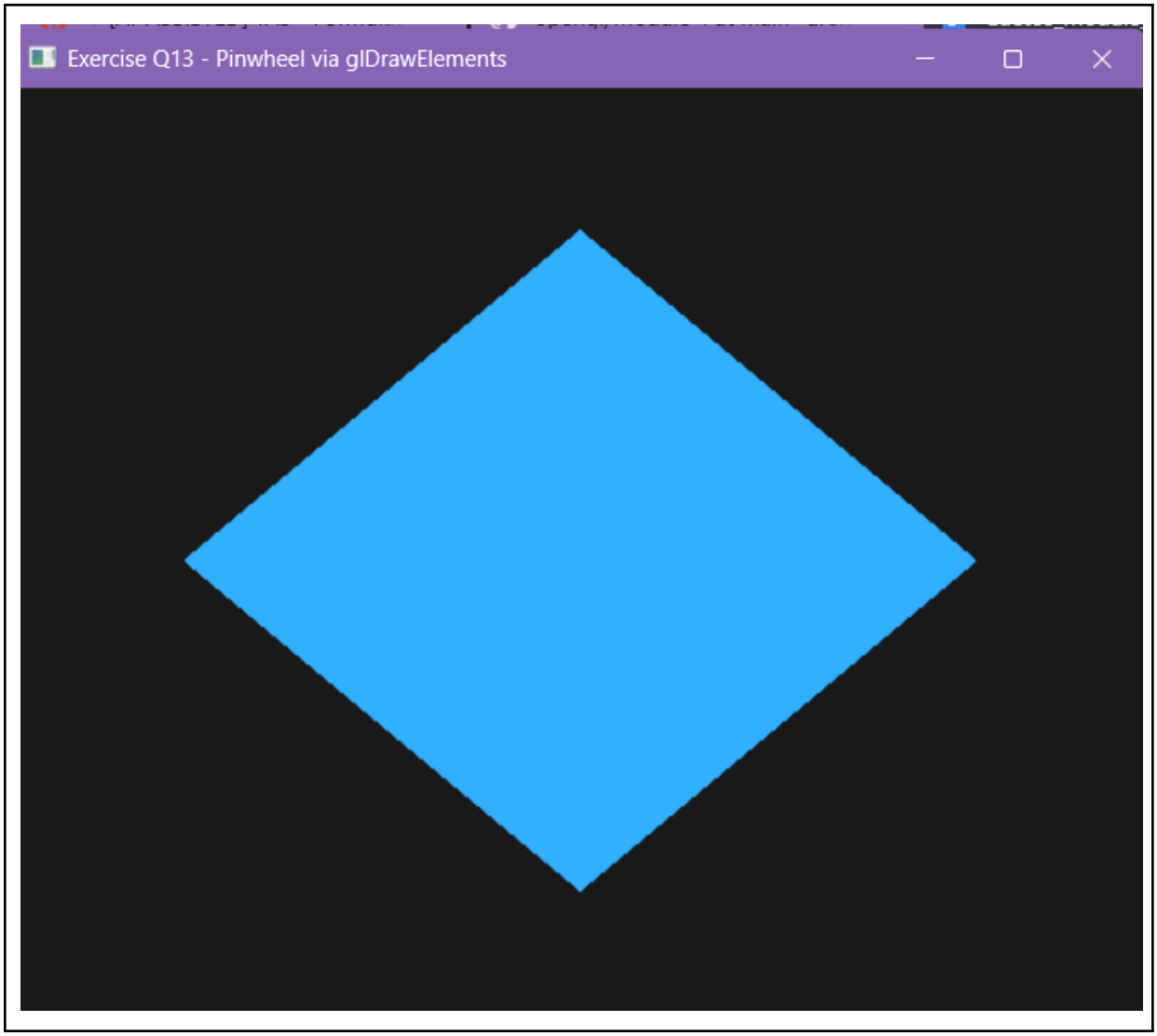
    // Draw 4 triangles (12 indices total) using indexed rendering
    glDrawElements(GL_TRIANGLES, 12, GL_UNSIGNED_BYTE, pinwheelIndices);

    glDisableClientState(GL_VERTEX_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Exercise Q13 - Pinwheel via glDrawElements");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q14 — Colored Quad Strip Staircase [Medium]

Objective: Practice combining an earlier primitive type (GL_QUAD_STRIP from Module 2) with vertex + color arrays.

Task (Question): Combine a GL_QUAD_STRIP 'staircase' shape with a vertex array AND a color array so each step is a different color, all drawn in ONE glDrawArrays call.

Hints:

- Remember GL_QUAD_STRIP vertices alternate bottom, top, bottom, top as you move across the strip.
- Give each step's bottom and top vertex the SAME color so each step reads as one solid color rather than blending into its neighbor.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// Vertex array forming a staircase using GL_QUAD_STRIP (alternating bottom,
top, bottom, top)
GLfloat stairVertices[] = {
    // Step 1
    -0.8f, -0.6f, 0.0f, // 0: Bottom-left
    -0.8f, -0.3f, 0.0f, // 1: Top-left
    -0.4f, -0.6f, 0.0f, // 2: Bottom-right
    -0.4f, -0.3f, 0.0f, // 3: Top-right

    // Step 2 (shares right edge of Step 1 as its left)
    -0.4f, -0.3f, 0.0f, // 4: Bottom-left (reuses previous top-right)
     0.0f,  0.0f, 0.0f, // 5: Top-left
     0.0f, -0.6f, 0.0f, // 6: Bottom-right
     0.0f,  0.0f, 0.0f  // 7: Top-right (simplified stair geometry)
};

// Color array: Give each step's bottom and top vertices the SAME color
GLfloat stairColors[] = {
    // Step 1 Color (Red)
    1.0f, 0.2f, 0.2f,
    1.0f, 0.2f, 0.2f,
    1.0f, 0.2f, 0.2f,
    1.0f, 0.2f, 0.2f,

    // Step 2 Color (Green)
    0.2f, 1.0f, 0.2f,
    0.2f, 1.0f, 0.2f,
    0.2f, 1.0f, 0.2f,
    0.2f, 1.0f, 0.2f
```

```
};

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    glVertexPointer(3, GL_FLOAT, 0, stairVertices);
    glColorPointer(3, GL_FLOAT, 0, stairColors);

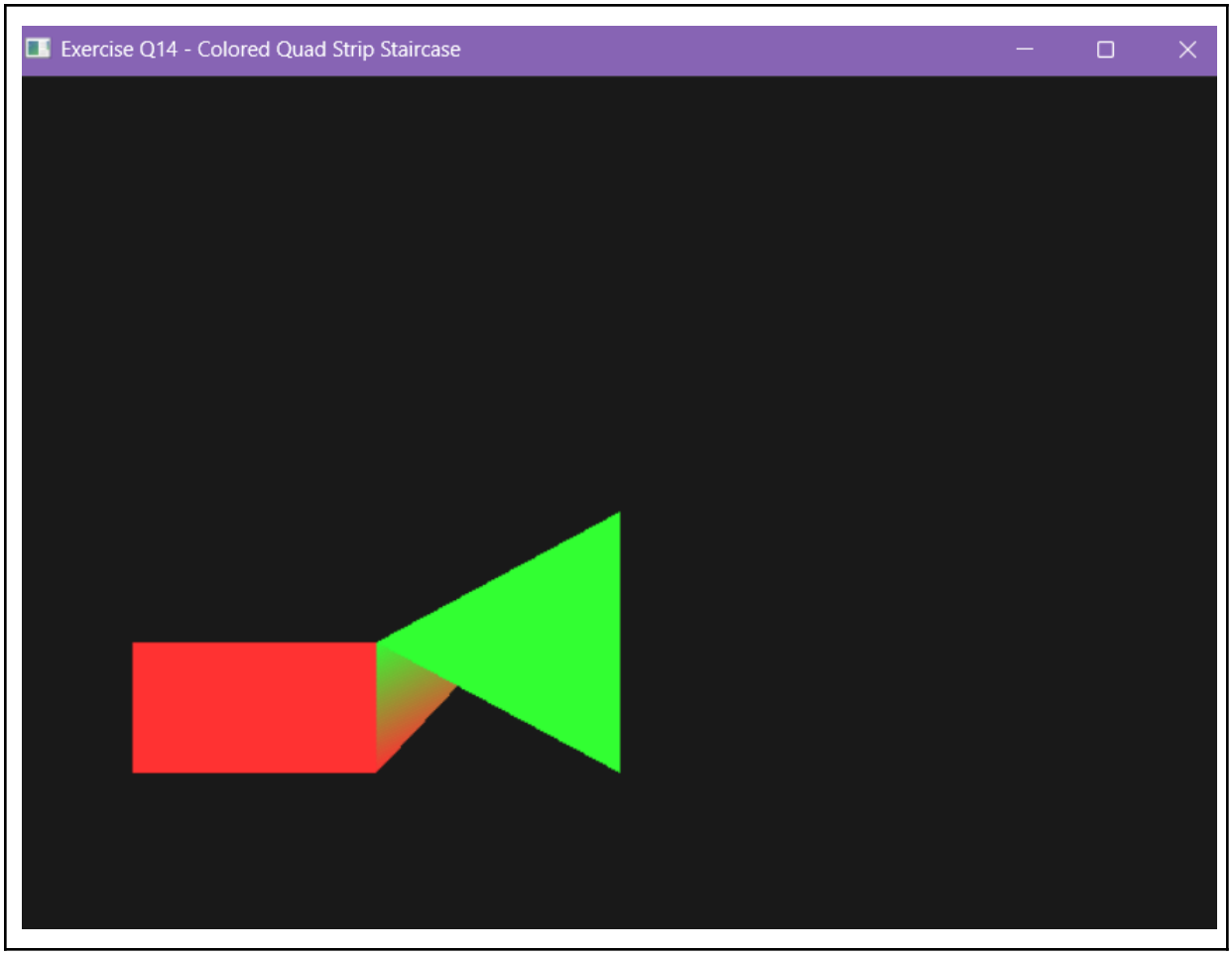
    // Draw the staircase using ONE glDrawArrays call
    glDrawArrays(GL_QUAD_STRIP, 0, 8);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(700, 500);
    glutCreateWindow("Exercise Q14 - Colored Quad Strip Staircase");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Hard Exercises

Exercise Q15 — Fully Array-Based Scene [Hard]

Objective: Practice building a multi-shape scene where every single shape uses vertex array data.

Task (Question): Build a full scene (at least 3 shapes: e.g., a sun, a mountain, and ground) where EVERY shape is drawn from vertex array data — no glVertex2f calls anywhere — mixing glDrawArrays and glDrawElements as appropriate for each shape.

Hints:

- A sun or star shape is a natural fit for glDrawElements with a shared center vertex.
- Simple shapes like the ground rectangle can stay as plain glDrawArrays calls — you don't need indexing everywhere.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// 1. Ground vertices (Quad using glDrawArrays)
GLfloat groundVertices[] = {
    -1.0f, -1.0f, 0.0f,
     1.0f, -1.0f, 0.0f,
     1.0f, -0.3f, 0.0f,
    -1.0f, -0.3f, 0.0f
};

// 2. Mountain vertices (Triangle using glDrawArrays)
GLfloat mountainVertices[] = {
    -0.8f, -0.3f, 0.0f,
    -0.1f,  0.6f, 0.0f,
     0.6f, -0.3f, 0.0f
};

// 3. Sun vertices (Shared center + outer ring for glDrawElements)[cite: 16]
GLfloat sunVertices[] = {
     0.6f,  0.6f, 0.0f, // Center (Index 0)[cite: 16]
     0.5f,  0.8f, 0.0f, // Top (Index 1)
     0.7f,  0.8f, 0.0f, // Right (Index 2)
     0.7f,  0.4f, 0.0f, // Bottom-Right (Index 3)
     0.5f,  0.4f, 0.0f  // Bottom-Left (Index 4)
};

GLubyte sunIndices[] = { 0, 1, 2, 0, 2, 3, 0, 3, 4, 0, 4, 1 };

void display() {
    glClearColor(0.5f, 0.8f, 0.9f, 1.0f); // Sky blue background
```

```
    glClear(GL_COLOR_BUFFER_BIT);

    glEnableClientState(GL_VERTEX_ARRAY);

    // Draw Ground (Green)
    glColor3f(0.2f, 0.6f, 0.2f);
    glVertexPointer(3, GL_FLOAT, 0, groundVertices);
    glDrawArrays(GL_QUADS, 0, 4);

    // Draw Mountain (Gray/Brown)
    glColor3f(0.4f, 0.4f, 0.4f);
    glVertexPointer(3, GL_FLOAT, 0, mountainVertices);
    glDrawArrays(GL_TRIANGLES, 0, 3);

    // Draw Sun (Yellow)[cite: 16]
    glColor3f(1.0f, 0.9f, 0.1f);
    glVertexPointer(3, GL_FLOAT, 0, sunVertices);
    glDrawElements(GL_TRIANGLES, 12, GL_UNSIGNED_BYTE, sunIndices);

    glDisableClientState(GL_VERTEX_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Exercise Q15 - Fully Array-Based Scene");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q16 — glDrawArrays vs. glDrawElements, Side by Side [Hard]

Objective: Practice directly comparing the vertex counts of both approaches on the identical shape.

Task (Question): Draw the SAME quad twice, side by side: once with glDrawArrays (6 vertices, 2 duplicated), and once with glDrawElements (4 unique vertices + a 6-entry index array), so both approaches can be visually and numerically compared.

Hints:

- Use a comment above each array noting exactly how many floats/vertices are stored, so the savings are obvious at a glance.
- Give the two quads different colors so it's clear they are two separate draw calls, not one shape.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// Left Quad: Drawn with glDrawArrays (6 vertices / 18 floats total, 2
duplicated for triangles)[cite: 1]
GLfloat arrayQuadVertices[] = {
    -0.7f, -0.4f, 0.0f, // 0: Bottom-left
    -0.3f, -0.4f, 0.0f, // 1: Bottom-right
    -0.3f,  0.4f, 0.0f, // 2: Top-right
    -0.7f, -0.4f, 0.0f, // 3: Bottom-left (duplicate)
    -0.3f,  0.4f, 0.0f, // 4: Top-right (duplicate)
    -0.7f,  0.4f, 0.0f  // 5: Top-left
};

// Right Quad: Drawn with glDrawElements (4 unique vertices / 12 floats
total)[cite: 1]
GLfloat elementQuadVertices[] = {
     0.3f, -0.4f, 0.0f, // 0: Bottom-left
     0.7f, -0.4f, 0.0f, // 1: Bottom-right
     0.7f,  0.4f, 0.0f, // 2: Top-right
     0.3f,  0.4f, 0.0f  // 3: Top-left
};

// 6-entry index array for the right quad[cite: 1]
GLubyte elementQuadIndices[] = { 0, 1, 2, 0, 2, 3 };

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glEnableClientState(GL_VERTEX_ARRAY);
```

```
    // 1. Draw Left Quad using glDrawArrays (Red/Orange color)[cite: 1]
    glColor3f(1.0f, 0.4f, 0.3f);
    glVertexPointer(3, GL_FLOAT, 0, arrayQuadVertices);
    glDrawArrays(GL_TRIANGLES, 0, 6);

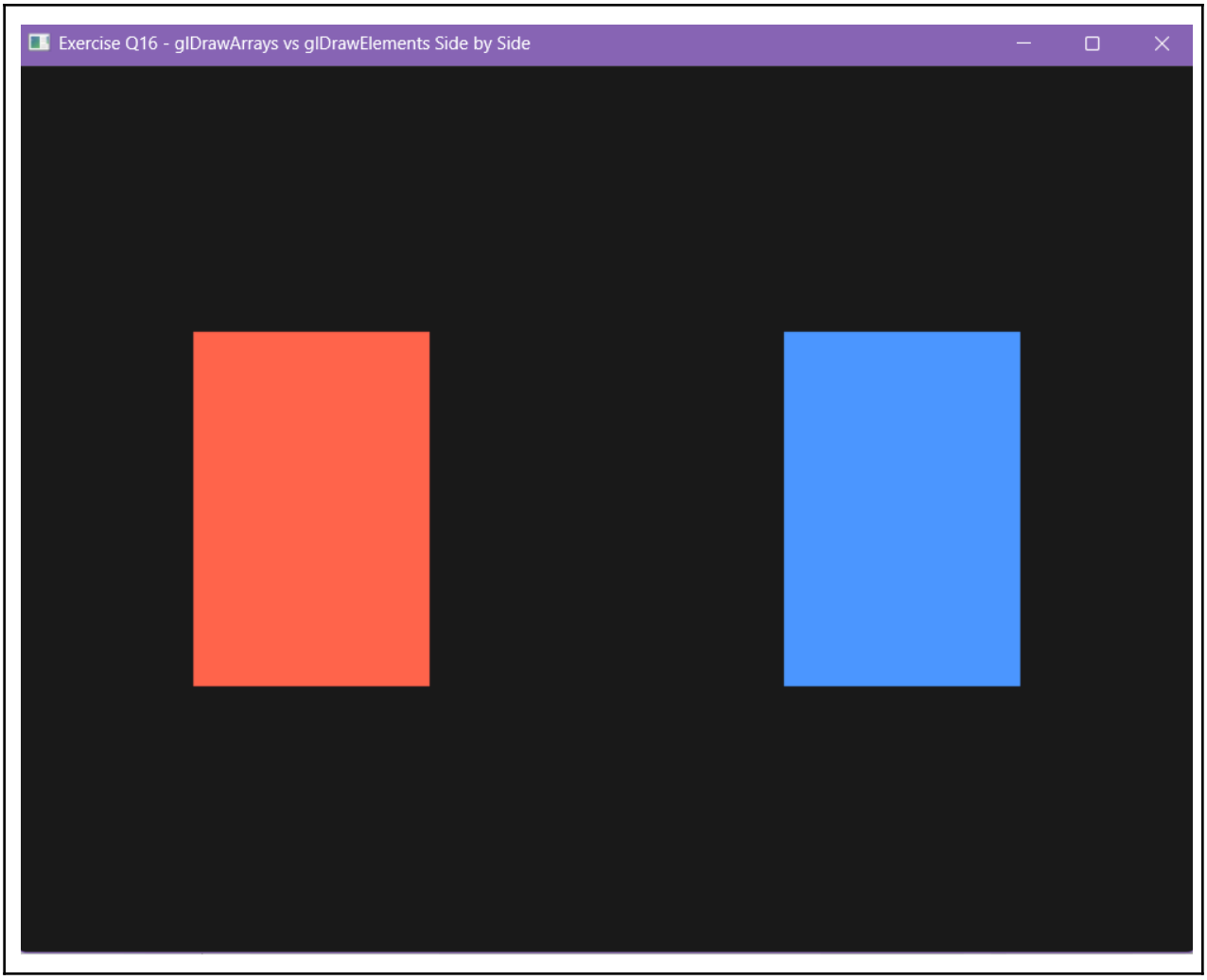
    // 2. Draw Right Quad using glDrawElements (Blue color)[cite: 1]
    glColor3f(0.3f, 0.6f, 1.0f);
    glVertexPointer(3, GL_FLOAT, 0, elementQuadVertices);
    glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_BYTE, elementQuadIndices);

    glDisableClientState(GL_VERTEX_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Exercise Q16 - glDrawArrays vs glDrawElements Side by
Side");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q17 — Procedural Gear Shape [Hard]

Objective: Practice generating a more complex indexed shape procedurally, with alternating colors across separate glDrawElements calls.

Task (Question): Build a procedurally generated ring of colored triangles (a simple 'gear' shape) using glDrawElements, where vertices are computed in a loop, and every other triangle uses a different color via TWO separate glDrawElements calls sharing ONE vertex pool.

Hints:

- Alternate between a larger radius and a smaller radius as you loop around the ring to create the 'teeth' look.
- Build two separate index arrays (one for 'even' teeth, one for 'odd' teeth) that both index into the SAME vertex array.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <vector>

using namespace std;

const int NUM_TEETH = 12;
const int TOTAL_VERTICES = 1 + (NUM_TEETH * 2);

GLfloat gearVertices[TOTAL_VERTICES * 3];
vector<GLubyte> evenIndices;
vector<GLubyte> oddIndices;

void generateGearData() {
    // Center vertex (Index 0)
    gearVertices[0] = 0.0f;
    gearVertices[1] = 0.0f;
    gearVertices[2] = 0.0f;

    float outerRadius = 0.6f;
    float innerRadius = 0.4f;

    // Generate vertices alternating between outer and inner radius
    for (int i = 0; i < NUM_TEETH * 2; i++) {
        float theta = 2.0f * 3.14159265359f * float(i) / float(NUM_TEETH * 2);
        float r = (i % 2 == 0) ? outerRadius : innerRadius;

        int idx = (i + 1) * 3;
        gearVertices[idx]     = r * cosf(theta);
```

```
        gearVertices[idx + 1] = r * sinf(theta);
        gearVertices[idx + 2] = 0.0f;
    }

    // Build separate index arrays for even and odd teeth triangles
    int numVerts = NUM_TEETH * 2;
    for (int i = 0; i < numVerts; i++) {
        GLubyte next = (i + 1) % numVerts + 1;
        GLubyte curr = i + 1;

        if (i % 2 == 0) {
            evenIndices.push_back(0);
            evenIndices.push_back(curr);
            evenIndices.push_back(next);
        } else {
            oddIndices.push_back(0);
            oddIndices.push_back(curr);
            oddIndices.push_back(next);
        }
    }
}

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glEnableClientState(GL_VERTEX_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, gearVertices);

    // 1st Draw Call: Even triangles (Color 1)
    glColor3f(0.8f, 0.5f, 0.1f);
    glDrawElements(GL_TRIANGLES, evenIndices.size(), GL_UNSIGNED_BYTE,
evenIndices.data());

    // 2nd Draw Call: Odd triangles (Color 2)
    glColor3f(0.9f, 0.7f, 0.2f);
    glDrawElements(GL_TRIANGLES, oddIndices.size(), GL_UNSIGNED_BYTE,
oddIndices.data());
```

```
    glDisableClientState(GL_VERTEX_ARRAY);

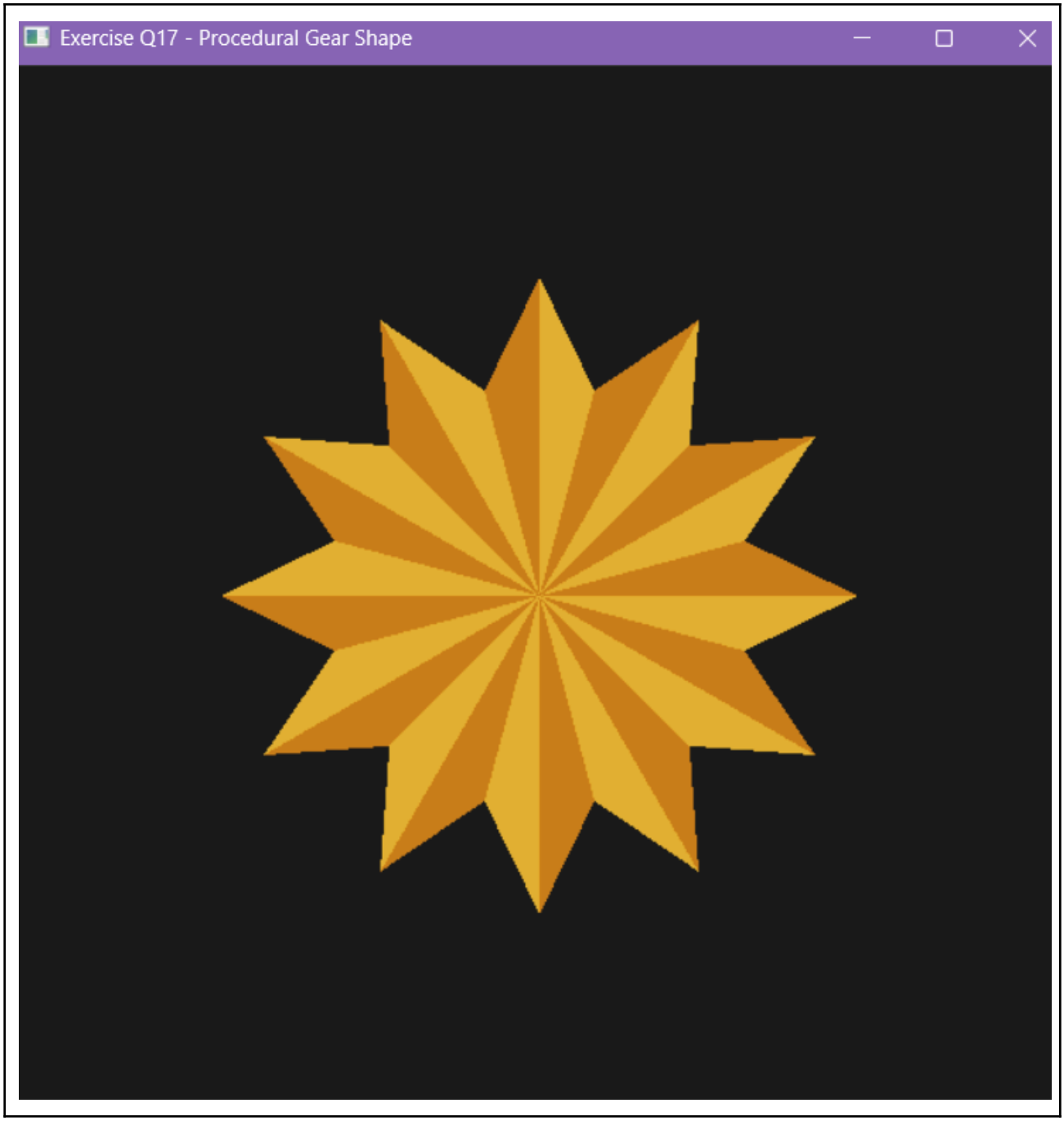
    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Exercise Q17 - Procedural Gear Shape");

    generateGearData();

    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q18 — Keyboard-Switched Vertex Arrays [Hard]

Objective: Practice combining vertex arrays with interactivity from Module 3.

Task (Question): Use glutKeyboardFunc so pressing '1', '2', or '3' switches which vertex array shape is displayed (a triangle, a quad, and a pentagon), WITHOUT rebuilding the arrays each time display()

runs.

Hints:

- Declare all three shapes' vertex arrays as global data, built once, outside of display().
- Store the current selection in a global int and branch on it inside display() to choose which array/mode to draw.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// Global selection state (1 = Triangle, 2 = Quad, 3 = Pentagon)
int currentShape = 1;

// Shape 1: Triangle vertices
GLfloat triangleVertices[] = {
     0.0f,  0.6f, 0.0f,
    -0.6f, -0.6f, 0.0f,
     0.6f, -0.6f, 0.0f
};

// Shape 2: Quad vertices
GLfloat quadVertices[] = {
    -0.5f, -0.5f, 0.0f,
     0.5f, -0.5f, 0.0f,
     0.5f,  0.5f, 0.0f,
    -0.5f,  0.5f, 0.0f
};

// Shape 3: Pentagon vertices
GLfloat pentagonVertices[] = {
```

```
     0.0f,  0.6f, 0.0f,
     0.6f,  0.2f, 0.0f,
     0.4f, -0.6f, 0.0f,
    -0.4f, -0.6f, 0.0f,
    -0.6f,  0.2f, 0.0f
};

void keyboard(unsigned char key, int x, int y) {
    if (key == '1') {
        currentShape = 1;
    } else if (key == '2') {
        currentShape = 2;
    } else if (key == '3') {
        currentShape = 3;
    }
    glutPostRedisplay(); // Request window redraw with new shape
}

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glEnableClientState(GL_VERTEX_ARRAY);

    // Branch based on global selection without rebuilding arrays
    if (currentShape == 1) {
        glColor3f(1.0f, 0.3f, 0.3f); // Red Triangle
        glVertexPointer(3, GL_FLOAT, 0, triangleVertices);
        glDrawArrays(GL_TRIANGLES, 0, 3);
    } else if (currentShape == 2) {
        glColor3f(0.3f, 1.0f, 0.3f); // Green Quad
        glVertexPointer(3, GL_FLOAT, 0, quadVertices);
        glDrawArrays(GL_QUADS, 0, 4);
    } else if (currentShape == 3) {
        glColor3f(0.3f, 0.3f, 1.0f); // Blue Pentagon
        glVertexPointer(3, GL_FLOAT, 0, pentagonVertices);
        glDrawArrays(GL_POLYGON, 0, 5);
    }
```

```
    glDisableClientState(GL_VERTEX_ARRAY);

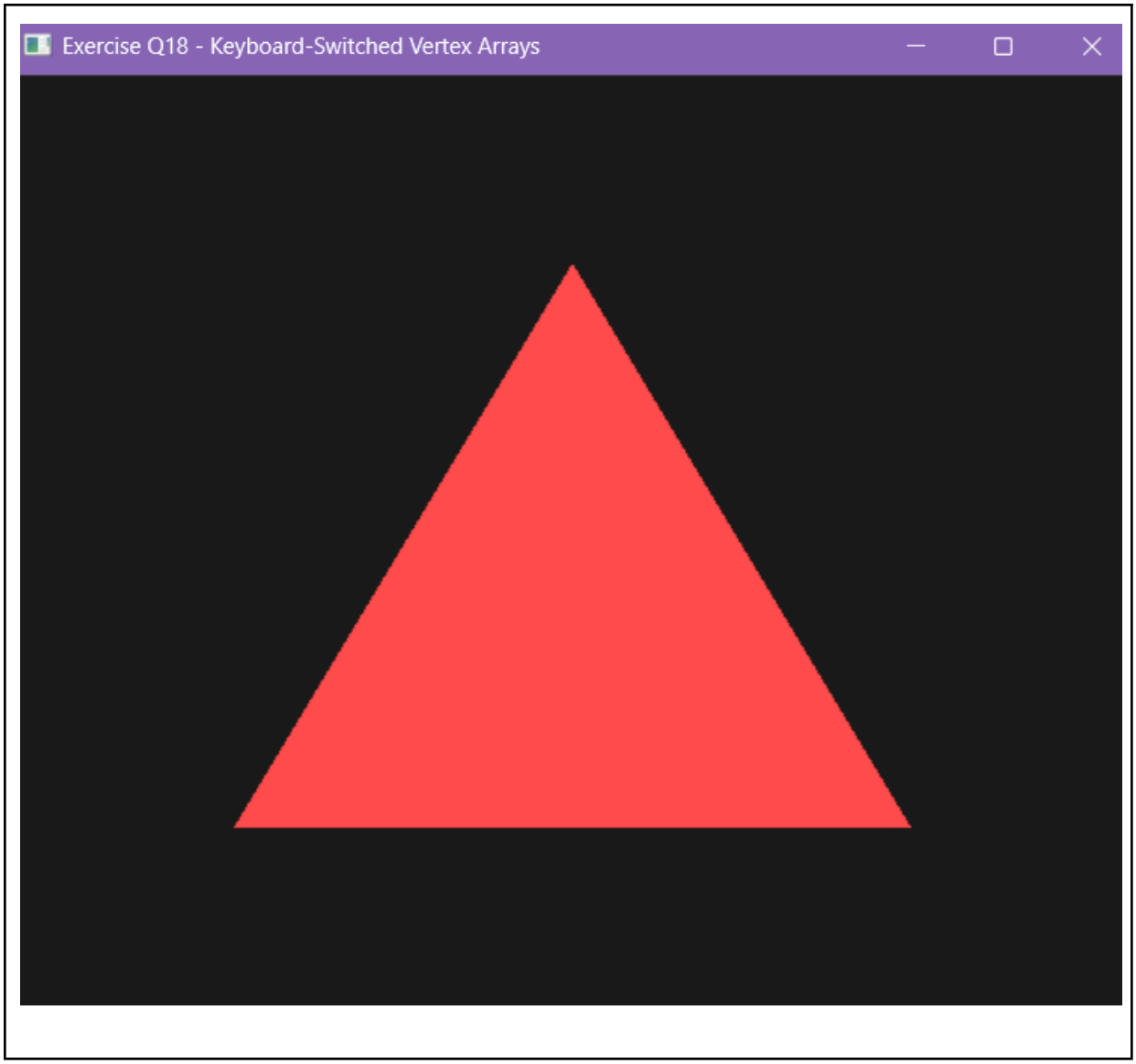
    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Exercise Q18 - Keyboard-Switched Vertex Arrays");

    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard); // Register keyboard callback

    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q19 — Interleaved + Indexed Hexagon [Hard]

Objective: Practice combining EVERY vertex array technique from this module in a single shape.

Task (Question): Build an interleaved vertex+color array (position AND color in one array, like Example 16) for a hexagon rendered with glDrawElements AND a non-zero stride.

Hints:

- Six vertices, no shared/duplicate positions needed here — the index array can simply be {0,1,2,3,4,5} for GL_POLYGON.

- Double-check your stride math: with 6 floats per vertex, stride = 6 * sizeof(GLfloat), and the color

pointer starts 3 floats after the vertex pointer.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>

using namespace std;

// Interleaved array for 6 vertices of a hexagon (x, y, z, r, g, b)
GLfloat hexagonData[] = {
    // x,       y,      z,    r,    g,    b
     0.0f,  0.6f,  0.0f,  1.0f, 0.2f, 0.2f, // Vertex 0: Top
     0.5f,  0.3f,  0.0f,  0.2f, 1.0f, 0.2f, // Vertex 1: Top-Right
     0.5f, -0.3f,  0.0f,  0.2f, 0.2f, 1.0f, // Vertex 2: Bottom-Right
     0.0f, -0.6f,  0.0f,  1.0f, 1.0f, 0.2f, // Vertex 3: Bottom
    -0.5f, -0.3f,  0.0f,  1.0f, 0.2f, 1.0f, // Vertex 4: Bottom-Left
    -0.5f,  0.3f,  0.0f,  0.2f, 1.0f, 1.0f  // Vertex 5: Top-Left
};

// Index array for GL_POLYGON
GLubyte hexagonIndices[] = { 0, 1, 2, 3, 4, 5 };

void display() {
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    GLsizei stride = 6 * sizeof(GLfloat);

    glEnableClientState(GL_VERTEX_ARRAY);
    glEnableClientState(GL_COLOR_ARRAY);

    // Vertex pointer starts at the beginning of the array
    glVertexPointer(3, GL_FLOAT, stride, hexagonData);
```

```
    // Color pointer is offset by 3 floats to point to r, g, b
    glColorPointer(3, GL_FLOAT, stride, hexagonData + 3);

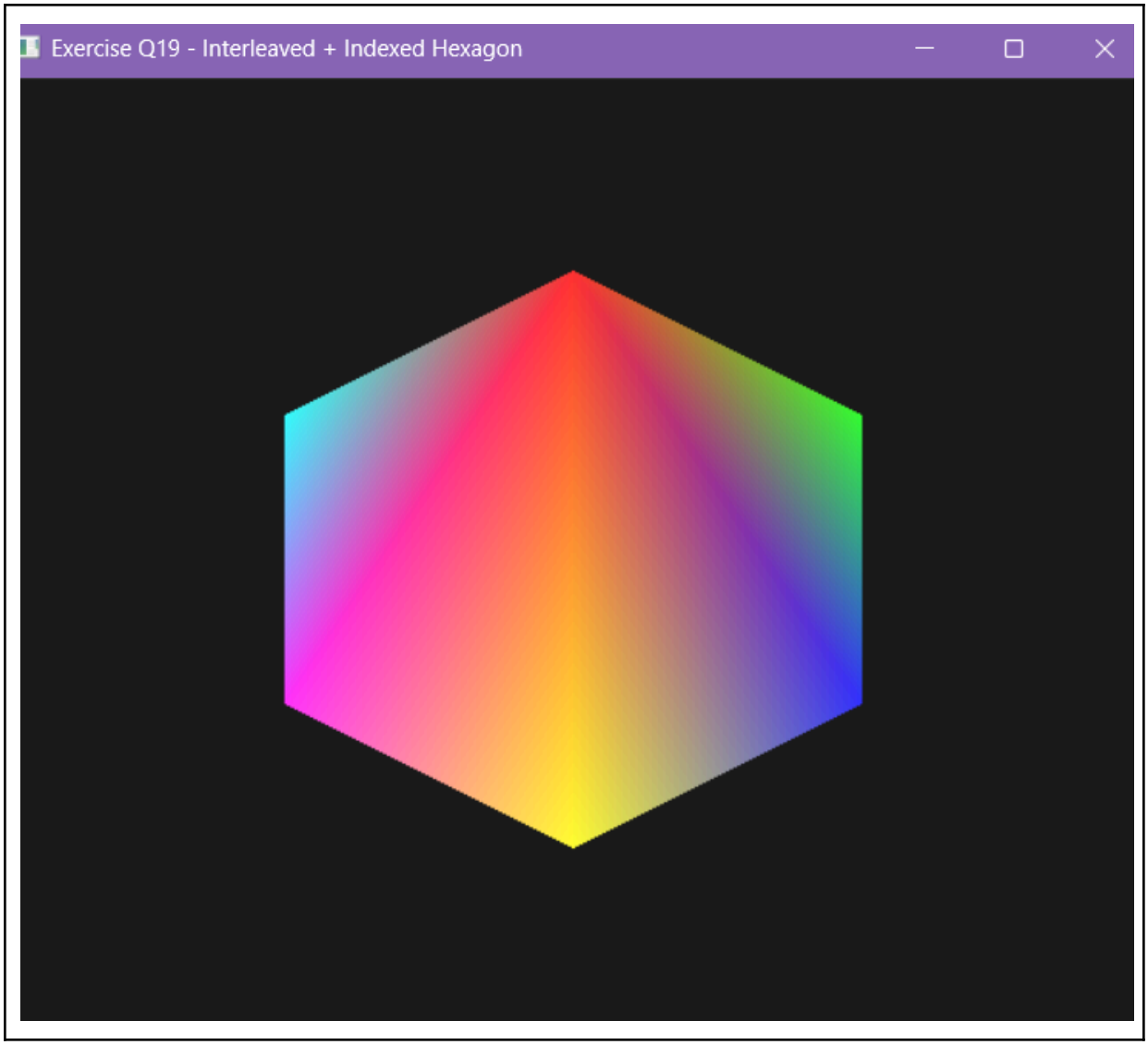
    // Render using glDrawElements and GL_POLYGON
    glDrawElements(GL_POLYGON, 6, GL_UNSIGNED_BYTE, hexagonIndices);

    glDisableClientState(GL_VERTEX_ARRAY);
    glDisableClientState(GL_COLOR_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 500);
    glutCreateWindow("Exercise Q19 - Interleaved + Indexed Hexagon");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:



Exercise Q20 — Capstone: Procedural Flower Scene [Hard]

Objective: Practice combining procedural generation, indexed rendering, color arrays, and multi-shape composition into one program.

Task (Question): Build a small procedurally generated 'flower' scene: a center disc (GL_TRIANGLE_FAN via glDrawElements), a ring of colored petals (GL_TRIANGLES via glDrawArrays with a vertex array + color array, generated in a loop, not hand-typed), and a ground quad.

Hints:

- Generate the petals' 3 vertices per petal (center point + two outer corners) inside a loop that walks around a circle by angle.
- Draw the petals FIRST, then the disc on top, so the disc doesn't get hidden behind petal geometry near the center.

CODE:

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <vector>

using namespace std;

// 1. Ground vertices
GLfloat groundVertices[] = {
    -1.0f, -1.0f, 0.0f,
     1.0f, -1.0f, 0.0f,
     1.0f, -0.2f, 0.0f,
    -1.0f, -0.2f, 0.0f
};

// Procedural Flower Data
const int NUM_PETALS = 8;
vector<GLfloat> petalVertices;
vector<GLfloat> petalColors;

const int DISC_SEGMENTS = 20;
vector<GLfloat> discVertices;
vector<GLubyte> discIndices;

void generateFlowerData() {
    float flowerCenterX = 0.0f;
    float flowerCenterY = 0.2f;

    // Generate Petals (GL_TRIANGLES, 3 vertices per petal: center + 2 outer
corners)[cite: 19]
    for (int i = 0; i < NUM_PETALS; i++) {
        float angle1 = 2.0f * 3.14159265359f * float(i) / float(NUM_PETALS);
```

```
        float angle2 = 2.0f * 3.14159265359f * float(i + 0.8f) /
float(NUM_PETALS);

        // Center point of petal
        petalVertices.push_back(flowerCenterX);
        petalVertices.push_back(flowerCenterY);
        petalVertices.push_back(0.0f);

        // Outer corner 1
        petalVertices.push_back(flowerCenterX + 0.4f * cosf(angle1));
        petalVertices.push_back(flowerCenterY + 0.4f * sinf(angle1));
        petalVertices.push_back(0.0f);

        // Outer corner 2
        petalVertices.push_back(flowerCenterX + 0.4f * cosf(angle2));
        petalVertices.push_back(flowerCenterY + 0.4f * sinf(angle2));
        petalVertices.push_back(0.0f);

        // Petal Colors (Pink/Magenta variations)
        for (int v = 0; v < 3; v++) {
            petalColors.push_back(0.9f);
            petalColors.push_back(0.3f + 0.05f * i);
            petalColors.push_back(0.6f);
        }
    }

    // Generate Center Disc (GL_TRIANGLE_FAN via glDrawElements)[cite: 19]
    discVertices.push_back(flowerCenterX);
    discVertices.push_back(flowerCenterY);
    discVertices.push_back(0.0f);

    for (int i = 0; i <= DISC_SEGMENTS; i++) {
        float theta = 2.0f * 3.14159265359f * float(i) / float(DISC_SEGMENTS);
        discVertices.push_back(flowerCenterX + 0.15f * cosf(theta));
        discVertices.push_back(flowerCenterY + 0.15f * sinf(theta));
        discVertices.push_back(0.0f);
    }

    discIndices.push_back(0);
```

```
    for (int i = 1; i <= DISC_SEGMENTS + 1; i++) {
        discIndices.push_back(i);
    }
}

void display() {
    glClearColor(0.5f, 0.8f, 0.9f, 1.0f); // Sky blue background
    glClear(GL_COLOR_BUFFER_BIT);

    glEnableClientState(GL_VERTEX_ARRAY);

    // Draw Ground (Green)
    glColor3f(0.2f, 0.6f, 0.2f);
    glVertexPointer(3, GL_FLOAT, 0, groundVertices);
    glDrawArrays(GL_QUADS, 0, 4);

    // Draw Petals FIRST using vertex + color arrays[cite: 19]
    glEnableClientState(GL_COLOR_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, petalVertices.data());
    glColorPointer(3, GL_FLOAT, 0, petalColors.data());
    glDrawArrays(GL_TRIANGLES, 0, NUM_PETALS * 3);
    glDisableClientState(GL_COLOR_ARRAY);

    // Draw Center Disc ON TOP so it isn't hidden by petal geometry[cite: 19]
    glColor3f(1.0f, 0.9f, 0.1f); // Yellow center disc
    glVertexPointer(3, GL_FLOAT, 0, discVertices.data());
    glDrawElements(GL_TRIANGLE_FAN, discIndices.size(), GL_UNSIGNED_BYTE,
discIndices.data());

    glDisableClientState(GL_VERTEX_ARRAY);

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Exercise Q20 - Capstone: Procedural Flower Scene");
```

```
    generateFlowerData();

    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

OUTPUT:

